From Grassroots Contract to Grassroots App (Full Version)

JAMES GOLIKE, London School of Economics and Political Science, United Kingdom
ANDY LEWIS-PYE, London School of Economics and Political Science, United Kingdom
EHUD SHAPIRO, London School of Economics and Political Science, United Kingdom and Weizmann Institute of Science, Israel
NIMROD TALMON, Ben-Gurion University, Israel

Almost two centuries ago Pierre-Joseph Proudhon envisioned a social contract that is an agreement of man with man; reciprocal; imposing no obligation upon the parties except that which results from their personal promise; subject to no external authority; freely accepted and signed by all the participants; and of the nature of a contract of exchange. He also envisioned that groups formed by a like contract could, by a similar contract, agree with every and all other groups; digitally, that independent platform instances may coalesce by mutual consent. Here we define *grassroots contracts* as social contracts meeting Proudhon's six conditions and two more: people are free to deal with each other, and there is no external register of people. We show that a grassroots contract can be transformed from its natural language presentation into a working smartphone grassroots app through an abstraction cascade: the contract's text is rendered as *speech-act schemas*, first-order universally-quantified implications guarded by the persons who must will the act, each denoting volition-guarded transactions among agents, a person with their will and a machine with its state, verified syntactically to be grassroots by three decidable conditions, *Introduction*, *Provenance* and *Volition*. Speech-act schemas are a restricted surface syntax for volition-guarded multiagent atomic transactions, the cascade from which to a working app has already been described. We prove that the protocol realising a syntactically grassroots contract is grassroots, and volitionally so: two instances coalesce only when a member of each wills it. We distinguish three legal modalities of clauses, *enforced*, *attested* and *undertaken*, by whether an agent can breach a clause and what non-repudiable evidence of the breach the contract leaves, and illustrate them with a social network and with grassroots coins. Contracts are amended, extended, restricted, instantiated and implemented, and the check of an amended contract is confined to what it adds. A checker of the three conditions, written in Grassroots Logic Programs, accompanies the paper.

Additional Key Words and Phrases: grassroots contract, speech-act schema, compilation, legal modalities, provenance, contract specification language, norm-governed systems, contract formation

1 INTRODUCTION

Almost two centuries ago Pierre-Joseph Proudhon envisioned a social contract [65, Fourth Study] that is (1) an agreement of man with man; (2) reciprocal; (3) imposing no obligation upon the parties except that which results from their personal promise; (4) subject to no external authority; (5) freely accepted and signed by all the participants; (6) of the nature of a contract of exchange; and a few other conditions, among them that it include all citizens with their interests and that it increase the well-being and liberty of every citizen. In his words, the social contract is "an agreement of man with man"; "essentially reciprocal: it imposes no obligation upon the parties, except that which results from their personal promise of reciprocal delivery: it is not subject to any external authority: it alone forms the law between the parties: it awaits their initiative for its execution"; "freely discussed, individually accepted, signed with their own hands, by all the participants"; and "of the nature of a contract of exchange: not only does it leave the party free, it adds to his liberty; not only does it leave him all his goods, it adds to his property; it prescribes no labor; it bears only

Authors' addresses: James Golike, London School of Economics and Political Science, London, United Kingdom; Andy Lewis-Pye, London School of Economics and Political Science, London, United Kingdom; Ehud Shapiro, London School of Economics and Political Science, London, United Kingdom and Weizmann Institute of Science, Rehovot, Israel; Nimrod Talmon, Ben-Gurion University, Beer Sheba, Israel.

upon exchange". He also envisioned a property of social contracts [65, Sixth Study], analogous to a property of digital platforms we term *grassroots* [74, 76, 77]: that (*i*) one could "make a contract with all, as … with some"; digitally, that a grassroots platform can have multiple instances, and (*ii*) "each group of citizens … formed by a like contract … could thereafter, and always by a similar contract, agree with every and all other groups"; digitally, that independent platform instances may coalesce by mutual consent, possibly into a single global platform instance. In full: "if I could make a contract with all, as I can with some; if all could renew it among themselves, if each group of citizens, as a town, county, province, corporation, company, &c., formed by a like contract, and considered as a moral person, could thereafter, and always by a similar contract, agree with every and all other groups, it would be the same as if my own will were multiplied to infinity. I should be sure that the law thus made on all questions in the Republic, from millions of different initiatives, would never be anything but my law; and if this new order of things were called government, it would be my government." We proposed grassroots platforms owned, operated and governed by their participants as a foundation for an equitable and democratic digital realm [76].

A *digital social contract* is one realised by software the agents run [12]. We call the mathematical object a *protocol* and the software implementing it a *platform*. A protocol or a platform *realises* a contract, a contract being an informal text; one abstraction of the cascade *implements* another, and implementations compose, so each implements every abstraction above it [47, 72]. A protocol is *grassroots* when its instances are independent of one another and of every global resource but the net, and can always coalesce, possibly into a single global instance [74, 76, 77], and a *grassroots platform* is the software implementing a grassroots protocol. The grassroots notion is a formal property of a protocol, recalled in Section 4.

Here we define *grassroots contracts* as social contracts meeting Proudhon's six conditions above and that (7) people are free to deal with each other; and (8) there is no external register of people, so people can come to know each other either directly or through people they already know. Our main theorem is that the protocol realising such a contract, written as speech-act schemas and meeting three decidable conditions, is grassroots: any two agents may come into relation without anyone's permission, and two instances formed independently coexist and may later coalesce.

A social contract is a contract among those who adopt it, and a grassroots contract is a social contract that runs: it is realised by software its agents run on their own phones. Participating in a grassroots platform requires adopting the grassroots contract that defines it, as participating in a global platform requires adopting an End-User Licence Agreement (EULA). A grassroots contract is mutual and symmetric among all participants; an EULA is between each person and the platform operator. A friendship or a mutual credit line is an act under the contract between particular agents; a friendship under GSG is mutual trust, so its social graph is a web of trust. We show that GSN, a contract for a social network [75], is grassroots, and GC, a contract for personal coins [73]. Not every social contract is. Examples that are not include a contract without coalescences, not allowing two strangers to come into relation; a contract with named third-party control, e.g. where introduction requires a token only a third party can make; and a contract with an act that changes a stranger's state without their will and requires no prior relation between the two; each fails one of the conditions.

We show that a grassroots contract can be transformed into a working smartphone-based grassroots platform through an *abstraction cascade* [47], starting from converting the social contract text to formal speech-act schemas, verified syntactically to be grassroots, and then to volition-guarded multiagent atomic transactions [51], hitherto the most abstract formalism used to specify grassroots platforms. The abstraction cascade from there to a working app has been described elsewhere [47]. A grassroots contract has two artefacts: a legal text, and a protocol realising it. A clause of the text describes an *act*, in the sense of a juridical act: a declaration by one or more

agents intended to create a legal effect. Each agent takes it as a digital speech act [32]: a speech act [4, 70, 71] performed digitally, by signing a text with their private key, as GKK:1 says. An agent is a person and a machine; the person has a will, the machine a state. We wish to specify the transactions among agents, which change the machines' states and are guarded by the persons' wills. *Speech-act schemas* specify them: each is a first-order universally-quantified implication, its body what the act requires of the machines, its head what it changes, guarded by the persons who must will it, and denotes a set of volition-guarded transactions, with the agents determined upon signature. They are a restricted surface syntax for volition-guarded transactions [51], whose semantics is a transition system per set of agents, with correct runs, safe and live. The key property is grassroots, which a contract of speech-act schemas satisfies when it passes a syntactic check: it meets the eight conditions above, provided it is *syntactically grassroots*, satisfying: (*i*) *Introduction*, that some act can introduce any two willing agents, following which the state of each stores an identifier of the other, as of a person they know, a friend, or the issuer of a coin they hold, and we say that it *records* the other; (*ii*) *Provenance*, that the state of an agent stores an identifier of an agent only by an act with that agent, or with an agent whose state stores that identifier; and (*iii*) *Volition*, that an act requires the consent of an agent in each connected part of the undirected graph formed by identifiers stored by its agents, unless that graph is connected. Conditions (1), (2) and (4) hold of every contract written as speech-act schemas, since every participant in an act takes a role in it and the schemas single out no one, and so does (5), since nobody is bound who has not adopted the contract (Section 3.7); (3) is Volition, (6) and (7) are Introduction, and (8) is Provenance (Section 3.8). Introduction is of one clause relative to all, Provenance is over all clauses, and Volition is per clause, given Provenance. The three are decidable, and a checker for them accompanies this paper, at https://github.com/EShapiro2/GLP/tree/main/programs/gsc; it certifies both of the contracts we carry through. A schema denotes volition-guarded transactions, and a contract the protocol over them, defined for contracts meeting the conditions.

We prove that the protocol realising a syntactically grassroots contract is grassroots, and volitionally so (Theorem 15). To do so, we first prove that *politeness* — any two agents can always come to interact, and no act joins two instances unless each of them holds a guard of it — is sufficient for the protocol over a contained set of volition-guarded multiagent atomic transactions to be grassroots, and volitionally so: two instances coalesce only when a member of each wills it (Theorem 26). We then prove that a syntactically grassroots contract denotes a contained, polite set of volition-guarded transactions (Section 7).

We distinguish three legal modalities of the clauses of a grassroots contract (Section 10), by whether an agent can breach a clause and what record of it the contract leaves in another agent's state: a clause is *enforced*, if performing it is an act of the contract and every correct run of it keeps the clause; *attested*, if an agent may breach it, with the contract recording the breach; or *undertaken*, if the contract records who gave the undertaking, and performing it is the person's, not the machine's. A breach of an *enforced* clause is impossible with a correct realisation of the contract; a breach of an *attested* or *undertaken* clause can be taken to court, with non-repudiable evidence attributable to the obligor of the act alleged to constitute the breach, in the case of an attested clause. We illustrate the three modalities with the grassroots social network [75], and with grassroots coins [73]. The semantics and the conditions are algorithmic, so the only informal step in this paper is the first: rendering a legal text as schemas.

Contracts are written from other contracts, as GSN is from GSG and GSG from GKK. We define five relations between contracts (Section 9): *amendment*, which deletes and adds clauses, *extension*, also at a new argument of some predicates, and *instantiation* of a parametric contract, on their schemas, and *implementation* and *restriction*, on their runs. Restriction is undecidable, and we give syntactic conditions that imply it, a set we expect to grow. We prove what the check preserves: an

amendment of a certified contract is certified by checking the clauses it adds and the introductory act, one that strengthens the bodies of clauses and deletes clauses by checking the introductory act, and an instance of a parametric contract is certified with it. An undertaken clause may say what its obligor undertakes as a formula over runs, that a condition eventually leads to a response, kept or breached by the person and not by the machines (Section 10); and the language is Turing-complete, a counter being the multiplicity of an atom (Section 3).

We carry two contracts through, a social network and personal coins, and certify both from their schemas. A coin can be handed on, so a holding may record an agent its holder has never transacted with; the certification then rests on traceable provenance, and conservation of money is its special case.

Three languages. The paper employs three languages: natural language for the social contract; speech-act schemas; and volition-guarded atomic transactions. Rendering the first as the second is by judgement; the second denotes the third; and the conditions of the main theorem, that a syntactically grassroots contract is grassroots, are verified in the second. GSG:1, of the social graph (Section 2.2), reads "Alice and Bob, who trust each other and are not friends, may jointly decide to become friends". Its schema (Section 3.4) is a clause: its head is what the act adds and deletes, its body what it requires and forbids, the holder of each atom its first argument, and its guard names who must will it:

$$\begin{aligned} &\mathit{befriend}: \ \mathit{friend}(\mathit{Alice},\mathit{Bob}),\ \mathit{friend}(\mathit{Bob},\mathit{Alice}) \leftarrow \\ &\quad \mathit{identity}(\mathit{Alice}),\ \mathit{knows}(\mathit{Alice},\mathit{Bob}),\ \neg\mathit{friend}(\mathit{Alice},\mathit{Bob}), \\ &\quad \mathit{identity}(\mathit{Bob}),\ \mathit{knows}(\mathit{Bob},\mathit{Alice}),\ \neg\mathit{friend}(\mathit{Bob},\mathit{Alice}) \quad \text{guarded by } \mathit{Alice},\mathit{Bob}. \end{aligned}$$

It denotes (Section 3.3) the volition-guarded transaction $(d \rightarrow d', \{\mathit{Alice}, \mathit{Bob}\})$, for every d with $\mathit{identity}, \mathit{knows}(\mathit{Bob}) \in d_{\mathit{Alice}}$, $\mathit{identity}, \mathit{knows}(\mathit{Alice}) \in d_{\mathit{Bob}}$, $\mathit{friend}(\mathit{Bob}) \notin d_{\mathit{Alice}}$ and $\mathit{friend}(\mathit{Alice}) \notin d_{\mathit{Bob}}$, where $d'_{\mathit{Alice}} = d_{\mathit{Alice}} \uplus \{\mathit{friend}(\mathit{Bob})\}$ and $d'_{\mathit{Bob}} = d_{\mathit{Bob}} \uplus \{\mathit{friend}(\mathit{Alice})\}$. The rest of the paper concerns the middle line. **The abstraction cascade.** The passage from a legal text to a running platform goes through the *abstraction cascade* [47] set out in Figure 1; this paper provides (1)→(3). A compiler carries the grassroots social graph and grassroots currencies [73] from their volition-guarded transactions down the cascade to Grassroots Logic Programs that run with scripted persons, and earlier programs of the social graph, the grassroots social network [75] and the currencies run as mini-apps of one app, deployed on a physical smartphone [47]. The screens of Figures 2 to 5 are screenshots of the GKK, GSG, GSN and GC mini-apps, each compiled from its social contract down the abstraction cascade; one instance of the grassroots app on an iPhone simulator runs the mini-app of every agent, the agents communicate over channels within it, and a screenshot shows one person's screen. All source code and all artefacts, including the social contracts and the artefacts the abstraction cascade produces from them, are in the public repository https://github.com/EShapiro2/GLP. GAS, the GLP app store in the same repository, makes an app of every contract the repository records down to its platform, and runs it with each person's agent on their own device; GKK has run between a laptop and a phone near each other, each running its own agent, with no server between them. A child-safe social network [79] is specified the same way. **The Human–Science–AI methodology.** The formal objects of the cascade are its abstractions and the implementations among them, each constructed, maintained, evolved and proved correct with AI, in tandem with the paper describing it. The methodology has two kinds of artefact: papers written for competitive peer review, and code. The papers serve as an evolving and cooperatively-constructed interface between us humans and AI, recording our shared understanding of the subject matter, and are the source of authority for coding. Papers are written by people and reviewed, revised and extended with the help of AI; the code, with few exceptions, is written by AI; and both evolve in the process of harmonising them. AI codes down the cascade, from paper to tested code, and every gap

coding finds is repaired in the paper before it is repaired in the code. It is at the antipode of "vibe coding", where a person describes the intended behaviour informally and AI guesses the rest [67]. Spec-driven development makes the specification the primary artefact, maintained by people while the code is regenerated from it [26, 30]; here that artefact is a scientific paper, and the feedback from coding revises it. The AI is Claude, Anthropic's models, among them Claude Opus 5.5 and Claude Fable 5.1. Through Claude Code it wrote, each from the paper that specifies it, the checker and the compiler of speech-act schemas, the compilers of the cascade below them, the apps of Figures 2 to 5, GAS, and the tests that run the apps and draw their screens; and it revised and extended the text of this paper, its definitions and proofs included, at the authors' direction. **Relation to contract specification languages.** Formalising legal text has a large literature, on contracts as automata, as defeasible deontic rules and as domain-specific languages, and on statute as code, and Section 12 reviews it. Two languages formalise legal contracts in a way close enough to ours to say plainly how they differ. *Symboleo* [63, 85] specifies a contract as obligations and powers with statechart lifetimes, for verification and for monitoring; *Stipula* [16] is a domain-specific language whose contracts are state machines over parties, fields and linear assets, with an agreement operator and a formal semantics. Both bind their parties at contract formation and both keep one global contract state. Our question is about the family of systems a contract induces over every finite set of people, and about what happens when two instances that formed independently meet; it cannot be posed in either language, and each requires a global resource that closure forbids. Section 12 compares the two languages, together with the literature on norms and on contract languages more broadly.

Structure. Section 2 presents the contracts and Section 3 the syntax of speech-act schemas, with listed roles, by which a leader closes a group with all its members, and cleared atoms, by which ending a friendship ends every membership between the two, and the contracts' schemas. Section 4 recalls what is needed of [51] and Section 5 states the protocol a contract denotes. Section 6 defines politeness and proves it sufficient for a protocol to be grassroots and volitionally grassroots, and Section 7 states the conditions on the schemas and certifies the contract from them. Section 9 defines amendment, extension, instantiation, implementation and restriction of contracts and what the check preserves. Section 10 explains the legal modalities of contract clauses, Section 11 how an agent enters the contract and what the code they run warrants, and Section 12 reviews related work. Appendix A states the framework of [51] formally, citing its proofs; Appendix B holds the proofs; Appendix C defines the values speech-act schemas allow; and Appendix D expands the related work.

2 GRASSROOTS CONTRACTS

A grassroots contract is a finite list of clauses its agents undertake towards one another. A clause either describes an *act*, which the protocol realising the contract carries out, or binds the agents without describing one. Each contract is in a box of its own, which holds, just before its clauses, a declaration of the contract's names, saying what each names, as "Alice, Bob and Carol are persons."; the declaration is part of the contract, which the schemas and the pragmas take their names from. A contract is self-contained: a definition that is part of it is in the box, before the declaration, as GSG defines trust; a definition outside the box is a formal definition, which the box cites explicitly. Two names are two persons; a clause that admits one person in both writes "Alice and Bob, possibly one person". Formally, contract variables are universally quantified per clause. A clause is cited by the contract's name and its number, as GSG:1; a contract that extends another cites its clauses and does not restate them. A contract written from others says so in its first line, after its title, by the relations of Section 9: each relation is followed by the contract it is to, or by 'it' for the contract named last, a composition written 'composes A with B' or 'composes it with B', and named 'the

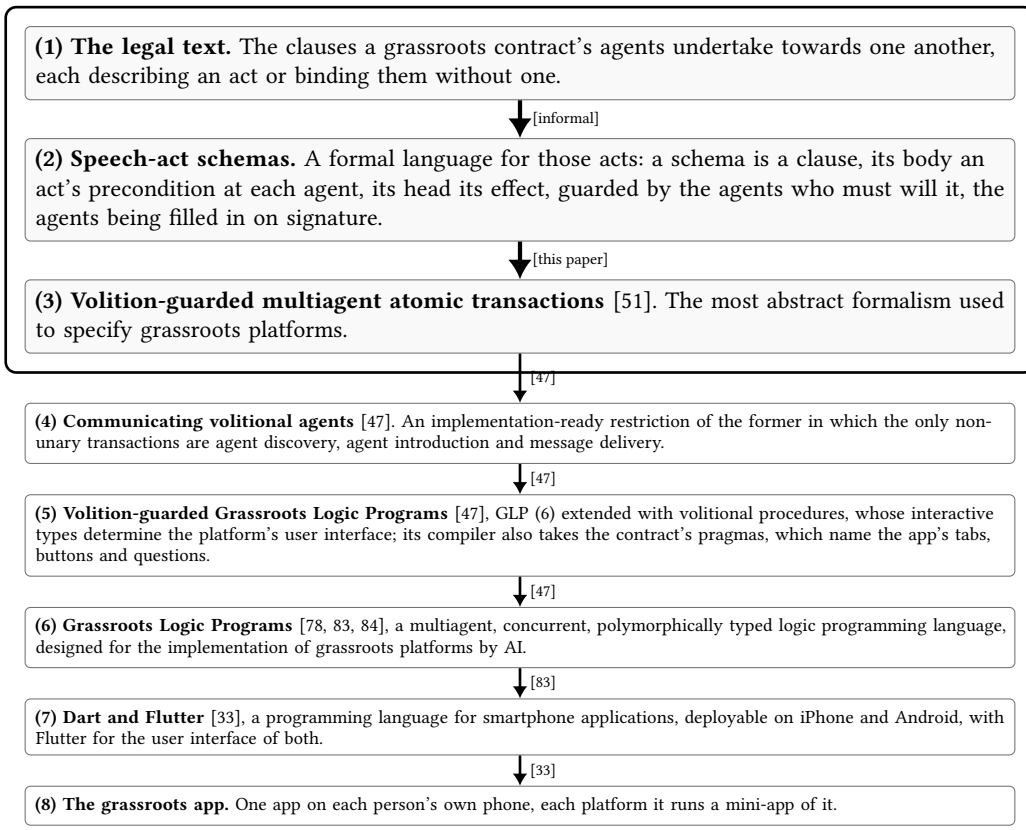


Fig. 1. The abstraction cascade, from a legal text to a running platform. The framed (1)–(3) are this paper. The first arrow, the realisation of the legal text, is left to judgement, and the second is the semantics of this paper. Each arrow from (3) to (6) is an implementation proved correct in the reference it carries, and they compose [47]; the GLP engine in Dart and the app are built and tested, and not proved. A condition on the transactions of (3), politeness, is sufficient for the platform they specify to be a grassroots platform, and conditions on the schemas of (2) are sufficient in turn for politeness.

composition' after it, the relations joined by 'and', and an instantiation may say after the contract's name, between commas, what it writes for what, as "CSSN instantiates GSNW, a child for a ward and a parent for a guardian, and restricts and extends it" [79]. A contract that extends another has the other's acts and its own, and an act of the other that its clauses change it replaces, keeping no act beside its replacement: GSN's unfriend, which also ends every membership between the two, replaces GSG's. Each clause opens with its legal modality, as the box below sets out (Section 10). A box ends with the contract's pragmas, which the app built from it follows, of two kinds: *interface pragmas*, the tabs, buttons and questions of the app, and *implementation pragmas*, what the app does beyond its screens, as the checks it makes before installing a program; each kind under its own title, and the title Pragmas alone means interface pragmas. An implementation pragma may declare that a person's act stands for every act of a clause that the person guards: "Encounter: Alice stands on [Open place] until [Close place]" says that Alice's act Open place is her standing request of every act of Encounter she guards, so that she is asked nothing, and that her act Close place withdraws it [47]. The interface pragmas may open with a line naming the app's look, its icon, an

emoji, and its family, as "Look: *emoji*, network", the colours of each family being one file of the repository beside the contracts, so that what an app shows comes from its contract; then a line for each tab of the app, naming the tab and what it lists. Each clause then has a line naming it, then the person who proposes an act of it, a button for each act they propose, in brackets, bearing its act's name, and any field they fill, in angle brackets; a line for each other person who proposes an act of the clause, written the same way; and a line for each other person whose will the act needs, their name, a colon, and the question they are asked. A button is offered for each way the proposer's own state satisfies the atoms of the act at the proposer, and a field is for a variable of the act those atoms do not bind, a text, a number, a new name, a person not yet known; so GKK's Acquaintance has a field, the person, whom Alice does not yet know, and its Introduction none, Alice knowing both Bob and Carol. A question line belongs to the nearest button line above it, so where a proposer's acts ask questions each act takes a line, and buttons share a line only when no question follows. A name in a pragma is the schema's role of that name, the schemas naming their roles as the clauses do, a listed role by the variable its quantifier binds. The contracts of this section are examples: the syntax of Section 3, the compilation of Section 5 and the conditions of Section 7 apply to any contract written the same way.

The clauses are stated here in prose, as a contract states them. The sections that follow give the acts a formal syntax and semantics, and certify the contracts from the schemas.

Legal Modalities of Contract Clauses

■ **Enforced:** no act of the contract can breach it.
◆ **Attested:** an agent may breach it, and the contract records the breach.
○ **Undertaken:** its performance is the person's, and the contract records the undertaking.

An agent running correct code, code that correctly implements the contract, cannot breach an enforced clause.

2.1 Identity and Keys

A *person* is a natural person of age. A person takes the pronoun 'they'; for readability we may write 'she' for Alice and 'he' for Bob etc. Alice, Bob and Carol are variables naming distinct persons. A person *knows* another when they hold the other's public key. Note that knowing is not a symmetric relation; a directed edge in the agent's knowledge graph.

GKK: Grassroots Keys and Knowledge

Alice, Bob and Carol are persons.

1. ○ *Identity.* Alice may establish her identity once by choosing a unique key pair. She makes herself known by the public key, signs her acts with the private key, and is accountable for every act signed with it.
2. ■ *Acquaintance.* Alice and Bob may come to know each other by an act of both.
3. ■ *Introduction.* If Alice and Bob know each other and Alice knows Carol, then Alice can make Carol known to Bob, with Bob's consent.

Pragmas

Contacts: those Alice knows.
Identity: `Alice [Establish]`
Acquaintance: `Alice [Acquaint] <person>`



Fig. 2. The GKK app: Alice, before establishing her identity, with the button Establish (GKK:1); Bob's contacts after Alice makes Carol known to him, each with the button Introduce (GKK:3), and Acquaint with its field (GKK:2).

```
   Bob: Alice wishes that you know each other.
Introduction: Alice [Introduce]
   Bob: Alice asks to introduce Carol to you.
```

GKK is the base contract, which every contract below extends: a person comes to know another by an act of both or through someone they know, which is condition (8) of Section 1. A key pair chosen at random is unique with high probability, and [18] replaces one that is lost or stolen. The GKK app is a mini-app of the grassroots app, compiled down the cascade from GKK; Figure 2 shows it realising GKK:1 and GKK:3.

2.2 Social Graph

Note that friendship is a symmetric relation; an undirected edge in the agent's social graph.

GSG: Grassroots Social Graph

GSG extends GKK.
Alice *trusts* Bob when she knows Bob personally and believes him to keep the undertakings he signs under grassroots contracts.
Alice and Bob are persons.

1. ◆ *Friendship.* Alice and Bob, who trust each other and are not friends, may jointly decide to become friends.
2. ■ *Ending a friendship.* Alice may end her friendship with Bob.
3. ○ *Loss of Trust.* Alice ends her friendship with Bob when she ceases to trust him.

Pragmas

Friends: the friends of Alice.
Friendship: `Alice [Befriend]`
`   Bob: Alice asks to be your friend.`
Ending a friendship: `Alice [Unfriend]`



Fig. 3. The GSG app: Alice asks to be Bob's friend, and Bob answers yes or no (GSG:1); Alice's friends after Bob says yes, Bob's row with the buttons Introduce (GKK:3) and Unfriend (GSG:2).

GSG extends GKK and is the grassroots social graph [75]. Trust is a belief, so once Alice knows that Bob breached an undertaking, in particular a breach of an attested clause, of which the contract holds the evidence, she can no longer trust him, and by GSG:3 she must end the friendship. A friendship is mutual trust on record, so the social graph is a web of trust [94, 95]. The GSG app is a mini-app of the grassroots app, compiled down the cascade from GSG; Figure 3 shows it realising GSG:1.

2.3 Forwarding with Provenance

An *item* is a speech act a person holds, created by signing it or received signed over by the friend who forwarded it; the signatures an item carries are its provenance, and its author is its first signer.

GFP: Grassroots Forwarding with Provenance

GFP extends GSG.
The author of an item is its first signer.
Alice and Bob are persons.

1. ■ *Signing.* Alice may create an item by signing it.
2. ■ *Forwarding.* Alice may forward an item she holds to her friend Bob, adding her signature to the signatures the item carries; Bob sees them.

Pragmas

Items: the items Alice holds, each with its signatures.
Signing: `Alice [Sign] <text>`
Forwarding: `Alice [Forward]`

GFP extends GSG with items that carry their provenance: an item forwarded along a chain of friends arrives with the signature of each, the author's inside. The dissemination of programs by the grassroots app, in preparation, extends it.

2.4 Social Network

GSN: Grassroots Social Network

GSN amends GSG.
Alice and Bob are persons.

1. ■ *Messaging.* Alice may send a message to her friend Bob.
2. ■ *Group.* Alice may create a group, of which she is the leader and first member; Alice may close the group by leaving it.
3. ■ *Joining and Leaving.* Alice, the leader of a group, may add her friend Bob to it, with Bob's consent; either Alice or Bob may then end Bob's membership in the group, which also ends when Alice and Bob cease to be friends.
4. ■ *Group messaging.* A member of a group may send a message to the group, and its leader delivers it to every current member.

Pragmas

Groups: the groups Alice is a member of.
Messages: the messages Alice received.
Posts: the posts Alice received in her groups.
Messaging: `Alice [Message] <text>`
Group: `Alice [Create] <name> [Close]`
Joining and Leaving: `Alice [Join]`

```
Bob: Alice asks you to join her group.
Alice [Remove]
Bob [Leave]
```

Group messaging: `Bob [Post] <text>`

```
Alice [Post own] <text>
```

GSN extends GSG with messages and groups [75], and the paper carries it through. The GSN app is a mini-app of the grassroots app, compiled down the cascade from GSN; Figure 4 shows it realising GSN:2, GSN:3 and GSN:4. The accountability in GKK:1, the trust GSG:1 requires and GSG:3 bind the persons and describe no act of the contract; the other clauses describe acts and determine the contract's transactions. Section 10 says what a realisation does with a clause that describes no act. In the repository GSN says 'agent' for 'person', so that children [79] and AI agents [80] can be added to it.

2.5 Guardians and Wards

A *ward* is an agent who acts with a guardian: a child with their parents, an AI agent with the person operating it, an organisation with its officers.

GnW: Guardians 'n' Wards

GnW amends GSG.
A *relation* is a friendship or the membership of a group. Bob trusts Carol, a ward of Alice, when he trusts Alice.
Alice, Bob and Erin are persons, Carol and Dave wards.

1. ■ *Creation.* Alice may create a *ward*, Carol, and establish Carol's identity by choosing Carol's key pair. Alice is then the *creator*, a *guardian* and a friend of Carol, a friendship Carol may not end.

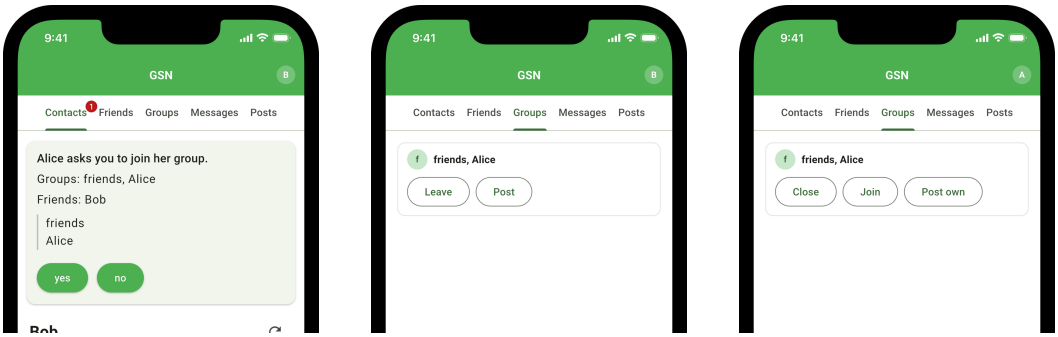


Fig. 4. The GSN app: on Bob's phone, Alice asks Bob to join her group *friends*, and Bob answers yes or no (GSN:3); Bob's Groups tab once he has joined, the group *friends*, led by Alice, with the buttons Leave and Post (GSN:3, GSN:4), the posts delivered to him being in his Posts tab; on Alice's phone, the group she leads, with the buttons of the leader's acts, Close, Join and Post own (GSN:2 to GSN:4).

2. ■ *Joint guardianship.* Bob, a friend of a guardian of Carol, may become a guardian and a friend of Carol by an act of Bob and the guardians of Carol; Bob's guardianship ends by an act of Bob, by an act of the other guardians of Carol, or when Bob ends his friendship with Carol. Carol may not end a friendship with her guardian.
3. ■ *Ward and person.* Carol, a ward of Alice, and Bob may come to know each other by an act of both. Carol enters a relation with Bob provided they know each other and Alice and Bob are friends, by an act of Carol, Alice and Bob. Carol, and the agent in a relation with Carol, may end it, as may Alice.
4. ■ *Ward and ward.* Carol, a ward of Alice, and Dave, a ward of Bob, may come to know each other by an act of both. Carol enters a relation with Dave provided they know each other, Alice knows Dave and is a friend of Bob, by an act of Carol, Alice, Dave and Bob, Alice and Bob possibly one person.
5. ■ *Retiring.* The guardians of Carol may retire Carol; every guardianship and relation of Carol then ends and Carol takes no act.

Pragmas

Wards: the wards Alice is a guardian of.
Creation: `Alice [Enter] <name>`
`Carol: Alice asks to be your guardian.`
Joint guardianship: `Alice [Make guardian]`
`Bob: Alice asks you to be a guardian of her ward Carol.`
`Erin: Alice asks that Bob be a guardian of your ward Carol.`
`Alice [End guardian] <person>`
`Erin: Alice asks to end Bob's guardianship of your ward Carol.`
`Bob [Resign] [Guardian unfriend]`
Ward and person: `Bob [Ward acquaint] <person>`
`Carol: Bob wishes that you know each other.`
`Alice [Ward befriend]`
`Carol: Alice asks that you and Bob befriend.`
`Bob: Alice asks that you and her ward Carol befriend.`

```
   Alice [Ward unfriend] <person>
   Carol [Ward end]
Ward and ward: Carol [Wards acquaint] <person>
   Dave: Carol wishes that you know each other.
   Alice [Wards befriend]
   Carol: Alice asks that you and Dave befriend.
   Dave: Alice asks that you and Carol befriend.
   Bob: Alice asks that Carol and your ward Dave befriend.
Retiring: Alice [Retire]
   Erin: Alice asks to retire your ward Carol.
```

GnW amends GSG: the acts of GKK and GSG by which two agents come to know each other and befriend are for persons, and a ward comes to know an agent, and befriends one, by the acts of GnW:3 and GnW:4. A ward is created with a guardian, takes no act without one, and may be retired by her guardians, after which she takes no act. GnW is parametric in its predicates of wards (Section 9), and three contracts instantiate it, each with wards of its own kind: the child-safe social network [79], whose wards are children and their guardians parents; accountable AI agents [80], whose wards are AI agents and their guardians the persons operating them; and a contract for grassroots organisations, in preparation, whose wards are organisations and their guardians officers.

2.6 Coins

A *coin* is a unit of debt naming its issuer, and an *Alice-coin* is a coin issued by Alice.

GC: Grassroots Coins [73]

GC extends GKK.
Alice and Bob are persons.

1. ■ *Minting.* Alice may decide to issue Alice-coins.
2. ■ *Swap.* Alice and Bob, who know each other, may jointly decide to swap coins they hold.
3. ■ *Payment.* Alice may decide to pay Bob-coins she holds to Bob, whom she knows.
4. ■ *Redemption.* Alice may decide to redeem a Bob-coin she holds against any coin held by Bob, whom she knows.
5. ○ *Pricing.* Alice prices her offerings in Alice-coins and accepts them in payment at those prices.

Pragmas

Coins: the coins Alice holds.

```
Minting: Alice [Mint] <amount>
Swap: Alice [Swap] <amount given> <issuer wanted> <amount wanted>
   Bob: Alice asks to swap coins with you.
Payment: Alice [Pay] <amount>
Redemption: Alice [Redeem] <issuer wanted>
```

GC is the grassroots coins contract [73], quoted from its owner's copy; it extends GKK and not GSG, no act of it requiring a friend, and the paper carries it through beside GSN. The first four clauses describe acts and determine the contract's transactions; a swap in which Alice gives an Alice-coin and Bob gives a Bob-coin opens a *mutual credit line* between them, each thereafter



Fig. 5. The GC app: Alice's holdings after minting an Alice-coin (GC:1); Bob's after swapping a Bob-coin for Alice's Alice-coin (GC:2), the coin with the button Pay (GC:3).

holding a coin of the other. The fifth describes no act: it gives a coin its worth. The GC app is a mini-app of the grassroots app, compiled down the cascade from GC; Figure 5 shows it realising GC:1 and GC:2.

3 SPEECH-ACT SCHEMAS

A clause of a contract describes an *act*, in the sense of a juridical act: a declaration by one or more agents intended to create a legal effect. Each agent takes it as a digital speech act [32]: a speech act [4, 70, 71] performed digitally, by signing a text with their private key, as GKK:1 says. This section defines their formal syntax and semantics. An agent is a person and a machine; the person has a will, the machine a state, here a multiset of atoms. Speech-act schemas specify the transactions among agents: each is a first-order universally-quantified implication, *Head* ← *Body* guarded by the persons who must will the act, its body what the act requires of the machines, its head what it changes. A *contract* C is a finite set of speech-act schemas over a set Σ of predicates, and S_C is its set of states.

3.1 Atoms and States

Write Π for a countable, potentially infinite set of at least two *agents*, of which every contract and every run concerns a finite subset, and Δ for a countable set of *utterances* disjoint from it: a message, a post or an item, taken as an identity and carrying no signature. A *speech act* is an utterance or a speech act, signed by an agent: D is the least set with $\Delta \times \Pi \subseteq D$ and $D \times \Pi \subseteq D$, and the *signer* of $(x, p) \in D$ is $\mathrm{signer}(x, p) := p$ [12, 32]; a speech act signed again carries its earlier signers inside it, and $D_P := \{d \in D : \mathrm{signer}(d) \in P\}$ for $P \subseteq \Pi$. A *community name* is formed from an agent and speech acts: the set $\mathcal{N}$ of community names is disjoint from $\Pi \cup D$ and generated freely by an injection $p \mapsto \langle p \rangle$ of Π into it and an injection $(v, y) \mapsto v \cdot y$ of $\mathcal{N} \times D$ into it with disjoint images; the *agents* of a name are $\mathrm{ppl}(\langle p \rangle) := \{p\}$ and $\mathrm{ppl}(v \cdot y) := \mathrm{ppl}(v) \cup \{\mathrm{signer}(y)\}$, and $\mathcal{N}_P := \{v \in \mathcal{N} : \mathrm{ppl}(v) \subseteq P\}$. Write Σ for a set of *predicate symbols*, *predicates* for short, and $\alpha : \Sigma \to \mathbb{N}$ for their arities; a predicate of arity n applied to n arguments is an atom, a syntactic object with no truth value. The arguments of an atom of a state are agents, speech acts and community names; those of an atom of a schema are agent variables, speech-act variables and name terms, and its first argument is its *holder*. An atom *names* what is among its arguments, the signer of a speech

act among them and the agents of a community name among them: in a state an agent, and in a schema an agent variable, a name term naming its base and the speech-act variables in it.

Definition 1 (Atoms and states). For any set W of arguments put

$$\Sigma[W] \;=\; \{\, e(u_1, \ldots, u_{\alpha(e)}) \;:\; e \in \Sigma,\; u_i \in W \,\},$$

call its members the *atoms over* W, and call $u_1, \ldots, u_{\alpha(e)}$ the *arguments* and e the *predicate* of $e(u_1, \ldots, u_{\alpha(e)})$. An atom over $\Pi \cup D \cup \mathcal{N}$ is *ground* , a *local state* is a finite multiset of ground atoms, the initial state is $s_0 = \emptyset$, and

$$S_C(P) \;=\; \{\, s \;:\; s \text{ a finite multiset over } \Sigma[P \cup D_P \cup \mathcal{N}_P] \,\} \qquad (P \subset \Pi).$$

Three predicates carry the example through this section: *identity*, of arity zero, and *knows* and *friend*, of arity one. After Alice establishes an identity, comes to know Bob and befriends him, the state of Alice is the multiset $\{identity, knows(Bob), friend(Bob)\}$, which lies in $S_C(P)$ for every P holding Bob and in none omitting him.

Multiset operations are written $\uplus$, $\setminus$, $\cap$ and $\subseteq$: multiplicities are added, subtracted, minimised and compared. A set is a multiset in which every multiplicity is one. The domain of the semantics is multisets: a machine configuration is read as a structure in which each predicate, its holder first, denotes a multiset of tuples, the multiplicity of a tuple being the number of copies of the atom that its holder holds. An atom of a schema with multiplicity κ holds at a binding where its image is held κ times or more, and its negation where fewer; a quantifier holds as it does over the tuples of positive multiplicity; and a count or a sum over the instances of a quantifier takes each instance as many times as its multiplicity (Definition 6, Appendix C), as the query languages for multisets evaluate aggregates, each copy counting [53].

A state is local, so its holder is no argument of its atoms: $friend(Bob)$ in the state of Alice says that Bob is a friend of Alice, and $¢(u)$ in the state of Alice is a coin issued by u and held by Alice; a friend set is a set of the former and a holding a multiset of the latter. A schema writes the holder first: its atom $friend(Alice, Bob)$ stands for $friend(Bob)$ in the state of the agent bound to *Alice*, and a predicate of arity n in a state has arity $n + 1$ in a schema.

3.2 The Syntax of a Schema

Fix a set X of *agent variables* disjoint from Π and a set Y of *speech-act variables* disjoint from D. A schema is a clause over atoms whose arguments are agent variables, speech-act variables and name terms, the first argument of each its holder; a *literal* is an atom or a negated atom $\neg\varphi$.

A schema is a contract template: its agent variables are its parameters, and the identities of the agents assuming them are filled in on signature. The guard names the agents the machine asks [12].

For example, take the predicates *identity*, of arity zero in a state, and *knows* and *friend*, of arity one. GSG:1, that Alice and Bob, who trust each other and are not friends, may jointly decide to become friends, is the schema

$$\begin{aligned} befriend :\; & friend(Alice, Bob),\; friend(Bob, Alice) \leftarrow \\ & identity(Alice),\; knows(Alice, Bob),\; \neg friend(Alice, Bob), \\ & identity(Bob),\; knows(Bob, Alice),\; \neg friend(Bob, Alice) \quad \text{guarded by } Alice, Bob. \end{aligned}$$

Read it holder by holder. Alice must hold *identity*, an identity established under GKK:1, and $knows(Bob)$, held after an acquaintance under GKK:2 or an introduction under GKK:3, and must not hold $friend(Bob)$; the head adds $friend(Bob)$ to her state, which is the effect of befriending; and the same holds of Bob with the two exchanged. Both are guards, so the act takes the will of both. Nothing is deleted, and both agents change state.

Closing a group is one act of its leader and of every member. The body and the head of a schema may have a *quantified conjunct*, $\forall \bar{x}\ (A \wedge \varphi \rightarrow \psi)$: for every value of the variables $\bar{x}$ under which an agent holds the atom A and φ holds, ψ holds, in the body, or is made to hold, in the head, where ψ is a conjunction of literals. In the body of close, "every Bob that Alice lists in the group is a member", $\forall Bob\,(listed(Alice, Bob, n) \rightarrow member(Bob, \langle Alice\rangle \cdot n))$, and in its head, "every Bob that Alice lists is no longer a member", $\forall Bob\,(listed(Alice, Bob, n) \rightarrow \neg member(Bob, \langle Alice\rangle \cdot n))$; Bob, quantified and holding the literals, is a *listed role* of the schema. A quantified conjunct may also be a condition on one agent's own state, as a ride-hailing contract in preparation awards a ride to Bob only if every bid Alice holds on the ride scores no more than Bob's, $\forall x, p', d'\ (bid(Alice, x, r, p', d') \rightarrow W(d') - p' \leq W(d) - p)$, which makes no listed role. Any first-order formula may serve as a condition in the body, provided it is *safe*: every variable a quantifier binds ranges over the arguments of an atom an agent holds, as in the safe queries of the relational calculus [1, 15], so the formula is evaluated over the agents' finite states. A *name term* names a community, such as a group. So the full power of universally quantified first-order logic is used: a schema is an implication, not a clause, close quantifying in its body and in its head:

$$\begin{aligned} close:\ & closed(Alice, n),\ \neg listed(Alice, Alice, n),\ \neg member(Alice, \langle Alice\rangle \cdot n),\\ & \quad \forall Bob\,(listed(Alice, Bob, n) \rightarrow \neg member(Bob, \langle Alice\rangle \cdot n)) \leftarrow\\ & member(Alice, \langle Alice\rangle \cdot n),\ listed(Alice, Alice, n),\\ & \forall Bob\,(listed(Alice, Bob, n) \rightarrow member(Bob, \langle Alice\rangle \cdot n)) \quad \text{guarded by } Alice. \end{aligned}$$

Definition 2 (Safe formula, name term, role). A *name term* of a schema is $\langle\pi\rangle$ for an agent variable π of it, or $\sigma \cdot y$ for a name term σ of it and a speech-act variable y of it or the wildcard _, and its *base* is π and the base of σ respectively. A *safe formula* of a schema is built from atoms, the first argument of each its holder, and constraints (Appendix C) by $\wedge$, $\vee$ and $\neg$ and by the anchored quantifiers $\exists \bar{x}\ (A \wedge \varphi)$ and $\forall \bar{x}\ (A \wedge \varphi \rightarrow \psi)$, φ and ψ safe formulas and A an atom, the quantifier's *anchor*, among whose arguments, or in name terms among them, every variable of $\bar{x}$ occurs. A *quantified conjunct* is a safe formula $\forall \bar{x}\ (A \wedge \varphi \rightarrow L_1 \wedge \cdots \wedge L_m)$, $m \geq 0$ and each L_j a literal. An *agent role* of a schema is an agent variable of it that no quantifier binds and that holds an atom of it or is a guard. An agent variable bound by a quantifier whose anchor is held by an agent role, and holding an atom of the schema, is a *listed role*, every quantifier binding it having the same anchor, whose holder is the listed role's *base*; it is written $\pi[A]$, π its base and the bound variables replaced by _ in A. A *role* is an agent role or a listed role, and the base of an agent role is itself.

A group of GSN (Section 2.4) is named $\langle Alice\rangle \cdot n$, by its leader Alice and a speech act n Alice signs, and Alice is its one agent. Closing it has two roles: the agent role *Alice*, its leader, and the listed role $Alice[listed(_, n)]$, every agent Alice lists as a member of the group n, the *Bob* of the two quantified conjuncts above.

Definition 3 (Speech-act schema). A *speech-act schema* is written

$$\tau:\quad H \leftarrow B \quad \text{guarded by } G,$$

with τ its name, B, its *body*, a finite multiset of literals and safe formulas, and H, its *head*, a finite multiset of literals and quantified conjuncts, over atoms whose arguments are agent variables, speech-act variables, name terms, the wildcard _ and, in H, *signed terms* $\mathrm{sgn}(y, \pi)$, y a speech-act variable that occurs in an atom an agent role requires and π an agent role, the speech act y signed by the agent in π, every agent variable a quantifier binds that holds an atom being a listed role, and G a set of agent variables of it, its *guards*. A wildcard in an unnegated atom is a variable of its own, one per occurrence; in a negated atom it is universally quantified, the atom forbidding, in B, and deleting, in H, every atom that matches it, and a negated atom of H with a wildcard

is *cleared*. Its *roles* $\pi_1, \ldots, \pi_k$, $k \geq 1$ its *arity*, are its agent roles and its listed roles, a role being a *guarding role* if it is a guard, and its *variables* are its agent variables that are not roles and its speech-act variables that no conjunct quantifies. An agent role *requires* the atoms of B it holds and *forbids* those negated in B there; it *adds* the atoms of H it holds, *deletes* those negated in H there with no wildcard, and *clears* those with one; a listed role does the same by the literals it holds in the conclusions of the quantified conjuncts of B and H that bind it, their other variables read as variables of the schema. Each such atom is written without its holder, and for the role π_i the five multisets are written $=(i)$, $\neg(i)$, $+(i)$, $-(i)$, $_(i)$. The literals an agent role holds in a quantified conjunct of H that binds no role it is are *conditional*: the role adds, deletes or clears them at each instance of the conjunct (Definition 7), and they are in none of its five multisets; the safe formulas of B are *conditions* on the agents' states, which the denotation reads and the three conditions read only as stated; where Definitions 11 and 12 and the guarded predicates of Section 10 speak of an atom a schema adds or deletes, a conditional one is included, and a listed role's at its conjunct; for both, the anchor of the conjunct counts as an atom its holder requires, and the conjunct's other variables as variables of the schema. It is required that $-(i) \subseteq =(i)$, as a deleted atom is held, and that $+(i) \uplus -(i) \neq \emptyset$ at some role, as a transaction changes a state [51]; a role that is not a guarding role and has $+(i) \uplus -(i) = \emptyset$ is a *reading role*.

The sentences of GSG:1 and its schema correspond: "may jointly decide" is the guard, "who trust each other" the two *knows* atoms of the body, trust presupposing that each knows the other and the rest of it describing no act, "are not friends" its two negated atoms, and "become friends" the head. The syntax is universally quantified first-order implications over multisets of atoms, with a guard, the body a safe formula, which with the cardinality of Appendix C is a formula of first-order logic with counting [52], and with its sum one that also sums, and nothing else: the wildcard is an anonymous variable, universally quantified in a negated atom, $\neg \mathit{listed}(\mathit{Alice}, \mathit{Bob}, _)$ being $\forall y \neg \mathit{listed}(\mathit{Alice}, \mathit{Bob}, y)$; a listed role, and a condition on one agent's state, is a quantified conjunct; and a multiplicity $k \cdot \varphi$ (Appendix C) is k copies of φ. The definitions restrict this syntax only where a proof below uses the restriction, and say which. What is not first-order logic is the arrow, which is read as a state change (Section 3.3).

At the role *Alice* of *befriend* the literals with holder *Alice* give $+(1) = \{\mathit{friend}(\mathit{Bob})\}$ and $\neg(1) = \{\mathit{friend}(\mathit{Bob})\}$, $=(1) = \{\mathit{identity}, \mathit{knows}(\mathit{Bob})\}$ and $-(1)$ empty, and at *Bob* the same with the two roles exchanged. Both roles guard, and $+(i) \uplus -(i) \neq \emptyset$ at each: befriending changes the state of both agents.

A guarding role may add and delete nothing; the agent in that role then wills the act, and their state is a precondition of it and does not change. A reading role neither wills the act nor changes state; the agent's state is a condition of the act, read by the others, so Definition 13 admits one in a schema with a guarding role only where a guarding role of the schema requires an atom naming it, as Alice, who points to Bob, who is connected to Carol, may decide to point to Carol, and in a schema with no guarding role only where the atoms it adds and deletes at one role, each the only one of its predicate, together name every other role, as Alice, who follows a feed through Bob, receives from Bob every item of it that Bob holds (Section 5.1). A cleared atom deletes every atom of the agent's state that it matches, and requires none: GSN's unfriend (Section 3.4) has $\neg \mathit{listed}(\mathit{Alice}, \mathit{Bob}, _)$ in its head, every membership of Bob in a group Alice leads, so that ending a friendship ends every membership between the two at once.

Speech-act schemas allow values of any type: an argument of an atom may be a value or a value variable, a schema may carry a constraint on its value variables, which every binding satisfies, and an atom of the head may carry a multiplicity, a count of copies and so a positive whole number, as

the bonds of [81] need for maturity dates and amounts (Appendix C). A value names no agent, so the conditions below do not concern it. A contract may also name *environment predicates*, such as the date, whose atoms are over values, which the machine holds from startup and the platform keeps current, and which no schema adds or deletes; a schema may require or forbid one (Appendix C).

Multiplicities make the language Turing-complete, with one agent and no guard.

Proposition 4 (Turing completeness). *For every two-counter machine M there is a syntactically grassroots contract C_M such that M halts from empty counters if and only if some correct run of the protocol realising C_M reaches a configuration in which an agent holds halt.*

The proof, in Appendix B, writes a counter as the multiplicity of an atom, incremented by adding a copy, decremented by deleting one and tested for zero by a negated body atom, and each instruction of M as a schema with no guard, which liveness takes.

A contract also carries pragmas, which the app built from it follows: its tabs, and for each act who proposes it, the button proposing it and the question the machine puts to each other agent in a guarding role.

Definition 5 (Binding). A *binding* β for a schema τ maps its agent variables and its speech-act variables that no conjunct quantifies to Π and D, distinct agent roles to distinct agents, but for two the schema marks as possibly one, which it may send to one agent if neither of them adds or deletes an atom naming the other, as Proposition 51 needs, and is extended to name terms by $\beta(\langle\pi\rangle) := \langle\beta(\pi)\rangle$ and $\beta(\sigma \cdot y) := \beta(\sigma) \cdot \beta(y)$, to signed terms by $\beta(\mathrm{sgn}(y, \pi)) := (\beta(y), \beta(\pi))$, to the wildcard by $\beta(_) := _$, and to atoms and to multisets of them componentwise. A ground atom *matches* $\beta(\varphi)$, for an atom φ with a wildcard, if it is obtained from $\beta(\varphi)$ by replacing each wildcard by an argument, a speech act where the wildcard is in a name term. It is *admissible* if every speech-act variable of τ that occurs in an atom added at a role, as an argument or in a name term, and in no atom required at a role is mapped to a speech act signed by the agent in a guarding role of τ.

A binding of *befriend* sends *Alice* to p and *Bob* to q, distinct because each role adds an atom naming the other, and carries $+(1)$ to $\{friend(q)\}$ and $+(2)$ to $\{friend(p)\}$. Befriending has no speech-act variable, so every binding of it is admissible.

Alice and Bob are two persons, and Alice is one person, in a clause and in its schema: distinct agent roles are bound to distinct agents, but for two a clause names as possibly one person, which its schema marks so, and a listed role and the role whose agent lists it, as close's two Alices, are carried out by that agent together, as Definition 7 states.

A speech-act schema is a STRIPS operator [23] in multiagent form: what a role requires and forbids is a precondition, what it adds and deletes are an add list and a delete list, and a schema carries one of each per role, where STRIPS has one precondition and one pair of lists over a single global state. The literals are also the four kinds of arc of a contextual net transition [61]: an atom required and deleted is consumed, one required and kept is a positive context condition, one forbidden is a negative context condition, and one added is produced. We take no result from either theory.

3.3 Semantics

A schema denotes a set of volition-guarded transactions [51], and speech-act schemas are a restricted surface syntax for them; the denotation is defined from the schema over the states of [51]. An *agent* is a person and a machine. A *machine state* is a local state, the atoms the machine holds; a *volitional state* is the person's, the set of classes of acts they are willing their machine to take part in, a class

being an act with its participants (Definition 16); an *agent configuration* over P gives each agent of P its two states, and the initial configuration has every machine empty and every person willing nothing. A schema denotes a relation on agent configurations, read off its literals and quantified conjuncts: at a binding β of its variables, it relates c to c' when, at c, every atom of its body holds at the machine of the agent in its holder's role and no negated atom of its body does, the person of each guard holds the class of the act, and c' is c with the head applied, the atoms of the head added at the machines of their holders and the negated ones deleted, and the class removed from every person, the will being fulfilled. A person changes their volitional state by a *volition change*, an act of theirs alone that no schema describes. In the terms of [51] the relation a schema denotes at a binding is the set of volition-guarded transactions it determines there, a machine transaction over the agents in its roles with the agents in its guarding roles as guards, which the two definitions below state.

For multisets $s_1, \ldots, s_n$ write $\bigsqcup_j s_j$ for their *join*, in which the multiplicity of each member is the greatest of its multiplicities in the s_j.

Definition 6 (Extent). Let β be a binding for a schema τ and d a machine configuration over a set of agents holding $\beta(\pi)$ for every agent role π of τ. An *instance* in d of a quantifier of τ with anchor A, within the instances of the quantifiers enclosing it, is an assignment α of the variables it binds under which the agent of the holder of A holds $(\beta \cup \alpha)(A)$ in d; a state being finite, a quantifier has finitely many, and the *multiplicity* of α is the number of copies of $(\beta \cup \alpha)(A)$ that agent holds. The *extent* in d of an agent role π of τ is $\mathrm{ext}_d(\pi) := \{\beta(\pi)\}$, and of a listed role x it is $\mathrm{ext}_d(x) := \{\alpha(x) : \alpha$ an instance in d of a quantifier binding x at which the condition φ of its conjunct holds$\}$, so a condition leaves out of an act those it excludes, as the guardians of a ward other than one of them end his guardianship (Section 3.5).

At a binding sending *Alice* to a and n to v, the extent of the listed role of closing a group in a configuration c is $\{q : \mathit{listed}(q, v) \in c_a\}$, every member a lists, a among them.

Definition 7 (Denotation). Let τ be a schema with roles $\pi_1, \ldots, \pi_k$, β a binding for it and d a machine configuration, and let $Q_i := \mathrm{ext}_d(\pi_i)$ and $Q := \bigcup_i Q_i$. For $p \in Q$ write $=_p$, $\neg_p$, $+_p$, $-_p$ and $_p$ for the joins, over the roles π_i with $p \in Q_i$, of $\beta(=(i))$, $\beta(\neg(i))$, $\beta(+(i))$, $\beta(-(i))$ and $\beta(_(i))$, a listed role's taken under $\beta \cup \alpha$ at each instance α of its conjunct with $\alpha(\pi_i) = p$ at which the conjunct's φ holds, together with the conditional literals held by p under $\beta \cup \alpha$ at each such instance of their conjuncts, sorted the same way. A machine transaction $d \to d'$ over Q is *determined by* τ *at* β, and the volition-guarded transaction it determines, below, is *denoted by* τ *at* β, if $\beta(\sigma \cdot y)$ is an argument of no atom of d for every name term $\sigma \cdot y$ of τ that occurs in no atom required at a role, every safe formula of B holds in d at β, d read as the structure over multisets of Section 3.1, and for every $p \in Q$

$$=_p \subseteq d_p,$$
$$d_p \text{ holds fewer than } \kappa \text{ copies of } \varphi, \text{ and no atom matching } \varphi, \text{ for each } \kappa \cdot \varphi \in \neg_p,$$
$$+_p \cap \left(-_p \uplus K_p\right) = \emptyset, \qquad d'_p = \left(\left(d_p \setminus -_p\right) \setminus K_p\right) \uplus +_p,$$

where K_p holds every atom of d_p, with its multiplicity, that matches an atom of $_p$; and the volition-guarded transaction it determines is the pair of it with its *guards*, $\bigcup\{Q_i : \pi_i$ a guarding role of $\tau\}$.

In the notation of [51], with $Q_i := \mathrm{ext}_c(\pi_i)$ and $Q := \bigcup_i Q_i$, τ *compiles to*

$$
\begin{array}{ll}
c'_p := \big((c_p \setminus -_p) \setminus K_p\big) \uplus +_p & (p \in Q),\\
\text{provided } =_p \subseteq c_p \text{ and } \neg_p \text{ is not held at } c_p & (p \in Q),\\
\text{provided } +_p \cap \big(-_p \uplus K_p\big) = \emptyset & (p \in Q),\\
\text{provided } \sigma \cdot y \text{ is an argument of no atom of } c & (\sigma \cdot y \text{ in no required atom}),\\
\text{provided the safe formulas of the body hold at } c, & \\
\text{guarded by the agents in the guarding roles of } \tau, &
\end{array}
$$

which at a binding β denotes the volition-guarded transactions determined by τ at β.

The holder of an atom in a schema is dropped in the state: $\beta(+(i))$ is added at the machine of $p \in Q_i$, and so on. The denotation of a contract is the protocol over the transactions its schemas denote (Definition 16): a transition system per set of agents, whose transitions are the volition changes and the transactions, and whose runs are correct when safe and live, every class that stays enabled being eventually taken. So a contract has the runs, the liveness and the grassroots notions of [51] by definition, and the theorems of Sections 6 and 7 are about its denotation. A schema is also a program of first-order dynamic logic, and the run properties are formulas of temporal logic (Section 12).

3.4 The Schemas of the Social Network

The schemas of GSN are those of GKK and GSG, GSG's unfriend replaced by GSN's, a message and the schemas of a group; those of GFP are GKK's, GSG's and two over *item*, of arity one: $item(x)$ in the state of Alice is a speech act Alice holds, which she signed or a friend signed over to her. Ten predicates, each shown with its arity in a state. *identity*, of arity zero: in the state of Alice it says that Alice has established an identity; every role of every schema but establish's requires it, and the clauses below show it only in establish. *knows*, of arity one: $knows(Bob)$ in the state of Alice says that Alice holds Bob's public key; nothing deletes or forbids it, a second copy meaning the same. *friend*, of arity one: $friend(Bob)$ in the state of Alice says that Bob is a friend of Alice. *received*, of arity two: $received(x, Alice)$ in the state of Bob is a speech act x Bob received from Alice, signed by Alice, what Alice received from others signed inside it. Alice and Bob are the roles, and Carol an agent variable.

GKK: Speech-Act Schemas

1. $identity(Alice) \leftarrow$
   $\neg identity(Alice)$ guarded by $Alice$.
2. $knows(Alice, Bob)$, $knows(Bob, Alice) \leftarrow$
   guarded by $Alice$, Bob.
3. $knows(Bob, Carol) \leftarrow$
   $knows(Alice, Bob)$, $knows(Alice, Carol)$, $knows(Bob, Alice)$ guarded by $Alice$, Bob.

GSG: Speech-Act Schemas

1. $friend(Alice, Bob)$, $friend(Bob, Alice) \leftarrow$
   $knows(Alice, Bob)$, $\neg friend(Alice, Bob)$, $knows(Bob, Alice)$, $\neg friend(Bob, Alice)$
   guarded by $Alice$, Bob.
2. $\neg friend(Alice, Bob)$, $\neg friend(Bob, Alice) \leftarrow$
   $friend(Alice, Bob)$, $friend(Bob, Alice)$ guarded by $Alice$.

GFP: Speech-Act Schemas

1. $item(Alice, x) \leftarrow$
   $\neg item(Alice, x)$ guarded by $Alice$.
2. $item(Bob, \text{sgn}(x, Alice)) \leftarrow$
   $item(Alice, x)$, $friend(Alice, Bob)$ guarded by $Alice$.

GSN: Speech-Act Schemas

1. $received(Bob, x, Alice) \leftarrow$
   $friend(Alice, Bob)$ guarded by $Alice$.
2. $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Alice, n) \leftarrow$
   $\neg member(Alice, \langle Alice\rangle \cdot n)$, $\neg closed(Alice, n)$ guarded by $Alice$.

   $closed(Alice, n)$, $\neg listed(Alice, Alice, n)$, $\neg member(Alice, \langle Alice\rangle \cdot n)$,
   $\forall Bob\,(listed(Alice, Bob, n) \rightarrow \neg member(Bob, \langle Alice\rangle \cdot n)) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Alice, n)$,
   $\forall Bob\,(listed(Alice, Bob, n) \rightarrow member(Bob, \langle Alice\rangle \cdot n))$ guarded by $Alice$.
3. $listed(Alice, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $friend(Alice, Bob)$, $\neg member(Bob, \langle Alice\rangle \cdot n)$
   guarded by $Alice$, Bob.

   $\neg listed(Alice, Bob, n)$, $\neg member(Bob, \langle Alice\rangle \cdot n) \leftarrow$
   $listed(Alice, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n)$ guarded by Bob.

   $\neg listed(Alice, Bob, n)$, $\neg member(Bob, \langle Alice\rangle \cdot n) \leftarrow$
   $listed(Alice, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n)$ guarded by $Alice$.

   $\neg friend(Alice, Bob)$, $\neg listed(Alice, Bob, _)$, $\neg member(Alice, \langle Bob\rangle \cdot _)$, $\neg friend(Bob, Alice)$,
   $\neg listed(Bob, Alice, _)$, $\neg member(Bob, \langle Alice\rangle \cdot _) \leftarrow$
   $friend(Alice, Bob)$, $friend(Bob, Alice)$ guarded by $Alice$.
4. $posted(Alice, x, n) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $\neg posted(Alice, x, n)$ guarded by $Alice$.

   $posted(Alice, x, n) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $listed(Alice, Bob, n)$, $friend(Alice, Bob)$, $\neg posted(Alice, x, n)$,
   $member(Bob, \langle Alice\rangle \cdot n)$ guarded by Bob.

   $delivered(Alice, x, Bob, n)$, $received_in(Bob, x, Alice, n) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $posted(Alice, x, n)$, $listed(Alice, Bob, n)$, $friend(Alice, Bob)$,
   $\neg delivered(Alice, x, Bob, n)$, $member(Bob, \langle Alice\rangle \cdot n)$ unguarded.

   $received_in(Alice, x, Alice, n) \leftarrow$
   $member(Alice, \langle Alice\rangle \cdot n)$, $posted(Alice, x, n)$, $\neg received_in(Alice, x, Alice, n)$
   unguarded.

A group is named by the community name $\langle Alice\rangle \cdot n$ (Section 3.1), Alice its leader and n a speech act Alice signs: $member(\langle Alice\rangle \cdot n)$ in the state of Bob says that Bob is a member of the group n of Alice, and in the state of Alice $listed(Bob, n)$ says the same, $closed(n)$ that the group has ended, $posted(x, n)$ that x was posted to it, and $delivered(x, Bob, n)$ that Alice delivered x to Bob in it; $received_in(x, Alice, n)$ in the state of Bob is a speech act x Alice delivered to Bob in the group n, signed by the member who posted it. In the schemas of a group Alice is the leader. A member's post adds the posted item at the leader, as the member signed it, and delivery adds it at each member. Remove, guarded by Alice, ends a member's membership at Alice's will. GSN's

unfriend, which replaces GSG's, also ends every membership between the two: it clears at each of them the other's listing in a group they lead and their own membership in a group the other leads. Close has a quantified conjunct in its body and in its head, over every Bob that Alice lists, the listed role $Alice[listed(_, n)]$ (Definition 2), so closing a group ends every membership in one act.

"Alice may establish her identity once" is establish, guarded by Alice and forbidding the *identity* it adds, and every other role requires it. "Alice and Bob may come to know each other by an act of both" is acquaint, guarded by both, each role adding *knows* of the other; "Alice can make Carol known to Bob, with Bob's consent" is introduce, guarded by both, Carol its agent variable, whom Alice must know and Bob comes to know, and "if Alice and Bob know each other" is the *knows* each role requires of the other. "Alice and Bob, who know each other … and are not friends, may jointly decide" is the two guarding roles of befriend, the *knows* each requires and its forbidden atom, and that they trust each other describes no act; "Alice may end her friendship" is the single guarding role of unfriend, which deletes in both roles. "Alice may send a message to her friend Bob" is message: only the sender guards, since only the sender decides; "her friend Bob" is the $friend(Bob)$ the sender requires; and the message is a speech act signed by the sender (Definition 5), what they received from others signed inside it. Create and close are GSN:2, join, guarded by both, leave, remove and GSN's unfriend GSN:3, and post_own, post, deliver and deliver_leader GSN:4. "Alice may create an item by signing it" is GFP's sign, x a speech act Alice signs (Definition 5); "may forward an item she holds to her friend Bob, adding her signature" is forward, Alice requiring $item(x)$ and $friend(Bob)$ and Bob receiving $item(\mathrm{sgn}(x, Alice))$, the item signed over by Alice, with its earlier signatures inside. Delivery is not guarded: the leader undertook it on creating the group.

The records. The schemas meet the clauses differently. Befriend is the only schema adding a *friend* atom, that atom's argument is a role, and befriend guards in both, so an agent's friends are only agents who have willed the friendship, and no act of the contract can put one there against that agent's will. Unfriend is the only schema deleting a *friend* atom and it deletes in both roles, so a friendship never ends for one agent without the other. A speech act a person signs records nothing against them until it is sent; the record is then the recipient's $received(x, Alice)$, signed by the sender, what the sender received from others signed inside it. Acquaint and introduce are the schemas adding a *knows* atom, so a person knows another only by an act of both or through someone who knows them, and befriend requires *knows* of the other at each role, so two persons can befriend only if they know each other. Knowing a person is not trusting them, so the schemas cannot stop the trust GSG:1 requires being absent; instead befriend leaves each of the two on record in the other's state.

3.5 The Schemas of Guardians and Wards

GnW amends GSG. The declaration of a contract says of each of its names what it is, a person or a ward, and GnW records it: an agent holds *person* or *ward*, of arity zero, from the act that establishes its identity, establish, which now adds *person* with *identity*, or the schema of GnW:1. A role named as a person requires *person*, and one named as a ward *ward*; so the acts of GKK and GSG are for persons, the person introduced being a role of introduce that reads, and no act of a person says anything of wards. Unfriend is a person's too, and not for a guardian and her ward, forbidding at Alice *guardian_of* of Bob. Five more predicates join them, each shown with its arity in a state. *creator* and *created*, of arity one: $creator(Alice)$ in the state of Carol and $created(Carol)$ in the state of Alice record that Alice created Carol, and no schema deletes them. *guardian_of* and *guarded_by*, of arity one: every guardian Bob of Carol, her creator at first among them, holds $guardian_of(Carol)$, and Carol holds $guarded_by(Bob)$. *retired*, of arity zero, records that a ward was retired. Every role requires *identity*, but establish's and Carol's in the schema of GnW:1, which

forbid it and add it, and every role of Alice, Bob and Erin requires *person*; the clauses below show neither.

GnW: Speech-Act Schemas

1. $created(Alice, Carol)$, $guardian_of(Alice, Carol)$, $friend(Alice, Carol)$, $knows(Alice, Carol)$, $identity(Carol)$, $ward(Carol)$, $creator(Carol, Alice)$, $guarded_by(Carol, Alice)$, $friend(Carol, Alice)$, $knows(Carol, Alice) \leftarrow$
   $\neg identity(Carol)$ guarded by *Alice*, *Carol*.
2. $guardian_of(Bob, Carol)$, $friend(Bob, Carol)$, $knows(Bob, Carol)$, $guarded_by(Carol, Bob)$, $friend(Carol, Bob)$, $knows(Carol, Bob) \leftarrow$
   $\neg guardian_of(Bob, Carol)$, $guardian_of(Alice, Carol)$, $friend(Alice, Bob)$, $ward(Carol)$, $guarded_by(Carol, Alice)$, $\neg guarded_by(Carol, Bob)$, $\neg friend(Carol, Bob)$, $\forall Erin\,(guarded_by(Carol, Erin) \rightarrow guardian_of(Erin, Carol))$ guarded by *Bob*, *Alice*, *Erin*.

   $\neg guardian_of(Bob, Carol)$, $\neg guarded_by(Carol, Bob) \leftarrow$
   $guardian_of(Bob, Carol)$, $guarded_by(Carol, Bob)$, $guardian_of(Alice, Carol)$, $guarded_by(Carol, Alice)$, $\forall Erin\,(guarded_by(Carol, Erin) \wedge Erin \neq Bob \rightarrow guardian_of(Erin, Carol))$ guarded by *Alice*, *Erin*.

   $\neg guardian_of(Bob, Carol)$, $\neg guarded_by(Carol, Bob) \leftarrow$
   $guardian_of(Bob, Carol)$, $guarded_by(Carol, Bob)$ guarded by *Bob*.

   $\neg friend(Bob, Carol)$, $\neg guardian_of(Bob, Carol)$, $\neg friend(Carol, Bob)$, $\neg guarded_by(Carol, Bob) \leftarrow$
   $friend(Bob, Carol)$, $guardian_of(Bob, Carol)$, $friend(Carol, Bob)$, $guarded_by(Carol, Bob)$ guarded by *Bob*.
3. $knows(Carol, Bob)$, $knows(Bob, Carol) \leftarrow$
   $ward(Carol)$, $guarded_by(Carol, Alice)$ guarded by *Carol*, *Bob*.

   $friend(Carol, Bob)$, $friend(Bob, Carol) \leftarrow$
   $ward(Carol)$, $guarded_by(Carol, Alice)$, $knows(Carol, Bob)$, $\neg friend(Carol, Bob)$, $knows(Bob, Carol)$, $\neg friend(Bob, Carol)$, $guardian_of(Alice, Carol)$, $friend(Alice, Bob)$ guarded by *Carol*, *Bob*, *Alice*.

   $\neg friend(Carol, Bob)$, $\neg friend(Bob, Carol) \leftarrow$
   $friend(Carol, Bob)$, $guarded_by(Carol, Alice)$, $\neg guarded_by(Carol, Bob)$, $friend(Bob, Carol)$, $guardian_of(Alice, Carol)$ guarded by *Alice*.

   $\neg friend(Carol, Bob)$, $\neg friend(Bob, Carol) \leftarrow$
   $ward(Carol)$, $guarded_by(Carol, Alice)$, $friend(Carol, Bob)$, $\neg guarded_by(Carol, Bob)$, $friend(Bob, Carol)$ guarded by *Carol*.
4. $knows(Carol, Dave)$, $knows(Dave, Carol) \leftarrow$
   $ward(Carol)$, $guarded_by(Carol, Alice)$, $ward(Dave)$, $guarded_by(Dave, Bob)$ guarded by *Carol*, *Dave*.

   $friend(Carol, Dave)$, $friend(Dave, Carol) \leftarrow$
   $ward(Carol)$, $guarded_by(Carol, Alice)$, $knows(Carol, Dave)$, $\neg friend(Carol, Dave)$, $ward(Dave)$, $guarded_by(Dave, Bob)$, $knows(Dave, Carol)$, $\neg friend(Dave, Carol)$, $guardian_of(Alice, Carol)$, $knows(Alice, Dave)$, $friend(Alice, Bob)$,

guardian_of(*Bob*, *Dave*) guarded by *Carol*, *Dave*, *Alice*, *Bob*; *Alice* and *Bob* possibly one person.

5. *retired*(*Carol*), ¬*friend*(*Carol*, _), ¬*guarded_by*(*Carol*, _),
   ∀*Bob* (*friend*(*Carol*, *Bob*) → ¬*friend*(*Bob*, *Carol*)),
   ∀*Erin* (*guarded_by*(*Carol*, *Erin*) → ¬*guardian_of*(*Erin*, *Carol*)) ←
   ward(*Carol*), *guarded_by*(*Carol*, *Alice*), *guardian_of*(*Alice*, *Carol*),
   ∀*Bob* (*friend*(*Carol*, *Bob*) → *friend*(*Bob*, *Carol*)),
   ∀*Erin* (*guarded_by*(*Carol*, *Erin*) → *guardian_of*(*Erin*, *Carol*)) guarded by *Alice*, *Erin*.

The schema of GnW:1 is enter: a ward is founded by its first guardian, who makes the ward's founding will on behalf of the ward, so both guard it. Those of GnW:2 are make_guardian, guarded by the new guardian Bob, by Alice, a guardian whose friend he is, and by every guardian Erin of Carol, a listed role; end_guardian, guarded by a guardian Alice and every other guardian Erin but Bob, whom the condition of its conjunct leaves out; resign; and guardian_unfriend. Those of GnW:3 are ward_acquaint, ward_befriend, ward_unfriend, by a guardian, and ward_end, by the ward, a person ending the friendship by unfriend. Those of GnW:4 are wards_acquaint and wards_befriend, and that of GnW:5 retire, guarded by the guardians of the ward, which takes every friend Bob and every guardian Erin of hers by its quantified conjuncts and clears her friendships and guardianships.

A ward takes no act without a guardian: every role a ward guards requires *guarded_by* of a guardian, Alice being an agent variable of ward_acquaint, ward_end and wards_acquaint and holding nothing in them. A guardian, the creator among them, may end their guardianship, so a ward may be left with none; she then goes on existing, with her records and her friends, who may each end their friendship with her, and takes no act, and nobody can become her guardian, there being no guardian for him to be a friend of. A retired ward is one with no guardian and no friend, and no guard need ask whether an agent is retired. GnW is parametric in six of its predicates, *ward*, *creator*, *created*, *guardian_of*, *guarded_by* and *retired* (Definition 38): a contract with wards of some kind instantiates them at predicates of its own, and has these schemas for its wards without restating one. A parametric contract has no realisation, so no platform runs GnW; each runs an instance of it. The relation these schemas write is the friendship; a contract with groups writes the membership of a group the same way, as the child-safe social network does [79]. A contract built on GnW may add to a ward's acts what its wards carry, as an AI agent carries its principal's attestation of the code it runs [80]; may restrict them, as a child comes to know only an agent a parent knows [79]; and may have wards of a kind that takes no act, their guardians taking every act for them, as the officers of an organisation do: no role of its schemas that guards requires the mark of that kind, and no schema says of an agent that it is not of it.

GnW is not syntactically grassroots, in the sense of Definition 14 below. Acquaint requires *person*, which a ward does not hold, and befriend requires *knows* of the other, which no schema of arity one adds; so the contract has no one act that any two agents can take. A ward enters the contract with a guardian recorded, who is a person, and while she has one comes to know any agent by an act of the two, and Section 8 certifies GnW by that.

3.6 The Schemas of the Coins

The schemas of GC are GKK's three and four over one predicate, ¢, of arity one: ¢(u) in the state of Alice is a coin issued by u and held by Alice. The roles are Alice and Bob; every role requires *identity*, shown only in establish. A negated atom of a head with a multiplicity k deletes k copies, so a deleted coin is also required, $k \cdot$ ¢(*Alice*, u) standing in the body and negated in the head.

GC: Speech-Act Schemas

1. $k \cdot ¢(Alice, Alice) \leftarrow$
   guarded by *Alice*.
2. $k_1 \cdot ¢(Alice, v)$, $knows(Alice, v)$, $\neg k \cdot ¢(Alice, u)$, $k \cdot ¢(Bob, u)$, $knows(Bob, u)$, $\neg k_1 \cdot ¢(Bob, v)$ $\leftarrow$
   $knows(Alice, Bob)$, $k \cdot ¢(Alice, u)$, $knows(Bob, Alice)$, $k_1 \cdot ¢(Bob, v)$ guarded by *Alice*, *Bob*.
3. $\neg k \cdot ¢(Alice, Bob)$, $k \cdot ¢(Bob, Bob) \leftarrow$
   $knows(Alice, Bob)$, $k \cdot ¢(Alice, Bob)$ guarded by *Alice*.
4. $\neg ¢(Alice, Bob)$, $¢(Alice, r)$, $knows(Alice, r)$, $\neg ¢(Bob, r)$, $¢(Bob, Bob) \leftarrow$
   $knows(Alice, Bob)$, $¢(Alice, Bob)$, $¢(Bob, r)$ guarded by *Alice*.

"Alice may decide" is the single guarding role of mint, of pay and of redeem; "Alice and Bob may jointly decide" is the two guarding roles of the swap; "Bob-coins she holds" is that pay and redeem require of Alice coins whose argument is Bob, pay k of them and redeem one; "whom she knows" and "who know each other" are the *knows* the roles require; "any coin held by Bob" is redeem's agent variable r; and whoever receives a coin comes to know its issuer, the swap and redeem adding *knows* of the issuer at the receiving role, so a holder can always redeem. The amounts k and k_1 are multiplicities (Section 3.2), positive whole numbers. The mutual credit line is the swap at a binding sending u to Alice and v to Bob, where each gives coins of its own issue.

The records. Selling at an advertised price is no act of the contract, so no act can breach GC:5 and no run shows whether the issuer honours it; but a mutual credit line leaves a coin of the issuer in another agent's holding, and that coin, naming the issuer, is the record of the undertaking. Minting records nothing against the issuer, putting the coin in its own holding; the undertaking becomes evidence only when the coin first passes to someone else, as a promissory note does when handed over. A clause that a holding contains a coin only if its issuer willed the act that put it there is breached by pay, which adds $¢(Bob)$ at Bob and guards only at Alice, and by the swap, which moves a coin between two agents by the will of the two. An issuer is not asked when its coin moves. A coin is therefore money rather than a promise between two people.

3.7 The Conditions Every Contract of Speech-Act Schemas Meets

Of the eight conditions of Section 1, (1), (2) and (4) are met by every contract written as speech-act schemas, and so is (5), in that nobody is bound who has not adopted the contract, by two properties of the language. Both are about the protocol realising a contract, so both are stated and proved in Section 5, where that protocol is defined, and used here. Conditions (3), (6), (7) and (8) are those of Section 3.8.

The first property is that every participant in an act takes a role in it: an agent's state changes only under a schema of the contract and only in a role of it, which is Proposition 19.

The second is that the language cannot name an agent. The agent roles R, the agent variables X and the speech-act variables Y are disjoint from Π, and a schema is written over atoms whose arguments are these and name terms built from them, so no schema mentions an agent; and by Definition 1 every agent begins in the same state $s_0 = \emptyset$. Permuting the agents therefore carries the system to itself, which is Proposition 20, and no agent can reach a position another could not, which is Corollary 21.

(1) *(1) An agreement of man with man.* Only agents hold states, and by Proposition 19 an agent bears an effect of the contract only in a role; by Corollary 21 no agent can reach a position another could not. No schema can require that an agent be a natural person.
(2) *(2) Reciprocal, (4) subject to no external authority.* An agent consents twice and both consents are their own: adopting the contract is consent to its schemas, and willing an act in a guarding role is consent to that act. By Proposition 19 nothing binds an agent outside a role of a schema, so nothing binds them that their adoption did not, and every participant in an act takes a role in it, so no registry, no log and no timing authority takes part in one; by Corollary 21 no agent is privileged by the text. There is nothing else in the model to be subject to. That an agent is bound only by their own promise, condition (3), is Volition, Definition 13; Proudhon's *reciprocal delivery*, that each agent has value for value, is a condition on the contract's content and is not carried over.
(3) *(5) Freely accepted and signed by all the participants.* This is how a contract is entered into rather than what it says. Section 11 says how: each participant signs a one-sided undertaking to keep the contract and presents it to every participant it interacts with, so every participant has signed it and, in an instance of two or more, another participant holds it (Proposition 54); and by Proposition 19 nobody else is bound.

The remaining conditions do not hold for every contract. A contract with no act by which two strangers can come into relation is written as speech-act schemas as readily as one that has such an act, and fails Introduction. Section 3.8 states them as conditions on the schemas.

condition	*formal counterpart*	*established by*
(1) an agreement of man with man	every participant takes a role; no privileged agent	Proposition 19, Corollary 21
(2) reciprocal	every participant takes a role	Proposition 19
(3) no obligation but by personal promise	volition, Definition 13	Proposition 29
(4) subject to no external authority	every participant takes a role	Proposition 19
(5) freely accepted and signed by all	the undertaking, Section 11	Proposition 54
(6) of the nature of a contract of exchange	an unobstructed introductory act, Definitions 9, 10	Proposition 28
(7) people free to deal with each other	an unobstructed introductory act	Proposition 28
(8) no external register of people	traceable provenance, Definition 11	Lemma 83

The counterpart of a condition in the table is structural. Proudhon's reciprocal delivery, that each agent has value for value, and exchange in its substantive sense are conditions on a contract's content, met by its clauses rather than by the form of its schemas.

3.8 What We Prove

Conditions (3), (6), (7) and (8) of Section 1 become three conditions on a contract's schemas, and the protocol realising a contract meeting them is grassroots. All three are properties of the text and are decidable, so whether a contract meets them is decided by a checker rather than left to its drafter. This section states them and the theorem; the sections that follow provide what is needed to prove it, and Section 7 proves it.

An agent's state names agents: those they know, their friends, the issuers of the coins they hold, the senders of the messages they hold. A state that names an agent stores an identifier of them, in the words of Section 1, and every condition below is about which agents a state names.

Definition 8 (Records). A local state *records* an agent q if one of its atoms names q.

The state $\{friend(q)\}$ of p records q. A state holding $received(x, p)$ records p, who signed x; an agent signed inside x is no argument of the atom and is not recorded.

A speech act names its signer and nobody else. A state holding one records the signer; an agent inside it, its earlier signers among them, is no argument of the atom carrying it and is not recorded: the chain of signatures inside a speech act is carried rather than required. A contract records, and its schemas can require, only the arguments of atoms and their signers, and an atom has finitely many.

Introduction. Conditions (6) and (7): any two agents must be able to come into relation by an act of the contract willed by both that leaves each recording the other, e.g. acquaintance.

Definition 9 (Introductory act). A schema τ of C of arity two, guarded in both roles, with no name term and with literals for its body and head, is an *introductory act* if any two agents $p \neq q$ have an *introduction*: an admissible binding β of τ into $\{p, q\}$ that sends π_1 to p and π_2 to q, under which some atom added at π_1 names q and some atom added at π_2 names p.

Acquaintance has arity two and guards in both roles, and at the binding sending *Alice* to p and *Bob* to q it adds $knows(q)$ at p and $knows(p)$ at q, each naming the other agent. Befriending is an introductory act too, but not an unobstructed one: it requires *knows* of the other, which no schema of arity one adds.

Having such an act is not enough, because it may be out of reach: it may require an atom an agent cannot obtain, or forbid one that somebody else can put there and leave. The condition is that neither happens.

Definition 10 (Unobstructed). An introductory act τ of C is *unobstructed* if any two agents $p \neq q$ have an introduction β such that, at each role π_i of τ:

(1) every atom τ requires at π_i is, under β, added by some schema of C of arity one that is guarded in its role, has no name term, has literals for its body and head, requires and clears nothing, and forbids nothing or only that atom, which τ then requires once at π_i, at some admissible binding that sends its role to $\beta(\pi_i)$;
(2) no listed role of a schema of C adds an atom whose predicate is that of an atom τ forbids at π_i, and every schema of C that, at some binding, adds at $\beta(\pi_i)$ an atom τ forbids at π_i has, at that binding, the other of p and q in a role that is not a reading role and adds at $\beta(\pi_i)$ an atom of its + naming them.

The proof of Proposition 28 takes an introduction and supplies every atom it requires by schemas of arity one, which is what the restrictions of Definitions 9 and 10 are for: a name term would need a fresh name, a formula other than a literal a condition on states those schemas do not set, and a constraint a binding satisfying it, whose existence is undecidable in general (Appendix C).

Acquaintance requires *identity* at each role, once, which establish adds, establish having arity one, being guarded in its role, requiring nothing and forbidding only *identity*, so clause 1 holds; and it forbids nothing, so clause 2 asks nothing.

The second clause concerns an atom somebody else can put in an agent's state and leave there. It excepts an act that puts it there while bringing the two agents together: its binding assigns the other agent a role, and the atom it adds names that agent, so every transaction it determines changes the state of both and leaves the first recording the second, and is an interaction between them. Befriending at the binding that swaps its two roles is one, and so is any more guarded act that specialises the introduction, e.g. a befriending that also takes the parents' will. The clause requires only that the act leave one of the two recording the other, which is enough for the act to be an interaction between them, and is all the proof of Proposition 28 uses of it.

Provenance. Condition (8): a record may name an agent its holder has never dealt with: a coin that changed hands, a message delivered to a group by its leader. Such a record must be traceable to someone who was there: either the act that made it had that agent as a participant, or another participant of the act already held a record naming them.

Definition 11 (Traceable provenance). The *base* of an argument of a schema is the argument itself where it is an agent role or an agent variable, and its base where it is a name term. Write Φ_E for the atoms of a schema whose predicate is in E. A set $E \subseteq \Sigma$ has *traceable provenance* in a contract C if for every schema τ of C with roles $\pi_1, \ldots, \pi_k$, every i, every $\varphi \in +(i) \cap \Phi_E$ and every argument u of φ that is an agent role, an agent variable or a name term, whose base is not that of π_i,

the base of u is the base of some π_j or of an argument of some $\psi \in =(j) \cap \Phi_E$, π_j an agent role,

or a safe formula of B names it through E, and every speech-act variable that is an argument of φ or occurs in a name term among its arguments is an argument of such a ψ, occurs in a name term among its arguments, is named by a safe formula of B through E, or occurs in no atom required at a role of τ. A safe formula *names* a base, or a speech-act variable, through E if it is an atom of Φ_E held by an agent role an argument of which has that base, or has the variable among its arguments or in a name term among them; a conjunction one conjunct of which names it, or one conjunct of which is the constraint $u = v$, of two agent variables, two speech-act variables or two name terms, and another names v, or its base, u, or its base, being then named; a disjunction both disjuncts of which name it; or $\exists \bar{x}\ (A \wedge \varphi)$ with $A \wedge \varphi$ naming it and $\bar{x}$ not binding it; for this the body is read as one conjunction, of its literals, its safe formulas and its constraint. A signed term $\mathrm{sgn}(y, \pi)$ among the arguments of φ counts as the two arguments y and π, here and in Definition 13. A predicate other than an environment predicate (Appendix C) has *traceable provenance* in C if it belongs to such a set, and an atom is said to have it when its predicate does. A conditional atom added at an agent role is read the same way (Definition 3), the anchor of its conjunct counting as an atom of $=$ at its holder.

> Delivery (Section 3.4) shows what the condition asks. It adds $\mathit{received_in}(x, \mathit{Alice}, n)$ at *Bob*: *Alice* is a role of it, and x is an argument of $\mathit{posted}(x, n)$, which *Alice* requires, so the speech act reaches Bob through an agent that held it already.

A predicate whose atoms are added only with roles among their arguments has traceable provenance. The condition is weaker: an act may give an agent a record of someone they do not yet record, provided another agent to that act records them already.

Volition. Condition (3): the agents to an act fall into parts connected by the records they hold, and the act either has one such part, as unfriending and messaging between friends and paying by a holder of the payee's coins do, or is willed by an agent in each part.

Definition 12 (Signature atoms). Let τ be a schema of C with no guarding role. Atoms $\varphi_1, \ldots, \varphi_m$ of $+(i) \uplus -(i)$, $m \geq 1$ and π_i an agent role of τ, are *signature atoms* of τ if every agent role of τ but π_i is the base of an argument of one of them, and each φ_j is either in $+(i)$, the only atom τ adds with its predicate, with no other schema of C with no guarding role adding an atom with that predicate, or in $-(i)$, the only atom τ deletes or clears with its predicate, with no other schema of C with no guarding role deleting or clearing an atom with that predicate.

Definition 13 (Volition). The *role graph* of a schema is the undirected graph on its roles with an edge between π_i and π_j when they have the same base or when one of the two requires an atom of traceable provenance an argument of which has the base of the other. A contract C satisfies *volition* if every schema of it has a connected role graph or has a guarding agent role in every connected

component of its role graph, every reading role of a schema of it with a guarding role has an edge to a guarding role from an atom that guarding role requires, or is a listed role whose base is a guarding role, which its anchor names, and every schema of it with no guarding role and a reading role has signature atoms and has only agent roles as reading roles.

A guarding role in each component is an agent role, a listed role's extent being possibly empty; the proof of Proposition 29 uses this, and Lemma 17 the signature atoms and the agent reading roles of a schema with no guarding role.

The role graph of *unfriend* joins its two roles: *Alice* requires *friend*(*Bob*), an atom of traceable provenance with *Bob* among its arguments. Only *Alice* guards unfriending, so that edge is what its role graph rests on.

The role graph is the undirected graph formed by the identifiers the agents store, of Section 1, taken on the roles of one schema. For a schema of two agent roles it is connected exactly when one of them requires such an atom naming the other. Above that arity connectedness is weaker than requiring it of every pair, and Proposition 29 uses it: lying in one instance is an equivalence, so a path through the roles puts all the participants there.

Definition 14 (Syntactically grassroots). A contract is *syntactically grassroots* if it has an unobstructed introductory act, the set of predicates occurring in it other than its environment predicates has traceable provenance, and it satisfies volition.

The denotation of a contract (Section 3.3) is defined for speech-act schemas, provided they have traceable provenance and satisfy volition; it is a grassroots protocol where they are syntactically grassroots, by the theorem below, and where they meet the conditions of Section 8.

Theorem 15 (Grassroots realisation). *The protocol realising a syntactically grassroots contract is grassroots and volitionally grassroots.*

Sections 4 to 7 prove it.

Whether a contract is syntactically grassroots is decidable, and a checker deciding it accompanies this paper, at https://github.com/EShapiro2/GLP/tree/main/programs/gsc. It certifies GSN and GC, with the sets of predicates of traceable provenance that Section 7 states. Everything the three conditions quantify over is finite once the bindings are taken up to renaming: an introductory act is an assignment of a schema's agent variables to its two roles; unobstructed is the matching of one atom against those the finitely many schemas add; volition is a connectivity check on the roles, together with a comparison of the atoms the schemas with no guarding role add and delete. Traceable provenance is not decided in a single pass: the largest set of predicates having it is reached by dropping any predicate that fails the condition and repeating, which ends because there are finitely many.

Section 4 recalls what *grassroots* means and Section 5 says what the protocol realising a contract is. Section 6 proves the theorem the proof rests on, and Section 7 proves this one.

4 VOLITION-GUARDED TRANSACTIONS AND GRASSROOTS PROTOCOLS

This section recalls from [51] what the statements below use, in brief. Nothing in it is ours. Appendix A states the same notions formally, together with the results used in the proofs of Appendix B, so that the paper is self-contained; the proofs of those results are in [51] and are not repeated. Volition-guarded transactions follow multiagent transition systems [72] and multiagent atomic transactions [82]. Throughout, Π is a countable, potentially infinite set of *agents*, an agent being a

person operating a machine, and $P \subset \Pi$ ranges over its finite nonempty subsets. For a set $\mathcal{S}$, $\mathcal{S}^P$ is the set of total functions from P to $\mathcal{S}$, and c_p is the value of c at p.

A *local-states function* S maps every $P \subset \Pi$ to a set $S(P)$ of local machine states containing the initial state s_0, with $P \subset P'$ implying $S(P) \subset S(P')$. A *machine transaction* over *participants* Q is a pair $d \to d' \in (\mathcal{S}^Q)^2$ with $d \neq d'$; it is *unary* if $|Q| = 1$. A *volition-guarded transaction* is a pair (t, Q') of a machine transaction t over Q with its *guards* $Q' \subseteq Q$: the agents in Q' must be willing for it to be taken. A *transaction equivalence* $\sim$ relates transactions with the same participants and has countably many classes; an agent wills a class, and taking one member of it fulfils that will. For a set R of volition-guarded transactions over S and $P \subset \Pi$, we write $R(P)$ for those of them whose transactions are over $S(P')$ for some $P' \subseteq P$, and call R *contained* if every transaction of it with participants in P and source over $S(P)$ is in $R(P)$.

An *agent state* is a pair of a *volitional state*, a set of classes the agent is willing their machine to take part in, and a machine state. A set of volition-guarded transactions over a local-states function induces, for each P, a transition system whose configurations are agent configurations over P and whose transitions are *volition changes*, by which an agent by themselves alters their volitional state, and *volitional machine transactions*, each induced by a volition-guarded transaction whose guards all hold its class. A transaction is *machine-enabled* in a configuration whose machine states restrict on the participants to its source, and *enabled* if in addition every guard holds its class. The family of these systems, one per P, is the *protocol* $\mathcal{F}$ over the set, $\mathcal{F}(P)$ being its member at P; runs are *safe* when consecutive states form transitions, *live* when no class stays enabled forever without a member being taken, and *correct* when both.

A transition of $\mathcal{F}(Q)$ is an *interaction* between two agents if it changes the state of each and its restriction to one of them is no transition of that agent's own system; it is an interaction between two disjoint sets of agents if it is one between a member of each. In the *interaction graph* of a run, two agents are joined whenever some transition of the run is an interaction between them; an *instance* in the run is a connected component of that graph, and two instances *coalesce* when they come to lie in one component.

A protocol is *oblivious* if any interleaving of correct runs of two disjoint sets of agents is a correct run of the two together, *interactive* if two sets that have not yet interacted can always still do so, and *grassroots* if it is both. An agent has *left* when no transaction they guard can be machine-enabled again. A protocol is *volitionally grassroots* if it is oblivious, two sets each holding an agent that has not left can always still interact, and the first interaction of two sets is induced by a transaction guarded by a member of each, so instances coalesce only by mutual consent; a grassroots protocol meeting the last condition is volitionally grassroots. In a grassroots protocol two instances can always coalesce.

5 THE REALISATION OF A CONTRACT

A schema denotes volition-guarded transactions (Definition 7), and a contract the protocol over them, which has the runs, the liveness and the grassroots notions of [51]. This section states that protocol and what is proved of it before the three conditions. The protocol is defined for contracts with traceable provenance that satisfy volition, and an app is compiled from a contract that is certified besides, as syntactically grassroots or with wards (Section 8), so a contract is checked before it is compiled. S_C is a local-states function in the sense of Appendix A: $s_0 \in S_C(P)$ for every P, and $P \subset P'$ implies $S_C(P) \subset S_C(P')$, strictly, provided some predicate has positive arity, as one does in every syntactically grassroots contract, its introductory act adding at π_1 an atom naming π_2: for $q \in P' \setminus P$ and such a predicate, an atom with q among its arguments gives a state in $S_C(P')$ and not in $S_C(P)$.

5.1 The Compilation

Compilation prints the transactions a schema denotes in the notation of [51], the form inside Definition 7, as the examples below show; the compiler of the repository prints it.

With Alice and Bob standing for the agents in those roles, befriend (Section 3.4) denotes, in that notation,

$$c'_{Alice} := c_{Alice} \uplus \{friend(Bob)\}, \qquad c'_{Bob} := c_{Bob} \uplus \{friend(Alice)\},$$
provided $identity, knows(Bob) \in c_{Alice}$ and $identity, knows(Alice) \in c_{Bob}$,
provided $friend(Bob) \notin c_{Alice}$ and $friend(Alice) \notin c_{Bob}$,
guarded by $\{Alice, Bob\}$,

and denotes, for every d with $identity, knows(Bob) \in d_{Alice}$, $identity, knows(Alice) \in d_{Bob}$, $friend(Bob) \notin d_{Alice}$ and $friend(Alice) \notin d_{Bob}$, the volition-guarded transaction $(d \longrightarrow d', \{Alice, Bob\})$ with $d'_{Alice} = d_{Alice} \uplus \{friend(Bob)\}$ and $d'_{Bob} = d_{Bob} \uplus \{friend(Alice)\}$. There is one for every admissible d, since the two may befriend whatever other friends each has, and $\sim$ relates them all: they are one act of befriending, available at different points of a run.

Write L for $\mathrm{ext}_c(Alice[listed(_, n)])$. Close (Section 3.4) denotes

$$c'_{Alice} := (c_{Alice} \setminus \{listed(Alice, n), member(\langle Alice\rangle \cdot n)\}) \uplus \{closed(n)\},$$
$$c'_q := c_q \setminus \{member(\langle Alice\rangle \cdot n)\} \qquad (q \in L,\ q \neq Alice),$$
provided $member(\langle Alice\rangle \cdot n), listed(Alice, n) \in c_{Alice}$,
provided $member(\langle Alice\rangle \cdot n) \in c_q \qquad (q \in L)$,
guarded by $\{Alice\}$.

Alice lists herself, so she is in the extent of the listed role, and the act deletes her membership with her listing. Its participants are Alice and every member she lists, read from her own state, so no member is left out of the act.

The swap (Section 3.6) denotes

$$c'_{Alice} := (c_{Alice} \setminus k \cdot \mathrm{¢}(u)) \uplus k_1 \cdot \mathrm{¢}(v) \uplus \{knows(v)\},$$
$$c'_{Bob} := (c_{Bob} \setminus k_1 \cdot \mathrm{¢}(v)) \uplus k \cdot \mathrm{¢}(u) \uplus \{knows(u)\},$$
provided $identity, knows(Bob) \in c_{Alice}$ and $k \cdot \mathrm{¢}(u) \subseteq c_{Alice}$,
provided $identity, knows(Alice) \in c_{Bob}$ and $k_1 \cdot \mathrm{¢}(v) \subseteq c_{Bob}$,
guarded by $\{Alice, Bob\}$.

The bindings with $u = v$ determine nothing, by Definition 7: under them each role adds coins it deletes. The swap at $u = Alice$ and $v = Bob$ is the mutual credit line.

The equivalence is on machine transactions, so two acts that no configuration distinguishes are one act, and a contract whose act names carry a distinction of their own writes that distinction into the effects. A contract compiles schema by schema. A schema with no guarding role compiles to transactions with an empty guard, which are enabled whenever they are machine-enabled, and one that stays enabled is eventually taken: no agent wills such an act. A clause saying that an act is taken whenever it can be is rendered by such a schema, as a contract for feeds and followers, in preparation, renders "Alice, who follows Carol's feed through Bob, receives from Bob every item of Carol's feed that Bob holds": Bob's role is a reading role requiring that Bob holds the item,

and Alice's role adds the item and the record that it was received from Bob, a signature atom (Definition 12).

Definition 16 (Realisation of a contract). Let C be a contract with traceable provenance that satisfies volition. Its *realisation* R_C is the set of volition-guarded transactions determined by the schemas of C at all admissible bindings for them, and $\sim$ is the equivalence on their underlying machine transactions generated by relating two of the same participants when one schema at one admissible binding determines both. The *protocol realising* C is the volitional transactions-based protocol over R_C, S_C and $\sim$.

A set of volition-guarded transactions over S_C with an equivalence $\sim$ is *well formed* if two machine transactions of different classes, each underlying an unguarded transaction of it, have disjoint P-closures for every $P \subset \Pi$, and $\sim$ is a transaction equivalence with countably many classes over any P, which is what Definitions 56 and 63 require.

Lemma 17 (The compilation is well defined). *R_C with $\sim$ is well formed.*

Lemma 18 (Containment). *R_C is contained.*

5.2 Every Participant Takes a Role

Throughout, C is a syntactically grassroots contract, $\mathcal{F}$ the protocol realising it and $P \subset \Pi$. No act reaches anyone without a role in it.

Proposition 19 (Every participant takes a role). *Every agent whose machine state a transition of $\mathcal{F}(P)$ changes is in the extent of a role of a schema of C at some binding.*

An agent therefore bears an effect of the contract only under a schema of it, and only in a role. The second property of Section 3.7 is that the schemas single out no agent.

A permutation σ of Π extends to speech acts by $\sigma(x, p) := (\sigma(x), \sigma(p))$, the identity on utterances, to community names by $\sigma(\langle p \rangle) := \langle \sigma(p) \rangle$ and $\sigma(v \cdot y) := \sigma(v) \cdot \sigma(y)$, to atoms by permuting their arguments, and to states, configurations, transactions and runs componentwise.

Proposition 20 (Anonymity). *For every permutation σ of Π, $\sigma(R_C) = R_C$, $\sigma(c_0(P)) = c_0(\sigma(P))$, and σ carries the runs of $\mathcal{F}(P)$ to the runs of $\mathcal{F}(\sigma(P))$.*

Corollary 21 (No privileged agent). *If a configuration is reachable in a run over P, so is its image under every permutation of P.*

Conditions (1), (2), (4) and (5) of Section 1 follow from these two properties, as Section 3.7 sets out. That the protocol realising the contract is grassroots does not follow from them.

6 POLITE SETS OF VOLITION-GUARDED TRANSACTIONS

Here we prove that the following condition on a contained set of volition-guarded transactions makes the protocol over it grassroots and volitionally grassroots: (1) any two agents can always come to interact, and (2) no act joins two instances unless each of them holds a guard of it. Section 7 states conditions on the acts of a contract sufficient for its compiled volition-guarded transactions to satisfy it, and GSN and GC (Section 2) satisfy it.

The proofs use two graphs: the interaction graph of Section 4, undirected, whose connected components are the instances; and the directed graph of who holds a record of whom, defined next.

Throughout this section, S is a local-states function, R a set of volition-guarded transactions over S with equivalence $\sim$, and $\mathcal{F}$ the protocol over R, S and $\sim$. The proofs, in Appendix B, use four results of [51] recalled in Appendix A: obliviousness rests on Lemmas 75 and 74 through

Proposition 76, and interactivity on Lemma 60; openness yields interactivity and closure yields consent.

Definition 22 (Records). A machine state s *records* an agent $q \in \Pi$ if $s \notin S(P)$ for every $P \subset \Pi$ with $q \notin P$.

Definition 23 (Recording graph). The *recording graph* of a machine configuration d is the directed graph on Π with an edge $p \to q$ whenever d_p records q. The recording graph of an agent configuration is that of its machine states, and the recording graph of a run is the union of the recording graphs of its configurations.

Definition 24 (Mutual recording). A volition-guarded transaction $(d \to d', \{p, q\})$ over participants $\{p, q\}$, $p \neq q \in \Pi$, is a *mutual recording of p and q* if it changes the state of each and the recording graph of d' has the edges $p \to q$ and $q \to p$.

A state records q when it could not have arisen among agents that leave q out: every set of agents whose states include it contains q. The recording graph of the initial configuration is empty, since $s_0 \in S(P)$ for every P. In GKK (Section 2.1) the edges out of an agent are those they know, in GSG their friends too, and in GC (Section 3.6) the issuers of the coins they hold. A mutual recording is an act of two agents, willed by both, after which there is an edge each way between them: acquaintance is one, and so is befriending. An edge may point at an agent its source has never transacted with, as when a message is delivered to a group by its leader or a coin changes hands, and nothing below forbids it. The recording graph of a configuration is not monotone along a run, as an act may delete the atoms that put an edge there; the recording graph of a run and the interaction graph are monotone by construction.

Definition 25 (Open, Closed, Polite). The set R is:

(1) *open* if for any two agents $p \neq q$ in any $Q \subset \Pi$, every finite safe run of $\mathcal{F}(Q)$ with no interaction between p and q has a finite safe extension, by volition changes and transitions that are not interactions between p and q, at whose end some transaction of $R(Q)$ whose induced transitions are interactions between p and q is machine-enabled;
(2) *closed* if for every $Q \subset \Pi$, every transaction of R enabled at the end of a finite safe run of $\mathcal{F}(Q)$ has all its participants in one instance in that run or a guard in every instance in that run holding one of them;
(3) *polite* if it is open and closed.

Openness says that any two agents can always be brought, by zero or more acts that are not between the two, to a configuration at which an act coupling the two can be taken if all its guards are willing. It does not require that no one else take part in the act: a mutual recording is one, and so is any act that leaves one of them holding a record of the other, whoever else takes part. Of two agents that have already taken part in one act nothing is required: they have interacted, and being interactive requires no more. Closure says that an act whose participants lie in two or more instances is enabled only where each of those instances holds a guard of it, and an act no one wills only where they already lie in one instance.

Theorem 26 (Polite). *The protocol over a contained, polite set of volition-guarded transactions is grassroots and volitionally grassroots.*

Politeness is a condition on the transactions of a protocol. Being grassroots is a property of its runs. The theorem states that the first is sufficient for the second. Section 7 proves the step before it, from the acts of a contract to the transactions realising them.

7 CONDITIONS SUFFICIENT FOR OPENNESS AND CLOSURE

We are now ready to prove Theorem 15, restated. Openness and closure are conditions on runs; the three conditions are on the schemas, and the proof has two halves: openness follows from the unobstructed introductory act, and closure from volition and traceable provenance. The proofs are in Appendix B. Throughout this section C is a syntactically grassroots contract and $\mathcal{F}$ the protocol realising it.

Theorem 15 (Grassroots realisation). *The protocol realising a syntactically grassroots contract is grassroots and volitionally grassroots.*

Lemma 27 (Records). *On local states, Definitions 22 and 8 agree.*

Proposition 28 (Openness). R_C *is open.*

Proposition 29 (Closure). R_C *is closed.*

Proof of Theorem 15. Let C be syntactically grassroots. By Proposition 28 R_C is open and by Proposition 29 it is closed, so it is polite; by Lemma 18 it is contained; and by Theorem 26 the protocol over it is grassroots and volitionally grassroots. □

Openness and closure. Openness lets any agent start an instance, and without permission. A contract that gives one agent of each instance a standing of their own is not open: two instances with no such agent can never coalesce. Federated architectures fail this condition, a server being such an agent [51].

Closure leaves an instance requiring no global resource and no central authority: a registry that issues identifiers, a log that orders acts, or an authority that timestamps them takes part in a transaction without a role in it. Every act of a grassroots contract is between its agents, so no third party takes part in one. A smart contract that depends on a fact outside the blockchain obtains it from an *oracle*, a third party whose report the contract cannot check [10]; here a fact about the world enters as an agent's signed assertion. A smart contract requires a consensus protocol; a contract realised as volition-guarded transactions requires none where the agents transact between themselves [39, 50], and where a collective decision is called for it constitutes its own consensus among the agents [46].

7.1 The Social Network Certified

Acquaint, which has no name term, is an introductory act: for any two agents, at the binding sending Alice and Bob to them, every transaction it determines adds *knows*(*Bob*) at Alice and *knows*(*Alice*) at Bob, changing both states, and it is guarded in both roles, so each is a mutual recording of the two. It is unobstructed: it requires *identity* at each role, once, which establish adds, establish having arity one and no name term, being guarded in its role, requiring nothing and forbidding only *identity*; and it forbids nothing. Acquaint and introduce are the schemas adding a *knows* atom: acquaint adds at each role an atom naming the other role, and introduce adds *knows*(c) at Bob, c an argument of *knows*(c), which Alice requires; so *knows* has traceable provenance, and so has *friend*, befriend being the only schema adding a *friend* atom and that atom's argument a role. So has the set of all ten predicates of the contract: establish adds *identity*, which has no arguments; message adds *received*(x, *Alice*) at Bob, whose arguments are the role Alice and the speech-act variable x, which occurs in no atom required at a role; and the schemas of a group meet the condition, as the paragraph below shows.

Volition holds, by Definition 13: acquaint and befriend are guarded in both their roles; establish has arity one; introduce's role Alice requires *knows*(*Bob*), and the role Alice of unfriend and of message requires *friend*(*Bob*), each an atom of traceable provenance naming the other role; and the schemas of a group meet it, as the paragraph below shows.

The schemas of a group. Create adds atoms whose arguments are its role, the name $\langle Alice \rangle \cdot n$ based on it, and a speech-act variable it requires nowhere; join and close add atoms whose arguments are roles and that name; post_own and post add $posted(x, n)$ at the leader, x required nowhere; deliver adds $delivered(x, Bob, n)$ at the leader and $received_in(x, Alice, n)$ at the member, whose arguments are roles, n and x, which the $posted(x, n)$ required at the leader carries; deliver_leader adds $received_in(x, Alice, n)$ at the leader, whose arguments are its role, n and x, which the $posted(x, n)$ it requires carries; and leave, remove and unfriend add nothing, so the six predicates of a group have traceable provenance, each traced through an agent role. Join is guarded in both its roles; create, post_own and deliver_leader have arity one; the two roles of close have the base Alice; and the role Alice of leave, remove and deliver requires $listed(Bob, n)$ and the role Bob of post requires $member(\langle Alice \rangle \cdot n)$, each an atom of traceable provenance an argument of which has the base of the other role, so volition holds of them. Acquaint forbids nothing, so clause 2 of Definition 10 asks nothing of the listed role of close.

The contract is therefore syntactically grassroots, and by Theorem 15 the protocol realising it is grassroots and volitionally grassroots, so the platform realising the contract is a grassroots platform. So is GFP: sign adds $item(x)$ at Alice, x required nowhere; forward adds $item(\mathrm{sgn}(x, Alice))$ at Bob, whose arguments are the role Alice and x, which the $item(x)$ Alice requires carries, so *item* has traceable provenance; sign has arity one and forward's role Alice requires *friend*(*Bob*), so volition holds; and acquaint is its unobstructed introductory act as in GKK.

The instances. The introductory act is acquaintance, and a *knows* atom was added by an acquaint, a mutual recording and so an interaction by Lemma 81, or by an introduce, an interaction of the two it joins, the one introduced being known to the introducer already; every *friend* atom was added by a befriend, a mutual recording. Every interaction of this contract between two agents requires or adds an atom in the state of one of them naming the other, so it puts an edge of the recording graph between them; and by Lemma 83, every edge of the recording graph joins two agents in one instance. The instances of a run are therefore the connected components of its recording graph, taken undirected; under GSG every friend edge was willed by both its ends, so the social graph is a web of trust and its instances are its connected components. A message adds an edge from the recipient to the sender, a participant; a delivery adds one from the member to the leader, a participant, and one to the member who posted the item, who signed it, reached through the leader (Lemma 83). A coin is different: it names its issuer, who may never have transacted with its holder (Section 7.2).

7.2 The Coins Certified

Acquaint is GC's introductory act, unobstructed, as in GKK (Section 7.1); the swap is an introductory act too, at the binding sending u to Alice and v to Bob leaving each holding a coin the other issued, the mutual credit line, but not an unobstructed one, requiring at each role *knows* of the other and a coin, which no schema of arity one that requires nothing adds, mint requiring *identity*.

¢ has traceable provenance, as *identity* and *knows* have in GKK; the swap and redeem add *knows* of the issuer of the coin a role receives, the coin an atom the other role requires. Mint adds ¢(*Alice*) at Alice and pay adds ¢(*Bob*) at Bob, each naming the role that holds it; the swap adds in each role a coin the other role requires; and redeem adds $¢(r)$ at Alice, which role Bob requires, and ¢(*Bob*) at Bob, naming its own role. Volition holds, by Definition 13: mint has arity one, the swap is guarded

in both its roles, and pay and redeem are guarded at Alice only and require there *knows*(*Bob*) and ¢(*Bob*), atoms of traceable provenance having Bob among their arguments. The contract is therefore syntactically grassroots, and by Theorem 15 the protocol realising it is grassroots and volitionally grassroots, so the platform realising the contract is a grassroots platform.

The origin of a record. A coin names its issuer, who is not a participant of the payment that moves it, so a ¢ edge may point at an agent its source has never transacted with, where a *friend* edge can only have been added by the two agents it joins. Traceable provenance covers it: an edge is added either to a participant of the act that adds it or to an agent another participant of that act records already, so no act ever leaves an edge whose ends lie in different instances, which Lemma 83 proves by induction on the run. For coins the chain is the passage of the coin from its issuer, and conservation of money [51] is the special case, here an invariant of the schemas rather than one proved of the runs.

8 CONTRACTS WITH WARDS

A ward comes to know an agent by an act of her own, acquaintance being for persons, so a contract with wards has no one act that any two agents can take and is not syntactically grassroots (Section 3.5). The protocol realising it is volitionally grassroots nonetheless: a ward enters the contract with a guardian recorded, who is a person, and while she has a guardian comes to know any agent by an act of the two; and a ward with no guardian has left (Definition 77). This section states conditions on the schemas under which a contract with wards is certified, decidable as those of Section 3.8 are and decided by the same checker, states that they are sufficient (Theorem 36) and certifies GnW by them. The conditions say what an agent is and holds, and none says what an agent is not: an act of a person requires that its agents be persons, and mentions no ward. A contract that confines a ward to the agents her guardian knows, as the child-safe social network does [79], writes its acts of knowing with the guardian among their roles, and the definitions admit both. The proofs are in Appendix B.

Reflexive predicates. Two agent roles a schema marks as possibly one are bound to one agent only if neither adds or deletes an atom naming the other (Definition 5), so no relation a contract records is reflexive. A contract may declare a predicate of arity one *reflexive*: every state is then taken to hold the atom of that predicate naming its holder, and no schema adds, deletes or clears, at a role, the atom of it naming that role. For a configuration d and an agent p, $\hat{d}_p$ is d_p together with the atom naming p of every reflexive predicate, and Definition 7 reads $\hat{d}_p$ for d_p in what it asks of a state: what is required, what is forbidden and the safe formulas of the body. In a contract declaring none, $\hat{d}_p = d_p$. Definition 10 then excepts, in clause 1, an atom of a reflexive predicate naming π_i, and asks besides that τ forbid no such atom at π_i. GnW declares *friend* and *knows* reflexive, so that two wards with one guardian may befriend by an act in which that guardian takes both guardian roles (GnW:4).

Lemma 30 (Reflexive predicates). *Lemmas 18 and 83 and Propositions 28 and 29 hold for a contract declaring reflexive predicates, with Definition 10 read as above.*

Kinds. A *mark* is a predicate of arity zero in a state, as *identity*, and its atom is written as the mark. The declaration of a contract says what each of its names is, a person or a ward, and a contract with wards has a mark for each: an agent holds the mark of its kind from the act that establishes its identity.

Definition 31 (Kinds). C has kinds (ι, f, M), for marks ι and f and a nonempty set M of marks not holding them, if:

(1) no schema of C deletes ι or f;
(2) a schema that adds ι at a role forbids it there and adds there f or one mark in M, and adds no other of them there; a schema adds a mark in M only so, and at one role; and it adds f only so, or at a role at which it deletes a mark in M;
(3) a schema that deletes a mark in M does so at one role, and adds f there;
(4) C has an *establishing act*, a schema of arity one, guarded in its role, with no name term and with literals for its body and head, that adds ι and f, requires and clears nothing and forbids only ι.

An agent holding f is *free*, and one holding $m \in M$ a *ward* of the kind m.

In GnW ι is *identity*, f is *person* and $M = \{ward\}$: establish adds *identity* and *person*, and enter adds *identity* and *ward* at the ward. An agent with an identity is of one kind (Lemma 84), and a ward becomes a person only by an act that deletes her mark, as a child comes of age [79].

Guardianship and relay. A ward holds a record of each of her guardians, and each guardian a record of her; a guardian is a person.

Definition 32 (Guardianship). Let C have kinds (ι, f, M). A *guardianship* of $m \in M$ in C is (m, v, u), with v and u predicates whose first argument is an agent and whose other arguments, if any, are values, their atoms written $v(\rho, \bar{z})$ and $u(\sigma, \bar{z})$, such that:

(1) no schema of C clears an atom of v, and a schema clears an atom of u only with a wildcard for each of its arguments;
(2) a schema that deletes m at a role requires an atom of u there;
(3) a schema adds $u(\sigma, \bar{z})$ at a role ρ if and only if it adds $v(\rho, \bar{z})$ at σ, a role of it at which it requires f; and it then adds m at ρ, or requires m at ρ and forbids $u(\sigma, \bar{z})$ there, and, unless m is passive, requires an atom of u there;
(4) a schema deletes $u(\sigma, \bar{z})$ at a role ρ if and only if it deletes $v(\rho, \bar{z})$ at σ; and it clears u at ρ if and only if it has the listed role $\rho[u(_, \bar{z})]$, its conjuncts with no condition, and deletes $v(\rho, \bar{z})$ at it.

The mark m is *passive* if no guarding role of a schema of C requires it and no schema of C deletes it. An agent variable π *can act* is the safe formula $f(\pi) \vee \bigvee_m (m(\pi) \wedge \exists x, \bar{z}\, u_m(\pi, x, \bar{z}))$, over the marks m of M that are not passive, u_m the u of the guardianship of m. C *has guardianships* if every mark in M has one and every guarding role π of every schema of C forbids ι, requires f, requires a mark in M and an atom of the u of its guardianship, or has in the body of its schema that π can act.

The ward x holds m and $u(b)$ of every guardian b, and b holds $v(x)$ (GnW:1 and GnW:2); the definition asks nothing of how a ward is created but that her mark come with her identity, so a ward created with no guardian has left from the start. The last condition says who takes an act: an agent establishing an identity, a person, or a ward with a guardian, or, where the act is for any of them, an agent who can act, as any agent who can act may sign an item and forward it [80]; so a ward takes no act without a guardian. A ward is founded by its first guardian, who makes the ward's founding will on behalf of the ward: the act that creates a ward is guarded by both, the guardian having chosen the ward's key pair, since nothing in the guardian's state names the ward before it (Definition 13). A ward of a passive kind wills nothing after its founding, every act of it being taken for it by its guardians, as the officers of an organisation do. In GnW v and u are *guardian_of* and *guarded_by*: enter adds *ward*, *identity* and *guarded_by*(*Alice*) at Carol, who had no identity, and *guardian_of*(*Carol*) at Alice, a person, and make_guardian adds the two records for another guardian; *creator* and *created* record the creation and are no part of the guardianship. The values $\bar{z}$ are for a guardian's office, as an officer of an organisation holds a role in it, and where a kind is passive a guardian may be given to a ward that has none, as its bylaws name who appoints an officer of an organisation that has none.

Definition 33 (Unobstructed among the free). Let C have kinds (ι, f, M). An introductory act τ of C that adds one atom at each role, of a predicate e of arity one, is *unobstructed among the free* if τ forbids nothing and every atom τ requires at a role π_i is ι; or f; or an atom of a reflexive predicate naming π_i; or, up to renaming the role, the one atom added by some schema of C of arity one that is guarded in its role, has no name term, has literals for its body and head, requires nothing but ι and f, clears nothing and forbids nothing but that atom.

The act neither requires nor forbids an atom of e, so two persons may take it whether or not each holds e of the other, and a local state, a multiset, counts the copies; an agent may hold e of an agent they never dealt with, as GKK:3 introduces one person to another. GnW's acquaint, which requires *identity* and *person* at each role and adds *knows* of the other there, is unobstructed among the free.

Definition 34 (Relay act). Let C have kinds (ι, f, M) and guardianships, and let τ be an introductory act of C unobstructed among the free, adding atoms of e. A schema χ of C with no name term and with literals for its body and head, which forbids nothing, is a *relay act* with *related roles* $\rho_1 \neq \rho_2$, roles of it guarded in χ at each of which χ adds an atom of e naming the other, if:

(1) each related role ρ is a *ward role* of some $m \in M$, requiring m, with at most one other role σ of χ, its *guardian role*, requiring an atom of v naming ρ, the v of the guardianship of m, a guardian of the ward, or a *free role*, requiring f; every role of χ is a related role or a guardian role; and χ adds and deletes at a guardian role no atom naming another guardian role;

(2) every atom χ requires at a role is ι; or f, at a free role or a guardian role; or an atom of a reflexive predicate naming the role; or, at a free role or a guardian role, up to renaming the role, the one atom of a schema of arity one as in Definition 33; or, at a ward role of m, its m, or one atom of the u of its guardianship, naming its guardian role or, where it has none, an agent variable of χ that occurs in no other atom of χ, the value arguments of the atom variables of χ that occur in no other atom χ requires but the guardian role's atom of v; or, at a ward role of m, an atom of a predicate h whose arguments are distinct speech-act variables of χ occurring in no other atom χ requires but its record below, where every schema of C that adds m at a role ρ adds there an atom of h whose arguments are speech-act variables or signed terms, and adds its *record*, an atom of a predicate h' whose arguments are ρ and the arguments of that h-atom, at the role at which it adds an atom of v naming ρ; every other schema of C that adds an atom of v naming ρ at a role requires a record $h'(\rho, \ldots)$ at a role at which it requires an atom of v naming ρ, and adds that record where it adds the atom of v; and no schema of C deletes an atom of h or of h'; or, at the guardian role σ of a ward role ρ, its atom of v naming ρ, or the record, the atom of h' whose arguments are ρ and the arguments of the h-atom ρ requires; or, at a guardian role or a free role, $e(\kappa)$ for another guardian role or free role κ, with e reflexive where both are guardian roles; or, where χ has two ward roles, at the guardian role of one, e of the other ward role and its record; and χ adds at a free role the record of each ward role.

A relay act is an act by which a ward and another agent come to know each other, with the guardian of a ward among them where the contract confines the ward to the agents her guardian knows. GnW's ward_acquaint is a relay act with a ward role, which requires *ward* and *guarded_by* of a guardian, and a free role, the person, and no guardian role, and wards_acquaint one with two ward roles and no guardian role. The predicate h of clause 2 is for what a ward carries and presents to each agent she comes to know, who records it: an AI agent carries the attestation its principal gave it on entering [80]; GnW has none. In the child-safe social network a child and an adult come to know each other by a relay act with the child's parent as the guardian role, a restriction of ward_acquaint by (d) of Proposition 45, requiring of the parent *knows* of the adult, and two children by one with two ward roles and their two parents, requiring of each parent *knows*

of the other, which binds both parents to one person where the two children share one, *knows* being reflexive [79]. Befriending, which requires *knows* of the other at each role, is no relay act.

Definition 35 (Introduction with wards). C has an *introduction with wards* if it has kinds (ι, f, M) and guardianships, an introductory act unobstructed among the free, and, for every $m, m' \in M$ that are not passive, a relay act with a ward role of m and a free role and a relay act with a ward role of m and a ward role of m'.

Theorem 36 (Grassroots realisation with wards). *The protocol realising a contract with an introduction with wards, traceable provenance and volition is volitionally grassroots.*

The proof follows that of Theorem 15. Closure and containment are as there, their proofs using nothing of the introductory act. Openness is among the agents that have not left: two free agents come to know each other by the introductory act, and a ward who has not left, and so has a guardian, and another agent by a relay act, what it requires of their guardians being supplied first by acts among free agents. A ward with no guardian has left, and so has a ward of a passive kind.

8.1 Guardians and Wards Certified

GnW has an introduction with wards. Its kinds are *identity*, *person* and $M = \{ward\}$: no schema deletes any of the three; establish, the establishing act, adds *identity* with *person* and enter adds it at Carol with *ward*, each forbidding it there, and no other schema adds one of them. The guardianship of *ward* has *guardian_of* for v and *guarded_by* for u. Only retire clears, *friend* and *guarded_by* at Carol, each with a wildcard, deleting *guardian_of*(*Carol*) at every Erin she holds *guarded_by* of. Enter adds *ward* at Carol with *guarded_by*(*Alice*), Alice a role of it that requires *person* and gains *guardian_of*(*Carol*). Make_guardian, the other schema adding the two records of a guardian, adds *guarded_by*(*Bob*) at Carol, who requires *ward* and *guarded_by*(*Alice*) and forbids *guarded_by*(*Bob*), and *guardian_of*(*Carol*) at Bob, who requires *person*; end_guardian, resign and guardian_unfriend delete the two together. Every guarding role forbids *identity*, as establish's and Carol's in enter; or requires *person*, as a person's and a guardian's, Erin's among them; or requires *ward* and *guarded_by*, as a ward's. Acquaint is an introductory act unobstructed among the free: it adds *knows* of the other at each role, requires *identity* and *person*, and forbids nothing. The relay acts are ward_acquaint, with the ward role Carol and the free role Bob, and wards_acquaint, with the ward roles Carol and Dave: each role is guarded and gains *knows* of the other, a ward role requires *identity*, *ward* and *guarded_by* of an agent variable, the free role *identity* and *person*, and they forbid nothing.

The set of the ten predicates has traceable provenance: every atom a schema adds has for its arguments roles of the schema. Volition holds: establish has arity one; acquaint, befriend, enter, ward_acquaint, ward_befriend, wards_acquaint and wards_befriend are guarded in every role; in introduce Alice requires *knows* of Bob and of Carol, the reading role, and in unfriend *friend*(*Bob*); in make_guardian Alice requires *friend*(*Bob*) and *guardian_of*(*Carol*), and in end_guardian *guardian_of*(*Carol*), which Bob requires too, as he does in resign and guardian_unfriend; in ward_unfriend Alice requires *guardian_of*(*Carol*) and Carol *friend*(*Bob*), which she requires in ward_end; in retire Alice requires *guardian_of*(*Carol*); and the listed roles of make_guardian, end_guardian and retire have the base Carol.

Proposition 37 (GnW is grassroots). *The protocol realising GnW is volitionally grassroots.*

So the platform realising GnW is a grassroots platform, in which a ward meets, befriends and leaves as her guardians will. GnW is parametric (Section 3.5), and every instance of it is certified with it (Theorem 46, clause 5). A contract built on an instance is certified the same way, with its own

kinds: accountable AI agents with *agent* and the attestation an AI agent carries [80], the child-safe social network with *child*, a parent a guardian role of its relay acts [79], and an organisation with a passive kind, its officers its guardians, each with a role.

9 RELATIONS AMONG CONTRACTS

Contracts are written from other contracts: GSG extends GKK, GSN and GnW amend GSG, and the child-safe social network [79] and the network with AI agents [80] are built on both. This section defines five relations between contracts: amendment, extension and instantiation, on their schemas, and implementation and restriction, on their runs; and it states what the check of Section 3.8 gives for a contract so obtained from one already certified. Throughout, C and C' are contracts over predicates Σ and Σ'.

Definition 38 (Parametric contract, instantiation). A *parametric contract* is a contract some of whose predicates and constants are *parameters*, a predicate declared with its arity and the kind of each of its arguments, agent, speech act, name or value, and a constant with its kind, value or utterance, each occurring in any of its schemas. An *instantiation* of it writes, for some of its parameters, predicates of the same arity and kinds, or, for a predicate that is not a mark, of the same kinds with further value arguments, and values and utterances of the same kind, that do not occur in it, a different one for each; in each schema, the atoms of such predicates over one set of agent variables share a fresh value variable for each added argument, bound by the quantifier that binds one of those agent variables. An instantiation may also write names of its own for schemas of the contract, a different one for each, so that an act bears the name its instance gives it, as a child's acquaintance is no ward's; the name of a schema has no part in what the schema denotes. The term of a parametric contract names its parameters, and that of an instantiation the contract it instantiates and what it writes for each parameter and for each schema it names. A parametric contract has no realisation: Definition 16 is for a contract none of whose predicates or constants is a parameter, and only an instantiation of every parameter of a parametric contract is realised, as a protocol and as an app.

Definition 39 (Amendment, extension, instantiation). (1) C' *amends* C if it is obtained from C by deleting schemas and adding schemas.
(2) C' *extends* C if it amends C deleting nothing.
(3) C' *instantiates* C if C is parametric and C' is obtained from C by an instantiation.

Definition 40 (Lift, extension at an argument). Let C be a contract and F a set of its predicates. The *lift* of C at F is the contract obtained from C by giving each predicate of F one more argument, a value, and, in each schema of C, adding one value variable, fresh to the schema, as that argument of every atom of a predicate of F. C' *extends* C *at* F if C' extends the lift of C at F.

Each act of C stays, in its lift, within one value of the added argument. A contract of denominated coins and bonds, in preparation, extends GB [81] at its coin and bond predicates, the denomination being the added argument; its acts across denominations are among the schemas it adds. GSN amends GSG, deleting its unfriend and adding its own and the schemas of messages and groups. The *composition* of two contracts amending a third is the schemas of the third that both keep together with the schemas each adds; it amends each, and where both extend the third, as GSN and GC extend GKK, it is their union, which extends each; one app runs it (Section 1). A contract extends several contracts when it extends their union, as a contract for grassroots banks, in preparation, extends the contracts for grassroots organisations and for denominated coins and bonds, and is checked as an extension of the union, the union an extension of each. GnW is parametric in its six predicates of wards, *ward*, *creator*, *created*, *guardian_of*, *guarded_by* and *retired*: the child-safe

social network [79] instantiates them at predicates for a child and her parents, accountable AI agents [80] at predicates for an AI agent and the persons operating it, and a contract for grassroots organisations, in preparation, at predicates for an organisation and its officers; so each has GnW's acts and restates none, a platform runs an instance and none runs GnW, and one with wards of two kinds has two instances and the acts by which a ward of one comes to know a ward of the other. GC with its coin predicate as a parameter is a contract for any asset issued by an agent, instantiated at ¢ by GC itself, and a contract for a bank whose bylaws are a constant parameter is instantiated by each bank at its own bylaws, as a contract for grassroots organisations, in preparation, does: its acts ask whether the bylaws admit them, so no bank runs the parametric contract, which has no bylaws, and each bank runs its own instance; a contract is parametric in the types of its values without declaration (Appendix C).

Definition 41 (Implementation, restriction). Let C and C' be contracts with traceable provenance that satisfy volition, and $\mathcal{F}$ and $\mathcal{F}'$ the protocols realising them (Definition 16). C' *implements* C through a map μ from the local states of C' to those of C if for every $P \subset \Pi$ and every correct run r' of $\mathcal{F}'(P)$ there is a correct run r of $\mathcal{F}(P)$ whose machine configurations, with consecutive repetitions removed, include those of r', mapped by μ and with consecutive repetitions removed, in order, from the first and, where these are finitely many, to the last. C' *restricts* C if $\Sigma' \subseteq \Sigma$ and C' implements C through the identity.

Where those configurations are all of those of r, this is the correct implementation of [51], read on machine configurations, its stutters removed; in addition, a run of C' may take in one act what C takes in several. A restriction allows fewer runs: every correct run of C' is, on the machines, one of C. An implementation may also add records the implemented contract lacks.

Proposition 42 (Restriction is undecidable). *Whether one syntactically grassroots contract restricts another is undecidable.*

Restriction is therefore approximated by syntactic conditions that imply it. The set of syntactic approximations is dynamic, and we hope and expect it to grow; the four below are proved in Appendix B. Each strengthens a body, in the following sense.

Definition 43 (Stronger body). Let τ and τ' be schemas with the same head, every role of τ a role of τ' and every variable of τ a role or a variable of τ'. The body of τ' is *stronger* than that of τ if τ' requires at each role of τ every atom τ requires there and its body holds every safe formula of the body of τ, the speech-act variables and name terms occurring in the atoms the two require at their roles are the same, and the body of τ holds at every machine configuration d and binding β for τ' at which that of τ' holds, a body *holding* at d and β when they meet the conditions Definition 7 sets on its atoms, its safe formulas and its constraint.

Whether one body is stronger than another is undecidable in general, a body being a formula of first-order logic with counting; the checker decides it where the stronger body holds every literal and every safe formula of the weaker and its constraint implies the weaker's, a sufficient condition.

Lemma 44 (Stronger body). *Let the body of τ' be stronger than that of τ, every role of τ' that is not one of τ an agent role, and every binding admissible for τ' admissible for τ. If τ' determines $d \to d'$ at an admissible binding β, then β is admissible for τ, τ determines at β the restriction of $d \to d'$ to the agents in the extents in d of the roles of τ, and every other agent p of d has $d'_p = d_p$.*

A binding admissible for τ' is admissible for τ where every guarding role of τ' is one of τ; where no speech-act variable of τ occurs in an atom it adds and in none it requires; and where the constraint of τ' has every such variable signed by the agent in a guarding role of τ (Definition 5).

The approximations go class by class, so two of them need the classes of the acts no person wills to be those of one schema at one binding. A contract has *simple unguarded classes* if every class of its realisation that contains a machine transaction underlying an unguarded transaction is the set of machine transactions one schema with no guarding agent role determines at one admissible binding, and no other schema, nor that schema at another binding, determines a member of it. A contract has them where each schema with no guarding agent role has an atom it adds or deletes at an agent role, the only atom with its predicate that a schema of the contract adds, deletes or clears, among whose arguments, or in name terms among them, every agent role and every variable of the schema occurs, as the delivery of a contract for feeds and followers, in preparation, has.

Proposition 45 (Syntactic restrictions). *Let C and C' have traceable provenance and satisfy volition, with $\Sigma' \subseteq \Sigma$. C' restricts C if it is obtained from C by one of the following, and, restriction being transitive, by a sequence of them:*

(a) replacing a schema τ with a guarding agent role by a schema τ' with its name and guards and a stronger body, every role of τ' that is not one of τ an agent role that is a reading role, or deleting such a schema, C and C' having simple unguarded classes;

(b) adding a schema with no guarding role, with the head and the roles of a schema τ of C and a body stronger than that of τ;

(c) replacing a schema τ with no guarding agent role by a schema τ' with its name, roles and guards and a stronger body, where at every configuration of every correct run of the protocol realising C' and at every admissible binding the body of τ' holds wherever that of τ does, C and C' having simple unguarded classes;

(d) replacing a schema τ with a guarding agent role by a schema τ' with its name and a stronger body, whose guarding roles are those of τ and agent roles that are no roles of τ, every other role of τ' that is not one of τ an agent role that is a reading role, and every binding admissible for τ' admissible for τ, C and C' having simple unguarded classes.

Under (a) a person may will fewer acts, and an act may read more agents' states; under (b) the machine takes, unasked, an act a person may will, when a stronger body holds; (c) needs an invariant of the contract, proved for each contract; and under (d) an act takes the will of more persons, who change no state, as a child comes to know an agent with the consent of a parent [79], the name of a group the child creates being still the child's to sign. Deleting a schema with no guard is not a restriction in general, as the proof of Proposition 42 shows, and nor is strengthening its body without such an invariant: either may drop an act the original owes. An undertaking formula (Definition 50) is part of a clause and denotes no transaction, so adding an undertaken clause with one leaves the schemas and the runs as they are: a contract for distrust among friends, in preparation, is restricted by strengthening the body of befriending, by (a), and by adding a clause whose formula says that Alice, whose friend Bob distrusts Carol, will eventually distrust Carol or end her friendship with Bob.

The three conditions are modular in the following sense, where the schemas an amendment *keeps* are those of C it does not delete.

Theorem 46 (What the check preserves). *Let C be syntactically grassroots.*

(1) If C' amends C, then C' is syntactically grassroots if and only if C' has an unobstructed introductory act, every atom an added schema adds meets Definition 11 in C' with E the predicates of Σ' other than environment predicates, every added schema meets Definition 13 in C', and every kept schema with no guarding role and a reading role has signature atoms in C'.

(2) If C' is obtained from C by deleting schemas and by replacing schemas τ by schemas τ' with the name and the head of τ and a stronger body, the guarding roles of τ' those of τ and agent roles

that are no roles of τ, every other role of τ' that is not one of τ an agent role that is a reading role and τ then having a guarding agent role, then C' is syntactically grassroots if and only if C' has an unobstructed introductory act and every such added reading role has an edge to a guarding role from an atom that guarding role requires.

(3) If C' instantiates C, then C' is syntactically grassroots.

(4) The lift of C at any set of its predicates is syntactically grassroots; so C' extending C at it is syntactically grassroots if and only if it meets the conditions of clause 1 as an amendment of that lift deleting nothing.

(5) Clauses 1 to 3 hold with 'has an introduction with wards, traceable provenance and volition' (Section 8) for 'is syntactically grassroots', of C and of C', and 'an introduction with wards' for 'an unobstructed introductory act'.

Implementable contracts. A contract written at the level of its clauses may be beyond what a compiler realises: vGLP's compiler [47] realises a schema with no guard only where one machine's state names the whole act. A contract is *implementable* if, in each schema with no guarding role, the atoms its first role requires name every other role and every variable of the atoms it adds and deletes; this is decidable on the schemas. The abstraction level of a contract is not restricted by it: an implementable contract restricts, or implements, one that is not. A contract for feeds and followers, in preparation, delivers to Alice every item of Carol's feed that Bob holds, Alice following Carol's feed through Bob, by a schema with no guard in which Alice's state names Bob and the feed and not the item, and Bob's names the item and not Alice; its implementable restriction requires that Bob know everyone who points to him, and so Alice, and restricts it by (a) and then (c), an invariant of the restriction ensuring that Bob knows Alice.

The proofs are in Appendix B. Provenance and Volition are conditions on each schema given the predicates of the contract, and hold of a kept schema in every amendment, but for signature atoms, which an added unguarded schema may take; Introduction is the one condition on the whole contract, since a schema added anywhere may obstruct the introductory act. So an owner amending a certified contract checks the added schemas and the introductory act, an owner strengthening bodies or adding guards checks the introductory act and any reading role it adds, and a composition of two certified contracts is checked as an extension of either; the checker does all. The three conditions read of a predicate only its arity and the kinds of its arguments, so a parametric contract is checked once for all its instances, and the converse of clause 3 holds too; the conditions of Section 8 read no more of one, so the same holds of a contract with wards. Definition 41 and Proposition 45 ask of a contract traceable provenance and volition, which is what makes its realisation contained and well defined, and not that it be certified; so a certified contract may restrict one that is not. Accountable AI agents [80] instantiate GnW, compose the instance with GFP, both amending GSG, and restrict the composition by (a): in the composition any agent with an identity signs an item and forwards it, a ward with no guardian among them, so it has no guardianships (Definition 32), and the restriction adds to the body of each of the two acts that its agent can act. No clause of GFP or of GnW is amended, and signing is one act with one name for a person and for an AI agent. The relations are the inheritance, refinement and generics of programming, compared in Section 12.

10 LEGAL MODALITIES OF CONTRACT CLAUSES

The schemas also determine, for each clause, whether an agent can breach it and what record of it the contract leaves in another agent's state: its *legal modality*, its *modality* for short. The two contracts show it. A clause may be one no act of the contract can breach, one an agent may breach with the contract recording the breach, or one whose performance is the person's and not the machine's, of which the contract records the undertaking. The question is about every clause, not

only about those that describe no act. The three are what a realisation does with a clause, and are not kinds of obligation: a friendship kept on paper can be entered against an agent's will, and under the schemas of Section 3.4 it cannot.

Throughout this section C is a syntactically grassroots contract. An agent joins the platform realising C by signing an *undertaking* to keep C towards all its other participants, which they present to every participant they interact with (Section 11).

Definition 47 (Clause). A *clause* of a contract C is a proposition about the conduct of its agents, and is *kept* or breached by that conduct; an agent the clause names as answerable is its *obligor*. The runs of the protocol realising C *determine* the clause if whether it is kept is a function of the run. A transaction of R_C *may breach* the clause if it is taken in some correct run and taking it breaches the clause.

Definition 48 (Witnessed). A volition-guarded transaction $(d \to d', Q')$ over participants Q is *witnessed against* $a \in \Pi$ if $d'_b \setminus d_b$ holds an atom naming a for some $b \in Q$ with $b \neq a$. A schema is *witnessed against* an agent role if every transaction it determines is witnessed against the agent in that role.

That atom is one the act put in the state of another agent, and by Lemma 27 an edge $b \to a$ of the recording graph of d' that d did not have. A record serves as evidence when the act put it there, it names the agent answerable, and someone else holds it.

Definition 49 (Enforced, Attested, Undertaken). Let φ be a clause of C, with obligor o where it has one. Then φ is

(1) *enforced* by C if the runs of the protocol realising C determine it and every correct run of that protocol keeps it;
(2) *attested* by C if some transaction of R_C may breach it and every transaction that may is witnessed against o;
(3) *undertaken* by C if no transaction of R_C may breach it, so that a breach, where the runs determine the clause, is an omission of o, and some transaction of R_C is witnessed against o or o's undertaking is held by another agent.

The three are exclusive and not exhaustive. Two kinds of clause have none. One that some transaction may breach without witnessing against the obligor can be breached invisibly. One that the runs neither determine nor record is owed by no one identifiable. A realisation does nothing with either.

An undertaken clause may say what its obligor undertakes in the contract's own terms, as a formula over runs: that whenever a condition on the agents' states holds, a response eventually holds, the *leads-to* of temporal logic [55, 64]. A clause of a restriction of a contract for distrust among friends, in preparation, reads "Alice, whose friend Bob distrusts Carol, will eventually distrust Carol or end her friendship with Bob": its formula is $\mathit{friend}(\mathit{Alice}, \mathit{Bob}) \wedge \mathit{distrusts}(\mathit{Bob}, \mathit{Carol}) \wedge \neg\mathit{distrusts}(\mathit{Alice}, \mathit{Carol}) \rightsquigarrow \mathit{distrusts}(\mathit{Alice}, \mathit{Carol}) \vee \neg\mathit{friend}(\mathit{Alice}, \mathit{Bob})$, and the acts by which Alice keeps it are the contract's.

Definition 50 (Undertaking formula). An *undertaking formula* of a clause of C with obligor o is $\varphi \rightsquigarrow \psi$, with φ and ψ quantifier-free formulas over atoms whose arguments are the clause's names, speech-act variables and name terms, the first argument of each atom its holder, universally quantified over the names and the variables. A run of the protocol realising C *keeps* it if, at every binding of the names and the variables, whenever φ holds at the machines of a configuration of the run, ψ holds at that configuration or a later one; otherwise o *breaches* the clause in the run.

A run that does not keep an undertaking formula is correct: the breach is the person's and no fault of the machines, and the formula adds nothing to liveness. The runs therefore determine the clause, no transaction may breach it, and it is undertaken; the theorems of a contract may be stated for the runs in which every person keeps their undertakings, and for those in which some do not. A formula says nothing to the three conditions, which do not read it.

A schema *names* its agent role π_j at another agent role π_i if an atom added at π_i names π_j.

Proposition 51 (Witnessing). *A schema is witnessed against every agent role it names at another agent role.*

A predicate e is *guarded* in C if whenever a schema of C adds an e-atom, the atom names only roles of that schema that will the act. The *consent clause* for e says that an agent's state holds an e-atom only if those the atom names willed the act that added it.

Proposition 52 (Enforcement by guarding). *If e is guarded in C, the consent clause for e is enforced by C.*

The clauses of the contracts. GKK:2, GKK:3 and GSG:2 are *enforced*, by the schemas requiring what the clauses require: a person knows another only by an act of both or through someone who knows them. GSG:1 is *attested*. Befriending is an act, and Proposition 51 says it is witnessed against both persons; so if Alice befriends Bob without trusting him, Bob holds the record of her breach. No act can breach the rest of the clause: nobody becomes a friend of Alice without her will (Proposition 52), and nobody befriends a person whose public key they do not hold. GSN:1 to GSN:3 permit acts and oblige nothing, unfriend ending every membership between the two at once, as GSN:3 says; and the delivery of GSN:4 is unguarded and so taken in every correct run in which it stays enabled, reaching every current member, a member when the leader's delivery to them is taken, since delivery to each member is an act of its own, taken after the message is posted: the runs determine GSN:1 to GSN:4 and every correct run keeps them, so they are *enforced*. GSG:3 is *undertaken*: ceasing to trust is no act of the contract, so no act can breach the clause and the runs do not determine it, and Alice's undertaking of Section 11, in Bob's hands, is its record. GFP:1 and GFP:2 are *enforced*: a person holds an item only as they signed it or as a friend signed it over to them, and Bob's $item(\text{sgn}(x, Alice))$ carries every signature. GKK:1 is *undertaken*: no act can breach the establishing of an identity, and being accountable for what is signed with a key is not an act of the contract, so the runs do not determine the clause, and the agent's undertaking of Section 11, which another agent holds, is its record.

The clauses of GC. GC:1 to GC:4 permit acts and oblige nothing, and the knowing GC:2 to GC:4 require the schemas require, so the runs determine them and every correct run keeps them: they are *enforced*. GC:5 is *undertaken*, a mutual credit line adding $¢(Bob)$ at Alice and $¢(Alice)$ at Bob and so being witnessed against both of them. The clause that a coin enters a holding only by its issuer's will has no modality, being breachable by pay without pay naming the payer.

Performance outside the system. GC:5 and GSG:1 rest on what is done outside the schemas, since selling at an advertised price is no act of the contract and neither is trusting a person, and they have different modalities, the first *undertaken* and the second *attested* (Section 10). The three modalities therefore do not divide by whether performance is digital.

Attestation and undertaking. The two require the same thing of the schemas, that the act naming the obligor leave the record in another agent's hands, and differ only in whether the breach is itself an act of the contract. The evidence depends on the schemas and the modality on the clause. A performance promised for later is never attested, because failing to perform is not an act; a

warranty given in the course of an act is attested exactly when that act is witnessed against the agent giving it.

Witnessing therefore depends on whether an effect names the agent performing the act, and not only the agents the atom concerns. Unfriend adds no atom, so no clause against ending a friendship at will can be attested under GSG; pay adds a coin naming its issuer and not its payer, so no clause against the payer can be attested under GC. A contract whose breaches are to be attested must have acts whose effects name their author. Signing provides it: an act taken as a digital speech act, an utterance signed with the key of the agent taking it, leaves an atom naming that agent, and the signed record is non-repudiable [12, 32].

11 SIGNING THE CONTRACT

A person joins a global platform by signing an End-User Licence Agreement with its operator. An agent joins a grassroots platform by signing a one-sided *undertaking* to keep its grassroots contract towards all its other participants, and presents it to every participant they interact with; the code realising the contract, or the platform hosting it, enforces this, refusing an interaction with a participant that has not presented one. The undertaking is a speech act, signed with the key of the agent giving it and non-repudiable [12, 32]. An agent signs by themselves, before their first interaction; where a participant invites them, the invitation, signed by the inviter, names the contract, and the invitee presents their undertaking first to the inviter.

Presentation is not part of the protocol: the code carries the undertaking in the messages of an interaction, and it leaves no atom in a state. Throughout this section C is a syntactically grassroots contract, $\mathcal{F}$ the protocol realising it and $Q \subset \Pi$; write $p \triangleright q$, for $p, q \in Q$, when p has presented its undertaking to q.

Definition 53 (Presenting run). A run of $\mathcal{F}(Q)$ with a relation $\triangleright$ on Q is *presenting* if $p \triangleright q$ and $q \triangleright p$ for every interaction of the run between p and q.

The code realising C, or the platform hosting it, keeps every run presenting.

Proposition 54 (Every participant has presented its undertaking). *In a presenting run, every agent of an instance of two or more agents has presented their undertaking to another agent of that instance.*

By Section 10 a record is evidence when it names the agent answerable and is held by someone else; the undertaking names its signer, and by the proposition another participant holds it. Condition (5) of Section 1 is therefore enforced by the code.

The code. Enforced clauses are enforced by the code the agents run: a breach of one is impossible with a correct realisation of the contract (Section 10), so a breach requires an agent to run other code. The code is warranted by its *creator*, a natural or legal person, and by its *compiler*. Each warranty is signed and carried with the code: the creator's, that the code realises the contract, carrying the contract's schemas and the checker's verdict that they are syntactically grassroots; the compiler's, that the compiled code is the creator's. Neither need have adopted the contract, so the warranties are not clauses of the contract and have no modality under Definition 49; a breach of either is provable from the warranted code and the contract by anyone holding both. In their undertaking an agent also promises to run the warranted code. This clause is *undertaken*: no act of the contract can breach it, no run shows whether an agent keeps it, and the undertaking, in another agent's hands, is its record. Where the platform hosting the code attests it, every interaction opening with proof of the application each agent runs, an application that runs only warranted code, the clause is *enforced* by the platform.

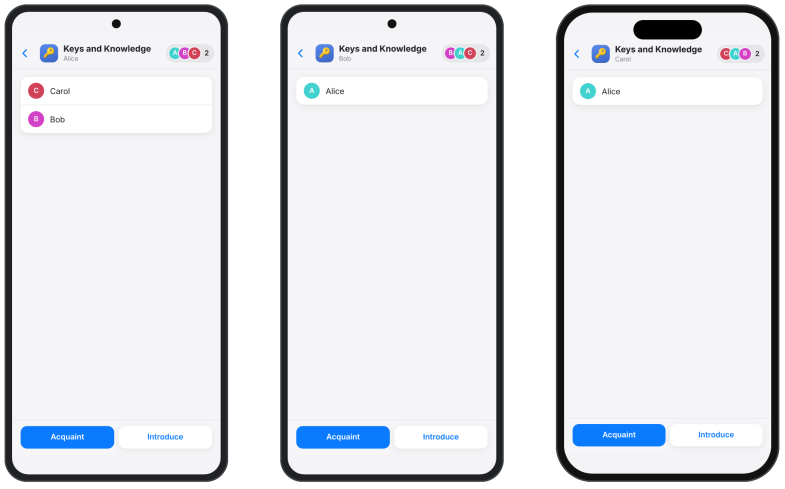


Fig. 6. An illustration of GKK as an app of GAS on three phones: Alice has become acquainted with Bob and with Carol (GKK:2), and Bob and Carol do not know each other. The screens are those of three agents, each with its own networking layer, run on one machine; the phones are drawn. Before their first message each agent presented their undertaking to the other.

GAS. GAS, the GLP app store of Section 1, makes an app of every contract the repository records down to its platform, and does what this section asks of the code. The code it warrants is the contract's own: the platform compiled from the contract's schemas and pragmas, and one person of it, which GAS's host code gives the person's name, network and screen, the day for a contract with a calendar, and nothing else. The compiler's warranty is the certificate GLP's compiler writes for every program it compiles: the compiler's key, the identity of the program's source and that of its compiled code, and the compiler's signature over the two. The compiler refuses a certificate to a program that reaches the network or the person other than through what its host gives it. The creator's warranty is the creator's signature over the identity of the source, a hash of the contract's schemas, the checker's verdict, and a hash of the host code that runs it. A device installs a contract only after compiling its code itself and checking the identities and signatures of both warranties, so it installs no contract whose own code is refused a certificate. The host code then loads the code the device compiled, which GLP's loader admits only as a program whose certificate checks, and runs it only where its compiler is the device's own and the creator's warranty is for the same source.

In GAS the undertaking names the contract's package, and with it the warranted code. An agent presents it to each agent before its first message to them, and the messages of an agent who has not presented theirs are held from the contract's code until they do; each agent keeps the undertakings presented to it. An invitation is the inviter's signed statement naming the package; the invitee checks it, keeps it, and presents their undertaking to the inviter first. On Android and the iPhone, where the phone's platform attests GAS itself, the platform attests each connection, naming the application at its other end, and GAS by default gives the contract's code the messages of a person only while a connection from them is attested as GAS; the clause that an agent runs the warranted code is then enforced, by the platform's attestation and GAS's own checks. Figure 6 illustrates GKK as an app of GAS.

Electronic signatures. An undertaking signed with an agent's key is an electronic signature. In the United States a signature, contract or record may not be denied legal effect, validity or enforceability solely because it is in electronic form [92]; in the European Union an electronic signature shall not be denied legal effect and admissibility as evidence solely on the grounds that it is in electronic form [21]; and in the United Kingdom an electronic signature, and its certification, are admissible in evidence on the authenticity and integrity of the data signed [91]. These statutes settle the form; whether an undertaking is enforceable is a question for the ordinary law of contract: the agents of a grassroots contract freely accept and sign it, as condition (5) requires, and its realisation keeps signed records of the acts that follow.

12 RELATED WORK

Each paragraph below is expanded in Appendix D.

Contracts as formal objects. The formal study of contracts in this field is dominated by the contract-as-automaton family — contract automata [5], a loan agreement as a finite automaton [24], the L4 language [93], timed contract automata [13], and encodings as defeasible deontic rules [36] and the comparison of imperative with declarative smart contracts [35]. All fix the parties when the contract is formed, keep one contract state, and assume an alphabet of events, a clock or a ledger, where our conditions quantify over the family of systems a contract induces on every finite set of people, need a local state per agent, and forbid a participant without a role in the act. Symboleo [63, 85] and Stipula [16] are the nearest to ours and make the same three commitments; Catala [56] does the corresponding work for statute. Befriending cannot be written in Stipula, which allows joint consent only in the constructor. We claim no new language: the conditions are on the four things a schema carries, which Stipula's functions and Symboleo's obligations also carry, given a semantics in which each agent holds its own state. BSPL [87] specifies an interaction protocol by message schemas between roles, with no global state: the local state of a role is the history of its messages, and a role may send a message once it knows the information the message requires, as a speech-act schema requires atoms at a role; a message passes between two roles, where an act may take any number of roles and the will of each.

Dynamic and temporal logic. A speech-act schema is a program of first-order dynamic logic [42], bind the variables, test the body and the wills, apply the head, and its denotation is that program's relation on states, with agent configurations as the states; a formula $[\tau]\varphi$ states what holds after an act. The body of a schema is a safe query of the relational calculus [15], with the aggregate of counting [1], and its head an update of the agents' states. Dynamic logic says nothing of runs: liveness, that an enabled act is eventually taken, and interactivity, that from every state some continuation interacts, are formulas of temporal logic [19, 64] over the transition system a contract denotes, and process logic [41] joins the two. Neither has persons with wills, nor obliviousness, which relates the systems of two sets of agents to that of their union; both are [51]'s.

Inheritance, refinement and generics. Amendment is the law's own word, and in programming extension is subclassing, a class adding methods to its parent's, and an amendment that rewrites a method is overriding. Behavioural subtyping [54] and design by contract [57] let a subclass weaken a method's precondition and strengthen its postcondition, so that it stands in for the parent, and so do the refinement calculus [6]; a restriction of Section 9 is CSP's refinement [44], fewer behaviours, every correct run of the restriction being, on the machines, one of the original, and its syntactic approximations strengthen preconditions, which none of them allows. Instantiation is parametric polymorphism [58], a contract generic in a predicate as a program is in a type, and the types of Grassroots Logic Programs [84] are the realisation's. In the specification calculi, including a Z schema in another [88] is extension and the B method's refinement [2] weakens preconditions as

the refinement calculus does; in algebraic specification [31] an extension that proves nothing new of the old symbols is conservative, which is what Theorem 46 shows of the check, and a theory morphism, renaming symbols with their signatures, is instantiation. Contract law has the words: an amendment modifies a contract's terms, a supplement adds to them, and a novation replaces the contract by another [3, §§89, 280]. None says what a syntactic check preserves; Theorem 46 does.

Norms and normative systems. Contracts are represented as defeasible rules with deontic operators, against which compliance is checked [34], and normative multi-agent systems concern how norms are operationalised and brought into conflict [9]; a guard [51] is a condition of enactment rather than a deontic operator. A social commitment [86] is of a debtor to a creditor in a context group, whose norms settle disputes between the two; an undertaken clause of Section 10 is a commitment of its obligor to every other participant, with no group above the agents, and another agent holds its record.

Juridical acts and declarative power. What we call an act is the *result-declaration* of Sartor [68], modelled by Hage [40], and a declarative power exercised jointly is a contract [28, 29]. There a proclamation has effect only if an institution provides for it [25, 28, 38, 45]; a grassroots contract has no institution, so an act's effect is the change in the agents' own states, and its guard takes the place of authorisation. An electronic institution [20] specifies the scenes in which agents in roles exchange speech acts, with institutional roles that external agents cannot play, and the institution keeps each agent's obligations; a grassroots contract has no institutional role, and the records of its acts are in the agents' own states.

Standard-form contracting. The End-User Licence Agreement is a *contract of adhesion*, treated by Radin [66], Kim [48] and Ben-Shahar and Schneider [7]; a grassroots contract has no drafting party, so the questions of that literature do not arise.

Terms a machine can evaluate. Surden [89], smart legal contracts [14] and enforceable security policies [69] each divide a contract's terms in two by what a realisation can do with them; Section 10 divides them in three, and the third class, a breach recorded against its obligor, is the one a remedy needs. Compliance monitors receive inputs from trusted observers [11, 60]; a monitor is a participant without a role in the act, and closure forbids one.

Code as a regulator. Lessig [49] argued that architecture regulates as code, and the digital social contract [12] and smart contracts [17] make the contract a program; a grassroots contract is not code, its conditions being checked on the text.

Self-governance. Ostrom [62] and commons scholarship [8, 43] concern communities governing a resource without an owner or a state, and platform regulation [22] binds an operator; a grassroots contract governs a relationship, and a platform constituted from its members' own devices [74, 76, 77] has no operator.

Grassroots systems. Grassroots protocols are due to [74] and were recast with volition-guarded transactions in [51]; platforms have been specified for social networking [75] and for personal cryptocurrencies [50, 73, 81]. Secure Scuttlebutt [90] is a peer-to-peer social network in which each person chooses their own key pair, signs each message of their own append-only log, and replicates the logs of those they follow, with no central authority; following takes no consent of the one followed, where befriending under GSG:1 takes the will of both. PGP's Web of Trust [94, 95] has every person as an introducer vouching for their friends' keys, no authority above them, and its authors call it arguably one of the first social networks; its trust is in a key, which PGP distinguishes from trust in the key's owner. GKK:3's introduction is PGP's introducer, and a friendship under GSG is trust in the owner. Added here are politeness, the legal text with decidable conditions on it, and what a realisation does with a clause.

Neighbouring instruments. A licence [27] presupposes an entitlement, bylaws an association, a code of conduct and terms of membership whoever runs the forum; a grassroots contract presupposes none. A Ricardian contract [37] is a document an issuer signs and offers to the holders of an instrument, readable by people and parsable by programs, whose hash every transaction record carries; a grassroots contract has no issuer, and each participant signs an undertaking to all the others (Section 11).

13 CONCLUSION

Proudhon's conditions on a social contract, with two more, formalised as conditions on its acts, give it the property he envisioned for it: any two agents may come into relation without anyone's leave, two instances formed independently can later become one, and every participant in an act takes a role in it. A grassroots contract is one meeting the conditions, and its realisation has all three.

Such a contract has two artefacts, a legal text and a protocol realising it, and the grassroots literature has had only the second. This paper translates the first into the second. A clause of the text describes an act, and the acts become speech-act schemas; the schemas denote volition-guarded transactions over a local-states function in which a state records exactly the agents its atoms name; and three conditions on the schemas make a contract *syntactically grassroots*, which is enough for the protocol realising it to be grassroots and volitionally grassroots. The semantics and the conditions are algorithmic and the conditions are decidable, so a contract can be certified from its text, and the only step left to judgement is the first, rendering a text as schemas. GAS's app maker takes a contract with its schemas, runs the checker on them, and makes a contract it certifies into an app, shared by a link.

Speech-act schemas have listed roles: the participants of a listed role are the agents one agent's state lists, as a group's leader lists its members, so closing a group is one act of the leader and of every member. A cleared atom deletes every atom of a state that it matches, so ending a friendship ends every membership between the two at once.

The same schemas determine what a realisation does with a clause, whether or not the clause describes an act: leave it unbreachable, record its breach against the agent in breach, or record the undertaking and who gave it. This depends on one thing: whether the act that names the agent answerable leaves the record in another agent's hands. Attestation and undertaking both require that of the schemas, and differ only in whether the breach is itself an act of the contract.

Both contracts are certified from their schemas. Who holds a record of whom is a directed graph, and an instance is a connected component of the undirected graph of interactions. In the social graph GSG the two coincide: acquaintance is the introductory act, a friend edge can only have been added by the two agents it joins, and the social graph is a web of trust whose instances are its connected components. A coin can be handed on and a message delivered to a group, so an edge may point at an agent its source has never transacted with; an edge is added either to a participant of the act that adds it or to an agent another participant of that act records already, and that condition on the schemas is enough for no edge ever to join two instances. For coins it is conservation of money.

The conditions also leave no participant without a role. Openness lets any agent start an instance, and without permission. Closure leaves an instance requiring no registry that issues identifiers, no log that orders acts, and no authority that timestamps them, each of which would take part in an act without a role in it. A community organised in this way requires no permission to form, no authority to operate, and no proprietor to govern it.

Three directions remain. The conditions are sufficient and not necessary, and a characterisation would say which contracts are grassroots. Which further invariants of a contract follow from its

schemas, as conservation of money does, is open. And the four things a schema carries are present, under other names, in the contract specification languages of Section 12; defining a semantics for one of them in which each agent holds their own state would let the conditions be checked on contracts already written in it.

REFERENCES

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases.* Addison-Wesley.

[2] Jean-Raymond Abrial. 1996. *The B-Book: Assigning Programs to Meanings.* Cambridge University Press.

[3] American Law Institute. 1981. *Restatement (Second) of Contracts.* American Law Institute Publishers, St. Paul, MN.

[4] J. L. Austin. 1962. *How to Do Things with Words.* Oxford University Press, Oxford.

[5] Shaun Azzopardi, Gordon J. Pace, Fernando Schapachnik, and Gerardo Schneider. 2016. Contract automata. *Artificial Intelligence and Law* 24, 3 (2016), 203–243. https://doi.org/10.1007/s10506-016-9185-2

[6] Ralph-Johan Back and Joakim von Wright. 1998. *Refinement Calculus: A Systematic Introduction.* Springer.

[7] Omri Ben-Shahar and Carl E. Schneider. 2014. *More Than You Wanted to Know: The Failure of Mandated Disclosure.* Princeton University Press, Princeton, NJ.

[8] Yochai Benkler. 2006. *The Wealth of Networks: How Social Production Transforms Markets and Freedom.* Yale University Press, New Haven, CT.

[9] Guido Boella, Leendert van der Torre, and Harko Verhagen. 2006. Introduction to Normative Multiagent Systems. *Computational & Mathematical Organization Theory* 12, 2 (2006), 71–79. https://doi.org/10.1007/s10588-006-9537-7

[10] Giulio Caldarelli. 2020. Understanding the Blockchain Oracle Problem: A Call for Action. *Information* 11, 11 (2020), 509. https://doi.org/10.3390/info11110509

[11] María-Emilia Cambronero, Luis Llana, and Gordon J. Pace. 2017. Timed Contract Compliance Under Event Timing Uncertainty. In *Legal Knowledge and Information Systems: JURIX 2017 (Frontiers in Artificial Intelligence and Applications, Vol. 302)*. IOS Press, 33–38.

[12] Luca Cardelli, Liav Orgad, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. 2020. Digital social contracts: A foundation for an egalitarian and just digital society. In *CEUR Proceedings of the First International Forum on Digital and Democracy*, Vol. 2781. CEUR-WS, 51–60. https://ceur-ws.org/Vol-2781/paper5.pdf

[13] Stefan Chircop, Gordon J. Pace, and Gerardo Schneider. 2022. An Automata-Based Formalism for Normative Documents with Real-Time. In *Legal Knowledge and Information Systems: JURIX 2022 (Frontiers in Artificial Intelligence and Applications, Vol. 362)*. IOS Press, 158–163. https://doi.org/10.3233/FAIA220460

[14] Christopher D. Clack, Vikram A. Bakshi, and Lee Braine. 2016. Smart Contract Templates: Foundations, Design Landscape and Research Directions. arXiv:1608.00771 https://arxiv.org/abs/1608.00771

[15] Edgar F. Codd. 1972. Relational Completeness of Data Base Sublanguages. In *Database Systems, Courant Computer Science Symposia Series 6*, Randall Rustin (Ed.). Prentice-Hall, 65–98.

[16] Silvia Crafa, Cosimo Laneve, and Giovanni Sartor. 2022. Pacta sunt servanda: Legal Contracts in Stipula. *Science of Computer Programming* (2022). https://arxiv.org/abs/2110.11069 arXiv:2110.11069.

[17] Primavera De Filippi, Chris Wray, and Giovanni Sileno. 2021. Smart contracts. *Internet Policy Review* 10, 2 (2021). https://doi.org/10.14763/2021.2.1549

[18] Ohad Eitan, Idit Keidar, and Ehud Shapiro. 2026. Securing People and their Machines Against Major Faults (Full Version). *Proceedings of The 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026), to appear* (2026). https://arxiv.org/abs/2607.02304 arXiv:2607.02304.

[19] E. Allen Emerson. 1990. Temporal and modal logic. In *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*, Jan van Leeuwen (Ed.). Elsevier, 995–1072.

[20] Marc Esteva, Juan A. Rodríguez-Aguilar, Carles Sierra, Pere Garcia, and Josep Lluís Arcos. 2001. On the Formal Specification of Electronic Institutions. In *Agent-Mediated Electronic Commerce (AMEC) (LNCS, Vol. 1991)*. Springer, 126–147.

[21] European Parliament and Council. 2014. Regulation (EU) No 910/2014 on Electronic Identification and Trust Services for Electronic Transactions in the Internal Market (eIDAS). Official Journal of the European Union, L 257.

[22] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). OJ L 119, 4.5.2016. Recital 26: anonymous information is information which does not relate to an identified or identifiable natural person; Art. 4(1) defines personal data..

[23] Richard E. Fikes and Nils J. Nilsson. 1971. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artificial Intelligence* 2 (1971), 189–208.

[24] Mark D. Flood and Oliver R. Goodenough. 2022. Contract as automaton: representing a simple financial agreement in computational form. *Artificial Intelligence and Law* 30, 3 (2022), 391–416. https://doi.org/10.1007/s10506-021-09300-9

[25] Nicoletta Fornara, Francesco Viganò, Mario Verdicchio, and Marco Colombetti. 2008. Artificial institutions: a model of institutional reality for open multiagent systems. *Artificial Intelligence and Law* 16, 1 (2008), 89–105. https://doi.org/10.1007/s10506-007-9055-z

[26] Martin Fowler. 2025. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl. *MartinFowler.com* (15 October 2025). https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

[27] Free Software Foundation. 2007. GNU General Public License, Version 3. https://www.gnu.org/licenses/gpl-3.0.html.

[28] Jonathan Gelati, Guido Governatori, Antonino Rotolo, and Giovanni Sartor. 2002. Declarative Power, Representation, and Mandate. A Formal Analysis. In *Legal Knowledge and Information Systems: JURIX 2002 (Frontiers in Artificial Intelligence and Applications)*, Trevor Bench-Capon, Aspassia Daskalopulu, and Radboud Winkels (Eds.). IOS Press, 41–52.

[29] Jonathan Gelati, Antonino Rotolo, Giovanni Sartor, and Guido Governatori. 2004. Normative autonomy and normative co-ordination: Declarative power, representation, and mandate. *Artificial Intelligence and Law* 12, 1 (2004), 53–81. https://doi.org/10.1007/s10506-004-1922-2

[30] GitHub. 2025. Spec Kit: Toolkit for Spec-Driven Development. GitHub repository. Available at https://github.com/github/spec-kit.

[31] Joseph A. Goguen and Rod M. Burstall. 1992. Institutions: Abstract Model Theory for Specification and Programming. *J. ACM* 39, 1 (1992), 95–146.

[32] James Golike and Ehud Shapiro. 2026. Digital Speech Acts Retain Control of Copyright with People, Not Platforms. *arXiv preprint arXiv:2606.19263* (2026). https://arxiv.org/abs/2606.19263

[33] Google. 2024. Dart Programming Language. https://dart.dev/

[34] Guido Governatori. 2018. Practical Normative Reasoning with Defeasible Deontic Logic. In *Reasoning Web. Learning, Uncertainty, Streaming, and Scalability*. Lecture Notes in Computer Science, Vol. 11078. Springer, 1–25. https://doi.org/10.1007/978-3-030-00338-8_1

[35] Guido Governatori, Florian Idelberger, Zoran Milosevic, Regis Riveret, Giovanni Sartor, and Xiwei Xu. 2018. On legal contracts, imperative and declarative smart contracts, and blockchain systems. *Artificial Intelligence and Law* 26, 4 (2018), 377–409. https://doi.org/10.1007/s10506-018-9223-3

[36] Guido Governatori and Antonino Rotolo. 2004. Modelling Contracts Using RuleML. In *Legal Knowledge and Information Systems: JURIX 2004 (Frontiers in Artificial Intelligence and Applications)*, Thomas F. Gordon (Ed.). IOS Press, 141–150.

[37] Ian Grigg. 2004. The Ricardian Contract. In *First IEEE Workshop on Electronic Contracting*. IEEE.

[38] Davide Grossi, John-Jules Ch. Meyer, and Frank Dignum. 2005. Modal Logic Investigations in the Semantics of Counts-as. In *Proceedings of the 10th International Conference on Artificial Intelligence and Law (ICAIL '05)*. ACM, 1–9. https://doi.org/10.1145/1165485.1165487

[39] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovič, and Dragos-Adrian Seredinschi. 2019. The consensus number of a cryptocurrency. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. 307–316. https://doi.org/10.1145/3293611.3331589

[40] Jaap Hage. 2011. A model of juridical acts: part 1: the world of law. *Artificial Intelligence and Law* 19, 1 (2011), 23–48. https://doi.org/10.1007/s10506-011-9105-4

[41] David Harel, Dexter Kozen, and Rohit Parikh. 1982. Process logic: Expressiveness, decidability, completeness. *J. Comput. System Sci.* 25, 2 (1982), 144–170.

[42] David Harel, Dexter Kozen, and Jerzy Tiuryn. 2000. *Dynamic Logic*. MIT Press, Cambridge, MA.

[43] Charlotte Hess and Elinor Ostrom (Eds.). 2007. *Understanding Knowledge as a Commons: From Theory to Practice*. MIT Press, Cambridge, MA.

[44] C. A. R. Hoare. 1985. *Communicating Sequential Processes*. Prentice-Hall.

[45] Andrew J. I. Jones and Marek Sergot. 1996. A Formal Characterisation of Institutionalised Power. *Logic Journal of the IGPL* 4, 3 (1996), 427–443. https://doi.org/10.1093/jigpal/4.3.427

[46] Idit Keidar, Andrew Lewis-Pye, and Ehud Shapiro. 2025. Constitutional Consensus. *arXiv preprint arXiv:2505.19216* (2025). https://arxiv.org/abs/2505.19216

[47] Idit Keidar and Ehud Shapiro. 2026. Volition Elicitation: Operational Semantics for People and Their Machines. arXiv:2607.14138. arXiv:2607.14138 [cs.DC] https://arxiv.org/abs/2607.14138

[48] Nancy S. Kim. 2013. *Wrap Contracts: Foundations and Ramifications*. Oxford University Press, New York.

[49] Lawrence Lessig. 1999. *Code and Other Laws of Cyberspace*. Basic Books, New York.

[50] Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. 2023. Grassroots Flash: A Payment System for Grassroots Cryptocurrencies. *arXiv preprint arXiv:2309.13191* (2023). https://arxiv.org/abs/2309.13191

[51] Andy Lewis-Pye and Ehud Shapiro. 2026. Volition-Guarded Multiagent Atomic Transactions: Describing People and their Machines. *arXiv preprint arXiv:2604.25596* (2026). https://arxiv.org/abs/2604.25596

[52] Leonid Libkin. 2004. *Elements of Finite Model Theory*. Springer.

[53] Leonid Libkin and Limsoon Wong. 1997. Query Languages for Bags and Aggregate Functions. *J. Comput. System Sci.* 55, 2 (1997), 241–272. https://doi.org/10.1006/jcss.1997.1523

[54] Barbara H. Liskov and Jeannette M. Wing. 1994. A Behavioral Notion of Subtyping. *ACM Transactions on Programming Languages and Systems* 16, 6 (1994), 1811–1841. https://doi.org/10.1145/197320.197383

[55] Zohar Manna and Amir Pnueli. 1981. Verification of concurrent programs: temporal proof principles. In *Proc. Workshop on Logics of Programs*, Dexter Kozen (Ed.). Lecture Notes in Computer Science, Vol. 131. Springer, 200–252.

[56] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: A Programming Language for the Law. *Proceedings of the ACM on Programming Languages* 5, ICFP (2021). https://doi.org/10.1145/3473582

[57] Bertrand Meyer. 1997. *Object-Oriented Software Construction* (2nd ed.). Prentice Hall. Introduces Design by Contract.

[58] Robin Milner. 1978. A Theory of Type Polymorphism in Programming. *J. Comput. System Sci.* 17, 3 (1978), 348–375. https://doi.org/10.1016/0022-0000(78)90014-4

[59] Marvin L. Minsky. 1967. *Computation: Finite and Infinite Machines.* Prentice-Hall, Englewood Cliffs, NJ.

[60] Sanjay Modgil, Nir Oren, Noura Faci, Felipe Meneguzzi, Simon Miles, and Michael Luck. 2015. Monitoring compliance with E-contracts and norms. *Artificial Intelligence and Law* 23, 2 (2015), 161–196. https://doi.org/10.1007/s10506-015-9167-9

[61] Ugo Montanari and Francesca Rossi. 1995. Contextual nets. *Acta Informatica* 32, 6 (1995), 545–596.

[62] Elinor Ostrom. 1990. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, Cambridge.

[63] Alireza Parvizimosaed, Sepehr Sharifi, Daniel Amyot, Luigi Logrippo, Marco Roveri, Aidin Rasti, Ali Roudak, and John Mylopoulos. 2022. Specification and Analysis of Legal Contracts with Symboleo. *Software and Systems Modeling* 21 (2022). https://doi.org/10.1007/s10270-022-01053-6

[64] Amir Pnueli. 1977. The temporal logic of programs. In *Proc. 18th Symp. Foundations of Computer Science*. IEEE, 46–57.

[65] Pierre-Joseph Proudhon and John Beverley Robinson. 2004. *General idea of the revolution in the nineteenth century.* Courier Corporation. Original work published 1851.

[66] Margaret Jane Radin. 2013. *Boilerplate: The Fine Print, Vanishing Rights, and the Rule of Law.* Princeton University Press, Princeton, NJ.

[67] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. Vibe Coding vs. Agentic Coding: Fundamentals and Practical Implications of Agentic AI. https://arxiv.org/abs/2505.19443 arXiv:2505.19443.

[68] Giovanni Sartor. 2006. Fundamental legal concepts: A formal and teleological characterisation. *Artificial Intelligence and Law* 14, 1 (2006), 101–142. https://doi.org/10.1007/s10506-006-9009-x

[69] Fred B. Schneider. 2000. Enforceable Security Policies. *ACM Transactions on Information and System Security* 3, 1 (2000), 30–50.

[70] John R. Searle. 1979. *Expression and Meaning: Studies in the Theory of Speech Acts.* Cambridge University Press, Cambridge.

[71] John R Searle and John Rogers Searle. 1969. *Speech acts: An essay in the philosophy of language*. Vol. 626. Cambridge university press.

[72] Ehud Shapiro. 2021. Multiagent Transition Systems for Composing Fault-Resilient Protocol Stacks. *arXiv preprint arXiv:2112.13650* (2021). https://arxiv.org/abs/2112.13650

[73] Ehud Shapiro. 2022. Grassroots Currencies: Turning Mutual Trust into Liquidity. *arXiv preprint arXiv:2202.05619* (2022). https://arxiv.org/abs/2202.05619

[74] Ehud Shapiro. 2023. Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391).* LIPICS, Italy, 47:1–47:7. https://arxiv.org/abs/2301.04391

[75] Ehud Shapiro. 2023. Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp. In *OASIS '23* (Rome, Italy). Association for Computing Machinery. https://doi.org/10.1145/3599696.3612898

[76] Ehud Shapiro. 2024. A Grassroots Architecture to Supplant Global Digital Platforms by a Global Digital Democracy. *arXiv:2404.13468, Proceedings of DAWO'24* (2024). https://arxiv.org/abs/2404.13468

[77] Ehud Shapiro. 2025. Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots. *arXiv preprint arXiv:2511.03286* (2025). https://arxiv.org/abs/2511.03286

[78] Ehud Shapiro. 2025. GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133* (2025). https://arxiv.org/abs/2510.15747

[79] Ehud Shapiro. 2026. Child-Safe Social Networking: From a Grassroots Social Contract to an App. *In preparation* (2026).

[80] Ehud Shapiro. 2026. Corralling AI Risks with Grassroots Social Contracts. *In preparation* (2026).

[81] Ehud Shapiro. 2026. Grassroots Bonds: Financing by the People, for the People. arXiv:2603.13671. arXiv:2603.13671 https://arxiv.org/abs/2603.13671

[82] Ehud Shapiro. 2026. Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations. In *Proceedings of the 27th International Conference on Distributed Computing and Networking*. 71–81. https://doi.org/10.1145/3772290.3772309 arXiv preprint arXiv:2502.11299.
[83] Ehud Shapiro. 2026. Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI (Full Version). *arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26* (2026). https://arxiv.org/abs/2602.06934
[84] Ehud Shapiro. 2026. Types for Grassroots Logic Programs (Full version). *arXiv preprint arXiv:2601.17957* (2026). https://arxiv.org/abs/2601.17957
[85] Sepehr Sharifi, Alireza Parvizimosaed, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. 2020. Symboleo: Towards a Specification Language for Legal Contracts. In *IEEE 28th International Requirements Engineering Conference (RE)*. IEEE, 364–369. https://doi.org/10.1109/RE48521.2020.00049
[86] Munindar P. Singh. 1999. An Ontology for Commitments in Multiagent Systems: Toward a Unification of Normative Concepts. *Artificial Intelligence and Law* 7 (1999), 97–113.
[87] Munindar P. Singh. 2011. Information-Driven Interaction-Oriented Programming: BSPL, the Blindingly Simple Protocol Language. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2011)*. International Foundation for Autonomous Agents and Multiagent Systems, Taipei, Taiwan.
[88] J. Michael Spivey. 1992. *The Z Notation: A Reference Manual* (2nd ed.). Prentice Hall.
[89] Harry Surden. 2012. Computable Contracts. *UC Davis Law Review* 46 (2012), 629.
[90] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. 2019. Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM conference on information-centric networking*. 1–11. https://doi.org/10.1145/3357150.3357396
[91] United Kingdom Parliament. 2000. Electronic Communications Act 2000. c. 7, s. 7, as amended by the Electronic Identification and Trust Services for Electronic Transactions Regulations 2016.
[92] United States Congress. 2000. Electronic Signatures in Global and National Commerce Act. Public Law 106–229, 114 Stat. 464.
[93] Seng Joe Watt, Oliver Goodenough, and Meng Weng Wong. 2023. Deontics and Time in Contracts: An Executable Semantics for the L4 DSL. In *Legal Knowledge and Information Systems: JURIX 2023 (Frontiers in Artificial Intelligence and Applications)*. IOS Press, 119–124. https://doi.org/10.3233/FAIA230954
[94] Philip Zimmermann. 1994. PGP User's Guide, Volume I: Essential Topics. PGP version 2.6.2, revised 11 October 1994; published as The Official PGP User's Guide, MIT Press, 1995.
[95] Philip Zimmermann and Jon Callas. 2009. The Evolution of PGP's Web of Trust. In *Beautiful Security*. O'Reilly Media, Chapter 7, 107–130.

A THE FRAMEWORK OF VOLITION-GUARDED MULTIAGENT ATOMIC TRANSACTIONS

This appendix states formally what Section 4 recalls in brief, so that the statements and proofs of Section 6 can be read without recourse to another paper. Everything in it is from [51], and the proofs of its results are there and are not repeated. It leaves out the volition-guarded if-then-else transactions of [51], which compiled schemas do not produce.

Earlier work introduced multiagent transition systems [72], grassroots protocols and platforms [74], and their definition via multiagent atomic transactions [82], which describe the behaviour of machines but not of the people operating them. A volition-guarded transaction is a machine transaction guarded by the volitions of some, all, or none of the agents whose machines participate; an agent's volitional state is a set of equivalence classes of machine transactions they are willing their machine to take part in. The social graph illustrates the two extremes: befriending is guarded by both agents, while unfriending is guarded by either. An agent may freely change their volitional state; an equivalence class is removed from every agent's volitional state when a machine transaction in that class is taken, the will having been fulfilled.

A.1 Agents, Machines and Volition-Guarded Transactions

We assume a countable, potentially infinite set of *agents* Π, an agent being a person operating a machine, but consider only finite subsets of it, so when referring to a particular set of agents $P \subset \Pi$ we assume P to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when

equality is also possible, and use $p \neq q \in P$ as a shorthand for $p \in P \wedge q \in P \wedge p \neq q$. As standard, we use S^P to denote the set of all total functions from P to S, and if $c \in S^P$ we use c_p to denote the value of c at $p \in P$.

Definition 55 (Machine State, Configuration, Transaction, Volition-Guarded Transaction [51]). Given an arbitrary set S of *machine states*, with a designated *initial state* $s_0 \in S$, and agents $Q \subset \Pi$, a *machine configuration* over Q is a member of S^Q, and a *machine transaction* over *participants* Q is a pair $c \rightarrow c' \in (S^Q)^2$ such that $c \neq c'$; it is *unary* if $|Q| = 1$. Given such a machine transaction t, a *volition-guarded multiagent atomic transaction* over t—henceforth, *volition-guarded transaction*—is a pair (t, Q') where $Q' \subseteq Q$ are its *guards*.

Machine transactions are atomic and asynchronous [72]: they can be carried out by their participants at any time, regardless of the states of non-participants. Participants include both active agents, whose state changes, and stationary agents, whose state is a precondition but does not change. Volition-guarded transactions can be carried out only if their guards $Q' \subseteq Q$ are willing, and do not distinguish between agents that initiate a transaction and those willing to take part in it. When we say a transaction is guarded by $\{p, q\}$, both must be willing; when we say it is guarded by either p or q, we mean there are two volition-guarded transactions over the same machine transaction, $(t, \{p\})$ and $(t, \{q\})$, so that either agent's volition suffices.

Distinct machine transactions can represent the same act in different configurations; a transaction equivalence relates them.

Definition 56 (Transaction Equivalence [51]). Given a set of machine transactions R, a *transaction equivalence* is an equivalence relation $\sim$ on R such that $t \sim t'$ implies t and t' have the same participants, and $R/\sim$ is countable. We write $[t]$ for the equivalence class of t under $\sim$.

All befriend transactions of p and q, differing only in the configurations in which they occur, form one equivalence class. An agent wills a class, and taking one member of it fulfils that will.

Definition 57 (Agent State and Configuration [51]). Given agents P, states S with initial state s_0, a set of machine transactions T each over its own participants $Q \subseteq P$ and S, and equivalence $\sim$ on T, an *agent state* is a pair $(V, m) \in \mathcal{A} = (2^{T/\sim} \times S)$ where V is its *volitional state* and $m \in S$ its *machine state*. The *initial agent state* is $(\emptyset, s_0)$. An *agent configuration* c over P, S, T, and $\sim$ is a member $c \in \mathcal{A}^P$ in which $c^v_p \subseteq (T/\sim)_p$ for every $p \in P$, where $(T/\sim)_p$ denotes the classes in $T/\sim$ in which p is a participant; we write c^v_p for the volitional state and c^m_p for the machine state of agent p in c.

Definition 58 (Volitional Transaction [51]). Given agents P, states S, machine transactions T over P and S, and equivalence $\sim$ on T:

(1) A *volition change of agent* $p \in P$ is a pair $c \rightarrow c'$ of agent configurations over $\{p\}$, S, T, and $\sim$ such that $c^v_p, {c'}^v_p \subseteq (T/\sim)_p$ and $c^v_p \neq {c'}^v_p$, and $c^m_p = {c'}^m_p$.
(2) A *volitional machine transaction* induced by a volition-guarded transaction (t, Q'), for some $t = (d \rightarrow d') \in T$ over $Q \subseteq P$ with guards $Q' \subseteq Q$, is a pair $c \rightarrow c'$ where $c \neq c'$ are agent configurations over P, S, T, and $\sim$ such that $[t] \in c^v_q$ for every $q \in Q'$; $c^m_p = d_p$ and ${c'}^m_p = d'_p$ for every $p \in Q$; $c^m_p = {c'}^m_p$ for every $p \in P \setminus Q$; and ${c'}^v_p = c^v_p \setminus \{[t]\}$ for every $p \in P$.
(3) A *volitional transaction* is a volition change or a volitional machine transaction.

When a volitional machine transaction induced by (t, Q') is taken, the class $[t]$ is removed from every agent's volitional state: the will is fulfilled by any equivalent transaction. An agent may independently change their volitional state via volition changes, which may add or remove classes; beyond these, the framework removes a class from c^v_p only upon fulfilment.

A.2 Transition Systems and the Induced Volitional System

Definition 59 (Transition System, Computation, Run, Safe, Live, Correct [51]). A *transition system* is a tuple $TS = (S, s_0, T, \sim)$, where:

(1) S is an arbitrary non-empty set, referred to as the set of *states*.
(2) Some $s_0 \in S$ is the designated *initial state*.
(3) $T \subseteq S^2$ is a set of *correct transitions over* S, where each transition $t \in T$ is a pair (s, s') of non-identical states $s \neq s' \in S$, also written as $t = s \longrightarrow s'$.
(4) $\sim$ is a *partial equivalence relation* on T: a symmetric and transitive relation on T, not necessarily reflexive. Its domain $\{t \in T : t \sim t\}$ is partitioned into countably many *liveness classes* $T/\sim$; a transition outside the domain belongs to no class.

A *computation* of TS is a nonempty, potentially infinite sequence of states $r = s_1, s_2, \cdots$; it is a *run* of TS if $s_1 = s_0$. A *prefix* of a computation is a finite initial segment of it. A computation $r = s_1, s_2, \ldots$ is *safe*, also written $r \subseteq T$, if $s_i \longrightarrow s_{i+1} \in T$ for every two consecutive states. A class $[t] \in T/\sim$ is *enabled* in a state s if $s \longrightarrow s' \in [t]$ for some $s' \in S$. A run r is *live* if no class $[t] \in T/\sim$ is enabled in every state of some suffix of r with no member of $[t]$ occurring in the suffix. A run is *correct* if it is safe and live.

A partial equivalence, rather than a total one, is used so that some transitions may carry no liveness requirement: only transitions in the domain of $\sim$ form classes and thereby incur a liveness obligation, while transitions outside the domain, belonging to no class, may occur in a correct run but are never required to.

Lemma 60 (Extension [51]). *Every finite safe run of a transition system is a prefix of a correct run of it.*

Definition 61 (Multiagent Transition System [51]). Given agents $P \subset \Pi$ and an arbitrary set S of *states* with a designated *initial state* $s_0 \in S$, a *multiagent transition system* over P and S is a transition system $TS = (C, c_0, T, \sim)$ with *configurations* $C := S^P$, *initial configuration* $c_0 := \{s_0\}^P$, *transitions* $T \subseteq C^2$ a set of transactions over P and S, and $\sim$ a partial equivalence on T.

Rather than specifying a multiagent transition system over a set of agents P directly, it is specified via machine transactions. A machine transaction over $Q \subseteq P$ defines a set of multiagent transitions over P in which all members of $P \setminus Q$ are stationary.

Definition 62 (Transaction Closure [51]). Let $P \subset \Pi$, S a set of machine states, and $C := S^P$. For any transition or transaction $t = c \longrightarrow c'$, we write $t_q := c_q \longrightarrow c'_q$ and say p is *stationary* in t if $c_p = c'_p$. For a machine transaction $t = (c \longrightarrow c')$ over S with participants Q, the *P-closure of t*, $t{\uparrow}P$, is the set of transitions over P and S defined by:

$$t{\uparrow}P := \begin{cases} \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge \forall p \in P \setminus Q.(p \text{ is stationary in } t')\} & \text{if } Q \subseteq P \\ \emptyset & \text{otherwise} \end{cases}$$

If R is a set of machine transactions, each $t \in R$ over some Q and S, then the *P-closure of R*, $R{\uparrow}P$, is the set of transitions over P and S defined by $R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$. Given a relation $\sim$ on R, its *P-closure* $\sim{\uparrow}P$ is the relation on $R{\uparrow}P$ with $\hat{t}\ (\sim{\uparrow}P)\ \hat{t}'$ iff $\hat{t} \in t{\uparrow}P$ and $\hat{t}' \in t'{\uparrow}P$ for some $t \sim t'$. A transition over P is *unary* if it lies in the P-closure of a unary transaction.

If distinct transactions in R have disjoint P-closures, as when every participant of every transaction in R changes state, then each transition in $R{\uparrow}P$ has a unique inducing transaction, $\sim{\uparrow}P$ relates two transitions exactly when their inducing transactions are $\sim$-related, and $\sim{\uparrow}P$ is a partial equivalence whenever $\sim$ is. For volition-guarded transactions, disjointness is required only of unguarded ones of different classes (Definition 63).

Definition 63 (Volitional Multiagent Transition System [51]). Given agents $P \subset \Pi$, machine states S with initial state s_0, a set R of volition-guarded transactions such that every $(t, Q') \in R$ has the participants of t contained in P, and an equivalence $\sim$ on the set $T_R := \{t : (t, Q') \in R \text{ for some } Q'\}$ of underlying machine transactions such that two of them of different classes, each underlying an unguarded transaction of R, have disjoint P-closures, the *volitional multiagent transition system induced by* $(S, R, \sim)$ *over* P is the multiagent transition system $(\mathcal{A}^P, c_0, T_V, \sim_V)$ where:

(1) $\mathcal{A} := 2^{T_R/\sim} \times S$ is the *agent state space*;
(2) $c_0 \in \mathcal{A}^P$ is the *initial agent configuration*, with ${c_0}_p^v = \emptyset$ and ${c_0}_p^m = s_0$ for every $p \in P$;
(3) T_V consists of all transitions $e \to e' \in (\mathcal{A}^P)^2$ of one of two forms: (*i*) a *volition change* of some $p \in P$—$e_p^v, {e'}_p^v \subseteq (T_R/\sim)_p$ and $e_p^v \neq {e'}_p^v$, $e_p^m = {e'}_p^m$, and $e_r = e'_r$ for every $r \in P \setminus \{p\}$; or (*ii*) a *volitional machine transaction* induced by some volition-guarded transaction $(t, Q') \in R$ per Definition 58(2);
(4) $\sim_V$ is the restriction of $\sim\uparrow P$ (Definition 62) to the volitional machine transactions, relating two of them whenever their inducing machine transactions are $\sim$-equivalent, and leaves every volition-change transition outside its domain, so volition changes belong to no class and carry no liveness obligation.

Every volitional machine transaction has $\sim$-equivalent inducing machine transactions, so $\sim_V$ is a partial equivalence. A step induced by a guarded transaction (t, Q'), $Q' \neq \emptyset$, removes $[t]$ from the volitional state of each guard, which held it, and a step of another class removes another class, so two guarded transactions of different classes may share their P-closures; two unguarded transactions of different classes have no transition in common, by the condition on P-closures.

Definition 64 (Enabled [51]). Given a set of volition-guarded transactions, each (t, Q') with $t = d \to d'$ a machine transaction over some $Q \subseteq P$ and S with guards $Q' \subseteq Q$, and an equivalence $\sim$ on machine transactions: the volition-guarded transaction (t, Q') is *machine-enabled* in agent configuration c over P if $c_p^m = d_p$ for every $p \in Q$, and *enabled* if in addition $[t] \in c_q^v$ for every $q \in Q'$. An equivalence class $[t]$ is *enabled* in c if some volition-guarded transaction (t', Q') with $t' \in [t]$ is enabled in c.

A volition-guarded transaction with an empty guard requires no volitions and is enabled whenever its machine precondition is met. A class of volitional machine transactions is enabled at a configuration in the sense of Definition 59 exactly when some volition-guarded transaction inducing it is enabled in the above sense, and this determines which runs are live and correct. Volition changes belong to no class and so impose no liveness obligation; personal choices remain free.

A.3 Protocols, Grassroots Protocols and Coalescence

A protocol is a family of multiagent transition systems, one for each set of agents $P \subset \Pi$, which share an underlying set of local machine states $\mathcal{S}$ with a designated initial state s_0. A *local-states function* maps every set of agents $P \subset \Pi$ to a set of local machine states $S(P) \subset \mathcal{S}$ that includes s_0 and satisfies $P \subset P' \subset \Pi \implies S(P) \subset S(P')$.

Definition 65 (Protocol [51]). A *protocol* $\mathcal{F}$ over a local-states function S is a family of multiagent transition systems that has exactly one transition system $\mathcal{F}(P) = (C(P), c_0(P), T(P), \sim(P))$ for every $P \subset \Pi$, with *agent states* $\mathcal{A}(P)$, configurations $C(P) := \mathcal{A}(P)^P$, initial configuration $c_0(P) \in C(P)$, and partial equivalence $\sim(P)$ on $T(P)$ determined by the protocol, such that $P \subseteq P' \subset \Pi$ implies $\mathcal{A}(P) \subseteq \mathcal{A}(P')$ and $c_0(P)_p = c_0(P')_p$ for every $p \in P$.

In a grassroots protocol two disjoint sets of agents can each operate independently — their interleaved correct runs are correct runs of the combined system — yet the combined system offers behaviours that neither set could produce by itself.

Definition 66 (Interleaving [51]). Let $P, P' \subset \Pi$ be disjoint nonempty sets of agents, $r = c_0, c_1, \ldots$ a run of $\mathcal{F}(P)$, and $r' = d_0, d_1, \ldots$ a run of $\mathcal{F}(P')$. An *interleaving* of r and r' is a sequence $e_0, e_1, \ldots$ of configurations in $C(P \cup P')$ for which there exist non-decreasing sequences of indices $(i_k)_{k\geq 0}$ and $(j_k)_{k\geq 0}$ with $i_0 = j_0 = 0$ such that for every $k \geq 0$:

(1) $(e_k)_p = (c_{i_k})_p$ for every $p \in P$,
(2) $(e_k)_q = (d_{j_k})_q$ for every $q \in P'$,
(3) if e_{k+1} exists, then exactly one of: (*i*) $i_{k+1} = i_k + 1$ and $j_{k+1} = j_k$, a P-step, or (*ii*) $i_{k+1} = i_k$ and $j_{k+1} = j_k + 1$, a P'-step.

Moreover, if r is finite of length n then $i_k = n$ for some k, and if r is infinite then for every $m \geq 0$ there is a k with $i_k = m$; likewise for r' and (j_k).

An interleaving is well defined: by Definition 65, $\mathcal{A}(P) \subseteq \mathcal{A}(P \cup P')$ and $\mathcal{A}(P') \subseteq \mathcal{A}(P \cup P')$, so each e_k, with p-components in $\mathcal{A}(P)$ and q-components in $\mathcal{A}(P')$, is a configuration in $C(P \cup P')$. Also $e_0 = c_0(P \cup P')$, by the agreement of initial configurations across $\mathcal{F}(P)$, $\mathcal{F}(P')$ and $\mathcal{F}(P \cup P')$.

Definition 67 (Interaction, Interactive Run, First Interaction [51]). Let $Q \subset \Pi$. For a configuration c over Q and $Q' \subseteq Q$, write $c|_{Q'}$ for the restriction of c to Q'. A transition $e \to e'$ of $\mathcal{F}(Q)$ is an *interaction between* $x \neq y \in Q$ if $e|_{\{x\}} \neq e'|_{\{x\}}$ and $e|_{\{y\}} \neq e'|_{\{y\}}$, and $e|_{\{x\}} \to e'|_{\{x\}}$ is not a transition of $\mathcal{F}(\{x\})$ or $e|_{\{y\}} \to e'|_{\{y\}}$ is not a transition of $\mathcal{F}(\{y\})$. It is an *interaction between* disjoint nonempty $P, P' \subseteq Q$ if it is an interaction between some $x \in P$ and some $y \in P'$. A run $\hat{r}$ of $\mathcal{F}(Q)$ is *interactive between P and P'* if some transition of $\hat{r}$ is an interaction between them; the *first interaction* of P and P' in $\hat{r}$ is the first such transition.

Definition 68 (Oblivious, Interactive, Grassroots [51]). A protocol $\mathcal{F}$ is:

(1) *oblivious* if for every disjoint nonempty $P, P' \subset \Pi$, every interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a correct run of $\mathcal{F}(P \cup P')$.
(2) *interactive* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P' is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.
(3) *grassroots* if it is oblivious and interactive.

Being oblivious means two disjoint sets of agents coexist without interference. Being interactive means two sets that have not yet interacted can still do so, and the condition is required at every such point and not at the initial configuration only, so a protocol whose sets of agents can reach a state from which they can never couple is not interactive.

Definition 69 (Interaction Graph, Instance, Coalescence [51]). Let $\mathcal{F}$ be a protocol, $Q \subset \Pi$, and r a prefix of a run of $\mathcal{F}(Q)$. The *interaction graph* of r has vertices Q and an edge between $a \neq b \in Q$ whenever some transition of r is an interaction between a and b. A nonempty $P \subseteq Q$ is an *instance* in r if it is a connected component of the interaction graph of r. Disjoint nonempty $P, P' \subseteq Q$ *coalesce* in a run $\hat{r}$ of $\mathcal{F}(Q)$ if they are instances in some prefix of $\hat{r}$ and lie in one component of the interaction graph of a longer prefix of $\hat{r}$.

Proposition 70 (Coalescence [51]). *Let $\mathcal{F}$ be a grassroots protocol, $Q \subset \Pi$, and $P, P' \subseteq Q$ disjoint instances in a prefix r of a correct run of $\mathcal{F}(Q)$. Then r is a prefix of a correct run of $\mathcal{F}(Q)$ in which P and P' coalesce.*

A.4 Volitional Transactions-Based Protocols

A volitional transactions-based protocol assigns to each set of agents the volitional multiagent transition system induced by its volition-guarded transactions.

Definition 71 (Transactions Over a Local-States Function [51]). Let S be a local-states function. A set of transactions R is *over* S if every transaction $t \in R$ is a multiagent transition over Q and $S(P')$ for some $Q \subseteq P' \subset \Pi$. Given such a set R and $P \subset \Pi$, $R(P) := \{t \in R : t \text{ is over } Q \text{ and } S(P'), Q \subseteq P' \subseteq P\}$.

Definition 72 (Volitional Transactions-Based Protocol [51]). Let S be a local-states function and R a set of volition-guarded transactions over S with equivalence $\sim$. The *protocol $\mathcal{F}$ over R, S, and $\sim$* assigns to each set of agents $P \subset \Pi$ the volitional multiagent transition system $\mathcal{F}(P)$ induced by $(S(P), R(P), \sim)$ over P (Definition 63). In particular, $C(P) = \mathcal{A}(P)^P$ with agent state space $\mathcal{A}(P) := 2^{T_{R(P)}/\sim} \times S(P)$, monotone in P as Definition 65 requires, and $c_0(P)$ has p-component $(\emptyset, s_0)$ for every $p \in P$.

Throughout, $\sim$ is an equivalence on T_R, the machine transactions underlying the whole set R; for $T' \subseteq T_R$ we write $T'/\sim$ for the set of $\sim$-classes with a representative in T', and $[t]$ for the class of t in $T_R/\sim$. Hence $P \subseteq P'$ implies $T_{R(P)}/\sim \subseteq T_{R(P')}/\sim$.

Definition 73 (Contained [51]). A set R of volition-guarded transactions over a local-states function S is *contained* if for every $P \subset \Pi$, every $(t, Q') \in R$ with $t = c \to c'$ over participants $Q \subseteq P$ and $c \in S(P)^Q$ is in $R(P)$.

The three results below are used in the proofs of Section 6. The first is the invariant that in $\mathcal{F}(P)$ an agent wills only transactions over agents of P, so the guard of a transaction that reaches outside P cannot will it while P runs by itself.

Lemma 74 (Volitional Containment [51]). *Let $\mathcal{F}$ be a volitional transactions-based protocol over a set of volition-guarded transactions R with equivalence $\sim$. For every $P \subset \Pi$, every configuration c of $\mathcal{F}(P)$, and every $p \in P$: $c_p^v \subseteq T_{R(P)}/\sim$, where $T_{R(P)} := \{t : (t, Q') \in R(P) \text{ for some } Q'\}$.*

Lemma 75 (Interleaving Safety [51]). *Let $\mathcal{F}$ be a volitional transactions-based protocol and $P, P' \subset \Pi$ disjoint and nonempty. Every finite prefix of an interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a finite safe run of $\mathcal{F}(P \cup P')$.*

Proposition 76 ([51]). *A volitional transactions-based protocol over a contained set of volition-guarded transactions is oblivious provided that for every disjoint nonempty $P, P' \subset \Pi$, no equivalence class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$.*

A person may die or become incapacitated, and an AI agent operating a machine may be retired; their will then has no further effect. An agent has left when no act they guard can be enabled again.

Definition 77 (Left [51]). Let $\mathcal{F}$ be a volitional transactions-based protocol over R, $Q \subset \Pi$, c a configuration of $\mathcal{F}(Q)$, and $x \in Q$. Then x *has left* in c if no $(t, Q') \in R(Q)$ with $x \in Q'$ is machine-enabled in any configuration c' of $\mathcal{F}(Q)$ with $c \xrightarrow{*} c' \subseteq T(Q)$.

Definition 78 (Interactive Among the Agents That Have Not Left [51]). A volitional transactions-based protocol $\mathcal{F}$ is *interactive among the agents that have not left* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', in whose last configuration some agent of P and some agent of P' have not left, is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.

An interactive protocol is interactive among the agents that have not left, the latter requiring a continuation of fewer prefixes.

In a run of $\mathcal{F}(P \cup P')$ an agent of P may will a transaction reaching P', which a run of $\mathcal{F}(P)$ forbids; the volitional notion constrains the resulting coupling.

Definition 79 (Volitionally Grassroots [51]). A volitional transactions-based protocol $\mathcal{F}$ is *volitionally grassroots* if it is oblivious, interactive among the agents that have not left, and, for every disjoint nonempty $P, P' \subset \Pi$ and every safe run of $\mathcal{F}(P \cup P')$ interactive between P and P', the first interaction of P and P' is induced by a volition-guarded transaction (t, Q') with $Q' \cap P \neq \emptyset$ and $Q' \cap P' \neq \emptyset$.

A grassroots protocol meeting the condition on first interactions is volitionally grassroots.

B PROOFS

The statements of Sections 3, 5, 6, 7, 9, 10 and 11 are restated and proved here in order, with the three lemmas and the one definition used only in these proofs. Throughout, C is a syntactically grassroots contract, $\mathcal{F}$ the protocol realising it and $P \subset \Pi$. An atom a transition adds, deletes or requires at an agent may come from a conditional literal or from a listed role's literal at an instance α of its conjunct (Definitions 3 and 6); every argument below about $\beta(\varphi)$ then applies to $(\beta \cup \alpha)(\varphi)$, with the anchor $(\beta \cup \alpha)(A)$, held by the agent of its holder, in the place of an atom required there; the atoms of a condition of the body are held by the agents in roles.

B.1 Compiling Schemas into Transactions

Lemma 17 (The compilation is well defined). *R_C with $\sim$ is well formed.*

Proof. Let $d \longrightarrow d'$ be determined by τ at β over Q, and let P' be Q together with the agents β assigns to the agent variables of τ, the agents among the arguments of the atoms of d, the signers of the speech acts among them and the agents of the community names among them, and the signers of the speech acts β assigns. Every atom of every d'_p is an atom of d_p or the image under β of an atom added at a role, and the arguments of either lie in $P' \cup D_{P'} \cup \mathcal{N}_{P'}$, so $d \longrightarrow d'$ is a machine transaction over Q and $S_C(P')$.

Let machine transactions t and t' underlie unguarded transactions of R_C, determined by schemas τ and τ' with no guarding role at admissible bindings β and β', and let a transition lie in the P-closures of both. An agent whose state t changes is not stationary in that transition, so it is a participant of t', which changes its state in the same way, and conversely; by Definition 7 the atoms a transaction adds at an agent are those of its state after less those before, and the atoms it deletes those before less those after, so t and t' add and delete the same atoms at the same agents. If neither τ nor τ' has a reading role, every role of each adds or deletes an atom, so by Definitions 3 and 7 the state of every participant of each changes, and t and t' have the same participants and the same restriction to them, hence are equal. Otherwise let τ have one, and let $\varphi_1, \ldots, \varphi_m$, at the agent role π_i, be signature atoms of τ (Definitions 12 and 13). Take $\varphi_j \in +(i)$, the case $\varphi_j \in -(i)$ being the same with deleting or clearing in place of adding. Then t adds $\beta(\varphi_j)$ at $\beta(\pi_i)$, so t' does, and τ' adds an atom with the predicate of φ_j at a role whose extent holds $\beta(\pi_i)$; by Definition 12 $\tau' = \tau$ and that atom is φ_j at π_i, so $\beta'(\pi_i) = \beta(\pi_i)$ and $\beta'(\varphi_j) = \beta(\varphi_j)$. This holds for every j, and every agent role of τ but π_i is the base of an argument of some φ_j, so β' and β agree on every agent role. The reading roles of τ are agent roles, and every other role of it adds or deletes an atom, so the participants of each of t and t' are the agents whose state it changes and the agents β sends the reading roles to, the same for both; t and t' share a transition of their P-closures, so they have the same restriction to each of these participants (Definition 62), and $t = t'$. Either way two of them of different classes have disjoint closures, which is what Definition 63 requires of R_C.

Two machine transactions related by the relation generating $\sim$ have the same participants by Definition 16, and so do two related by $\sim$. A class over P is a union of sets of transactions each determined by a schema at a binding sending its agent roles and agent variables into P and its speech-act variables into D; a contract has finitely many schemas, each has finitely many such

bindings of its agent roles and agent variables, and D is countable, Δ and Π being countable, so the classes over P are countably many, as Definition 56 requires. □

Lemma 18 (Containment). *R_C is contained.*

Proof. Let $(t, Q') \in R_C$, with $t = d \to d'$ determined by a schema τ at an admissible binding β over participants $Q \subseteq P$ and $d \in S_C(P)^Q$. An atom of d'_p, $p \in Q$, is an atom of d_p or $\beta(\varphi)$ for an atom φ added at a role, and it suffices that every agent $\beta(\varphi)$ names is in P (Definition 1). Such an agent is $\beta(u)$ for u the base of an argument of φ that is an agent role, an agent variable or a name term, or the signer of $\beta(y)$ for a speech-act variable y that is an argument of φ or occurs in a name term among its arguments. By Definition 11, the set of predicates of C having traceable provenance, u is the base of a role of τ, and then $\beta(u)$ is the agent in an agent role of τ, an agent of Q; or u is the base of an argument of an atom ψ required at an agent role π_j, and then $\beta(\psi)$ is an atom of $d_{\beta(\pi_j)} \in S_C(P)$ naming $\beta(u)$, so $\beta(u) \in P$; or a safe formula of B names u through E, and then, the formula holding in d at β (Definition 7), by induction on it some atom ψ of Φ_E in it, held by an agent role π_j, has $\beta(\psi) \in d_{\beta(\pi_j)}$ naming $\beta(u)$: for an atom, itself; for a conjunction, the conjunct naming u, or, for $u = v$, the conjunct naming v, as $\beta(u) = \beta(v)$; for a disjunction, a disjunct that holds; for $\exists \bar{x}\ (A \wedge \varphi)$, an instance under which $A \wedge \varphi$ holds; so $\beta(u) \in P$. Likewise y is an argument of such a ψ or occurs in a name term among its arguments, or a safe formula of B names it, and then the signer of $\beta(y)$ is in P, or y occurs in no atom required at a role, and then $\beta(y)$ is signed by the agent in a guarding role (Definition 5), an agent of Q. So $d'_p \in S_C(P)$ for every $p \in Q$, and $(t, Q') \in R_C(P)$. □

The proof of Lemma 18 uses traceable provenance and no other condition of Definition 14.

Lemma 80 (Every participant changes state). *Every transition of $\mathcal{F}(P)$ induced by a transaction of R_C changes the state of every participant of that transaction that is in no reading role.*

Proof. Let the transaction be (t, Q'), with t determined by a schema τ at a binding β, and let p be a participant of t, so p is in the extent of a role of τ (Definition 7). If one such role is neither guarding nor reading, the machine state of p changes by Definitions 3 and 7. Otherwise $p \in Q'$, and the transition is taken where (t, Q') is enabled, so $[t]$ is in the volitional state of p there, and the transition removes it (Definition 58). □

Proposition 19 (Every participant takes a role). *Every agent whose machine state a transition of $\mathcal{F}(P)$ changes is in the extent of a role of a schema of C at some binding.*

Proof. A volition change leaves every machine state unchanged (Definition 58), so a transition changing one is a volitional machine transaction induced by some $(t, Q') \in R_C(P)$, and by that definition it leaves the machine state of every agent outside the participants of t unchanged. By Definition 16 t is determined by a schema τ of C at a binding β, and by Definition 7 the participants of t are the agents in the extents of the roles of τ. □

Proposition 20 (Anonymity). *For every permutation σ of Π, $\sigma(R_C) = R_C$, $\sigma(c_0(P)) = c_0(\sigma(P))$, and σ carries the runs of $\mathcal{F}(P)$ to the runs of $\mathcal{F}(\sigma(P))$.*

Proof. σ maps $\Sigma[P \cup D_P \cup \mathcal{N}_P]$ bijectively onto $\Sigma[\sigma(P) \cup D_{\sigma(P)} \cup \mathcal{N}_{\sigma(P)}]$, carrying signers and the agents of names with it, hence $S_C(P)$ onto $S_C(\sigma(P))$, and fixes $s_0 = \emptyset$; so $\sigma(c_0(P)) = c_0(\sigma(P))$. Let t be determined by a schema τ at a binding β. Then $\sigma \circ \beta$ is again a binding for τ, sending distinct agent roles to distinct agents, and two marked possibly one to one agent, as β does and leaving Y mapped into D, and it is admissible, σ carrying the signer of $\beta(y)$ to the agent in the same guarding role under $\sigma \circ \beta$. Each clause of Definition 7 is an inclusion, a disjointness, a match

or an identity between images under β and the local states of d and d', σ carries the extent of a role at β in d to its extent at $\sigma \circ \beta$ in $\sigma(d)$ and a name in no atom of d to one in no atom of $\sigma(d)$, and σ is a bijection on atoms, so applying it to both sides of each shows $\sigma(t)$ determined by τ at $\sigma \circ \beta$. The guarding roles of τ do not depend on the binding, so σ carries the volition-guarded transaction at β to the one at $\sigma \circ \beta$; by Definition 16 $\sigma(R_C) \subseteq R_C$, and applying σ^{-1} proves the reverse inclusion. Two members of R_C are $\sim$-related exactly when one schema at one binding determines both, a condition σ preserves, and $R_C(P)$ is carried to $R_C(\sigma(P))$ because σ carries $S_C(P')$ to $S_C(\sigma(P'))$ for every $P' \subseteq P$. The volitional construction and the P-closure are defined from $R_C(P)$, $\sim$ and $c_0(P)$ only, so σ carries $\mathcal{F}(P)$ to $\mathcal{F}(\sigma(P))$ and runs to runs. □

Corollary 21 (No privileged agent). *If a configuration is reachable in a run over P, so is its image under every permutation of P.*

Proof. Extend the permutation of P by the identity on $\Pi \setminus P$ and apply Proposition 20, which carries the run to a run over P ending in the image configuration. □

B.2 Turing Completeness

Proposition 4 (Turing completeness). *For every two-counter machine M there is a syntactically grassroots contract C_M such that M halts from empty counters if and only if some correct run of the protocol realising C_M reaches a configuration in which an agent holds halt.*

Proof. A two-counter machine [59] has states $q_0, \ldots, q_h$, q_0 initial and q_h halting, two counters holding natural numbers, initially zero, and for each state $q \neq q_h$ one instruction: $\mathrm{inc}(i, q')$, add one to counter i and go to q'; or $\mathrm{dec}(i, q', q'')$, if counter i is positive subtract one and go to q', else go to q''. Let C_M be GKK together with the following schemas over the predicates at_q, q a state of M, ctr_1, ctr_2, *started* and *halt*, all of arity zero in a state, with the one agent role *Alice*: the schema *start*, $started(Alice)$, $at_{q_0}(Alice) \leftarrow identity(Alice)$, $\neg started(Alice)$ guarded by *Alice*; for each $\mathrm{inc}(i, q')$ at q the schema $\neg at_q(Alice)$, $at_{q'}(Alice)$, $ctr_i(Alice) \leftarrow at_q(Alice)$; for each $\mathrm{dec}(i, q', q'')$ at q the two schemas $\neg at_q(Alice)$, $at_{q'}(Alice)$, $\neg ctr_i(Alice) \leftarrow at_q(Alice)$, $ctr_i(Alice)$ and $\neg at_q(Alice)$, $at_{q''}(Alice) \leftarrow at_q(Alice)$, $\neg ctr_i(Alice)$; and the schema $halt(Alice) \leftarrow at_{q_h}(Alice)$, $\neg halt(Alice)$; the schemas of M have no guard. Deleting ctr_i removes one copy, and the negated body atom $\neg ctr_i(Alice)$ holds when the machine holds none, so the multiplicity of ctr_i in the state of an agent is the value of counter i. C_M is syntactically grassroots: acquaint is its unobstructed introductory act as in GKK, the added schemas add atoms with no arguments, so traceable provenance stands, and each has one role, so its role graph is connected and it has no reading role. In a correct run, an agent p that establishes an identity and wills *start* takes it, by liveness, and thereafter holds at_q for exactly one q; while $q \neq q_h$ exactly one schema of M is enabled at p, since M is deterministic and the two schemas of a dec have exclusive bodies, it has no guard, so it stays enabled until taken and liveness takes it, and taking it carries the state and the counters of p as the instruction of M does. So the states at_q and the multiplicities of ctr_1 and ctr_2 at p along the run are the computation of M from empty counters, and p comes to hold *halt* if and only if that computation reaches q_h. Since two-counter machines compute every partial recursive function [59], the halting problem for them is undecidable, and so is whether some correct run of the protocol realising a contract reaches a configuration holding a given atom. □

B.3 Polite Sets of Volition-Guarded Transactions

Throughout this subsection, S is a local-states function, R a set of volition-guarded transactions over S with equivalence $\sim$, and $\mathcal{F}$ the protocol over R, S and $\sim$, as in Section 6.

Let $Q \subset \Pi$.

Lemma 81 (A Mutual Recording is an Interaction). *Every transition of $\mathcal{F}(Q)$ induced by a mutual recording of p and q in $R(Q)$ is an interaction between p and q.*

By Definition 67 it is then an interaction between any two disjoint sets of agents one of which holds p and the other q.

Proof. The machine states of p and of q both change, by Definition 24. Since d'_p records q, $d'_p \notin S(P)$ for every P with $q \notin P$, and in particular $d'_p \notin S(\{p\})$; the machine states of every configuration of $\mathcal{F}(\{p\})$ lie in $S(\{p\})$ by Definition 72, so the restriction of the transition to p is no transition of $\mathcal{F}(\{p\})$. By Definition 67 the transition is an interaction between p and q, hence between any two disjoint sets of agents one of which holds p and the other q. □

Theorem 26 (Polite). *The protocol over a contained, polite set of volition-guarded transactions is grassroots and volitionally grassroots.*

Proof. Let R be polite. One fact is used throughout: an agent whose state a volitional machine transaction changes is a participant of the transaction inducing it, as only a participant changes machine state and only a participant holds its class in their volitional state (Definitions 57 and 58).

Oblivious. Let $P, P' \subset \Pi$ be disjoint and nonempty, r a correct run of $\mathcal{F}(P)$, r' a correct run of $\mathcal{F}(P')$, and $e = e_0, e_1, \ldots$ an interleaving of r and r', with index sequences (i_k) and (j_k). By Lemma 75, every finite prefix of e is a finite safe run of $\mathcal{F}(P \cup P')$. Every transition of e is a P-step or a P'-step and so alters the state of agents of one of the two sets only, so no transition of e is an interaction between an agent of P and an agent of P', and no instance in a prefix of e holds agents of both.

R is contained, and we verify the other hypothesis of Proposition 76. Let $[t]$ be a class of $T_{R(P\cup P')}/\!\sim$ whose transactions have participants Q meeting both P and P'—the same Q for every member of the class, by Definition 56—let $(t'', Q'') \in R$ with $t'' \in [t]$, and take $a \in Q \cap P$. Suppose (t'', Q'') is enabled at some e_k. Its participants Q meet both sets and no instance in the prefix of e ending at e_k holds agents of both, so they do not all lie in one instance in that prefix; by closure the instance holding a there holds some $b \in Q''$, and $b \in P$ as that instance holds no agent of P'. Since Q meets P', $t'' \notin R(P)$, so $[t] \notin T_{R(P)}/\!\sim$; and by Lemma 74 applied to r, $e_k{}^v_b = (c_{i_k})^v_b \subseteq T_{R(P)}/\!\sim$. Hence $[t] \notin e_k{}^v_b$ and the guard on b fails, so (t'', Q'') is not enabled at e_k—a contradiction.

So no class whose transactions have participants in both sets is ever enabled in e, and by Proposition 76, $\mathcal{F}$ is oblivious.

Interactive. Let $Q \subset \Pi$, let $P, P' \subseteq Q$ be disjoint and nonempty, let $\hat{r}$ be a prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', and take $p \in P$ and $q \in P'$. An interaction between p and q is one between P and P' by Definition 67, so $\hat{r}$ contains none, and by openness some finite safe extension of $\hat{r}$ by volition changes and transitions that are not interactions between p and q ends where some $(t, Q') \in R(Q)$ whose induced transitions are interactions between p and q is machine-enabled. If a transition of that extension is an interaction between P and P', the extension is a finite safe run having $\hat{r}$ as a prefix and containing one, and by Lemma 60 a prefix of a correct run of $\mathcal{F}(Q)$; so suppose none is.

Extend further by a volition change of each agent of Q', where needed, so that $[t]$ lies in the volitional state of each; these leave every machine state unchanged, so (t, Q') is now enabled. Extend by the volitional machine transaction it induces, an interaction between p and q and hence between P and P'. The result is a finite safe run of $\mathcal{F}(Q)$ having $\hat{r}$ as a prefix and containing an interaction between P and P', and by Lemma 60 it is a prefix of a correct run of $\mathcal{F}(Q)$. Hence $\mathcal{F}$ is interactive, and with obliviousness, grassroots.

Volitionally grassroots. Let $P, P' \subset \Pi$ be disjoint and nonempty, let $\hat{r}$ be a safe run of $\mathcal{F}(P \cup P')$ interactive between P and P', and let $e \to e'$ be its first interaction. It changes the state of an agent of P and of an agent of P', and a volition change alters one agent only, so it is a volitional machine transaction induced by some $(t, Q') \in R(P \cup P')$ whose participants Q meet both sets; take $a \in Q \cap P$ and $b \in Q \cap P'$.

The participants of t do not all lie in one instance in the prefix ρ of $\hat{r}$ ending at e: were they, a and b would lie in one connected component of the interaction graph of ρ; every agent of ρ is in $P \cup P'$, so a path from a to b in that component has an edge between some $x \in P$ and some $y \in P'$, and the transition of ρ that puts that edge there is an interaction between x and y, hence between P and P', and it precedes $e \to e'$—contradicting that $e \to e'$ is the first. The same argument shows that every instance in ρ lies wholly in P or wholly in P', so the instance of a and the instance of b are distinct. As (t, Q') is enabled at e, by closure each of the two holds an agent of Q', the first in P and the second in P', so Q' meets both. $\mathcal{F}$ is oblivious and interactive, so it is grassroots, and interactive among the agents that have not left, so it is volitionally grassroots (Definition 79). □

B.4 Conditions Sufficient for Openness and Closure

Lemma 27 (Records). *On local states, Definitions 22 and 8 agree.*

Proof. Suppose an atom of s names q, as Definition 8 requires, and let $P \subset \Pi$ with $q \notin P$. That atom has an argument in $\Pi \setminus P$, a speech act among its arguments whose signer is in $\Pi \setminus P$, or a community name among its arguments outside $\mathcal{N}_P$, so $s \notin S_C(P)$ by Definition 1. As P was arbitrary, s records q. Suppose no atom of s names q, and let P be the agents among the arguments of the atoms of s together with the signers of the speech acts among them, the agents of the community names among them and some agent other than q, which exists as Π has at least two agents. Then P is nonempty, $q \notin P$ and $s \in S_C(P)$, so s does not record q. □

The recording graph of a configuration of the protocol realising C therefore has an edge $p \to q$ exactly when some atom of the state of p names q. Its edges are classified by the predicate of the atom.

Definition 82 (E-recording graph). Let E be a set of predicates. The *E-recording graph* of a configuration is the spanning subgraph of its recording graph whose edges $p \to q$ are those for which the state of p contains an atom with predicate in E that names q; the E-recording graph of a run is the union of those of its configurations.

Proposition 28 (Openness). *R_C is open.*

Proof. Let τ be an unobstructed introductory act of C, let $p \neq q$ in Q, let r be a finite safe run of the protocol over Q containing no interaction between p and q, and let β be an introduction of p and q meeting Definition 10. Extend r, for each role π_i of τ and each $\varphi \in =(i)$, by a volition change of $\beta(\pi_i)$ and a unary transition adding $\beta(\varphi)$ there, induced by a schema of arity one of clause 1 at an admissible binding sending its role to $\beta(\pi_i)$ and its agent variables into Q, which that agent takes by itself; where that schema forbids $\beta(\varphi)$ and the state of $\beta(\pi_i)$ holds it already, τ requires it once there, and the transition is skipped. Each transition taken is machine-enabled, the schema requiring nothing, forbidding nothing or only $\beta(\varphi)$, which is then absent, and having no name term, so no name need be fresh. Such a schema deletes nothing, so the extension is finite and safe and every atom τ requires at β is present at its last configuration. A schema of arity one has neither of p and q in a role other than the one sent to $\beta(\pi_i)$, so by clause 2 of Definition 10 no atom it adds at $\beta(\pi_i)$ is one τ forbids at π_i: the extension adds no forbidden atom, and no atom τ requires at β is one it forbids.

Let $\varphi \in \neg(i)$ and suppose the state of $\beta(\pi_i)$ there contains a copy of $\beta(\varphi)$. The initial state is empty and the extension added no forbidden atom, so some transition of r added it; that transition is induced by a transaction of R_C determined by a schema τ' at a binding β' with $\beta(\varphi) \in \beta'(+(j))$ for a role whose extent holds $\beta(\pi_i)$; by clause 2 of Definition 10 it is no listed role, so it is an agent role sent to $\beta(\pi_i)$. By that clause β' sends a role of τ' to the other of p and q, and $\beta'(+(j))$ names that agent. Both are then participants in no reading role, so by Lemma 80 the state of each changes, and the state of $\beta(\pi_i)$ afterwards records the other, so it lies in no $S(P)$ omitting them and the transition restricted to $\beta(\pi_i)$ is no transition of that agent's own transition system: the transition is an interaction between p and q, which r does not contain. So no forbidden atom is present, and τ, having no name term, is machine-enabled at β at the last configuration of the extension, and every transaction it determines there leaves p holding an atom naming q and q one naming p, changing both states, and is guarded by both; by Lemma 27 each records the other, so it is a mutual recording of p and q in $R_C(Q)$, β mapping into $\{p, q\} \subseteq Q$, and by Lemma 81 the transitions it induces are interactions between p and q. □

Closure requires that the participants of an unguarded act already lie in one instance. Where an atom can be added only by an act of the two agents it relates, that follows directly. An edge may also come from a transfer: a coin passes from hand to hand, and the agent it points at is reached through the agents that carried it rather than by a single act. The lemma below covers both: no act ever leaves an edge whose ends lie in different instances.

Let E be the set of predicates occurring in C other than its environment predicates, which has traceable provenance, $Q \subset \Pi$, and r a finite safe run of $\mathcal{F}(Q)$.

Lemma 83 (No edge crosses instances). *Every edge of the E-recording graph of r joins two agents that lie in one instance in r.*

Proof. The E-recording graph of r is the union of those of its configurations, and every configuration of r ends a prefix of r; the interaction graph of a prefix of r is a subgraph of that of r, so two agents in one instance in a prefix are in one instance in r. It suffices to treat an edge $p \to q$ of the E-recording graph of the last configuration. The claim is trivial for $q = p$, and we argue by induction on the length of r. The initial state is empty, so a run of one configuration holds it vacuously. Let r end with the transition $e \to c$, and let the state of p at c contain an atom with predicate in E naming $q \neq p$. If the state of p at e contains such an atom, the induction hypothesis applied to the prefix of r ending at e gives the claim. Otherwise $e \to c$ added it, so it is a volitional machine transaction induced by a transaction determined by a schema τ at a binding β, and the atom is $\beta(\varphi)$ for an atom φ added at a role π_i, with predicate in E, where p is in the extent of π_i, and $q = \beta(s)$ for s the base of an argument of φ that is an agent role, an agent variable or a name term, or q is the signer of $\beta(y)$ for a speech-act variable y that is an argument of φ or occurs in a name term among its arguments.

By Lemma 80 the state of every participant in no reading role changes, and by Lemma 27 the state of p at c records q, so it lies in no $S_C(P)$ with $q \notin P$ and the restriction of the transition to p is no transition of $\mathcal{F}(\{p\})$. By Definition 67 the transition is therefore an interaction between p and every other participant in no reading role. A participant p'' in a reading role π_j lies in one instance with p in r too. If τ has a guarding role, by Definition 13 a guarding role π_g requires an atom naming the base of π_j, or π_j is a listed role whose base π_g is a guarding role, which holds its anchor at the instance sending π_j to p''; either way $\beta(\pi_g)$ holds at e an atom of E naming p'', and $\beta(\pi_g)$ and p'' lie in one instance in the prefix ending at e by the induction hypothesis, while $\beta(\pi_g)$, in no reading role, is joined to p by an edge of the interaction graph. If τ has none, its role graph is connected (Definition 13), and the proof of Proposition 29, which uses this lemma only

for the prefix ending at e, where the induction hypothesis gives it, puts every participant of the transaction, p'' and p among them, in one instance in that prefix, hence in r.

By Definition 11 one of two cases holds, in the first case of s and in the second of y. If s is the base of a role of τ, an agent role whose agent is in its own extent, or y occurs in no atom required at a role and so by Definition 5 has $\beta(y)$ signed by the agent in a guarding role, then q is a participant, so p and q are joined by an edge of the interaction graph of r. Otherwise s is the base of an argument of an atom ψ required at an agent role π_j, with predicate in E, or y is an argument of such an atom or occurs in a name term among its arguments, or a safe formula of B names s or y through E, and then, the formula holding at e under β (Definition 7), by induction on it as in the proof of Lemma 18 some atom ψ of Φ_E in it, held by an agent role π_j, has $\beta(\psi)$ in the state of $\beta(\pi_j)$ at e naming q; put $p' := \beta(\pi_j)$. By Definition 7 $\beta(\psi)$ is an atom of the state of p' at e, its predicate is in E and it names q, so by the induction hypothesis p' and q lie in one instance in the prefix ending at e, hence in r. If $p' \neq p$ then p' is a participant, so p and p' lie in one instance in r, by the edge of the interaction graph or, p' in a reading role, as shown above. Either way p and q lie in one instance in r. □

Proposition 29 (Closure). *R_C is closed.*

Proof. Let (t, Q') of R_C be enabled at the last configuration c of a finite safe run r of the protocol, determined by a schema τ at a binding β, and let $Q_i := \mathrm{ext}_c(\pi_i)$, so that the participants of t are $\bigcup_i Q_i$ (Definition 7). Let E be the set of predicates occurring in C other than its environment predicates, which has traceable provenance.

First, let π_i be a listed role with base π_k and $p \in Q_i$ (Definition 6). $\beta(\pi_k)$ holds $(\beta \cup \alpha)(A)$ at c, A the anchor of a quantifier binding π_i and α an instance with $\alpha(\pi_i) = p$, an atom naming p, so $p = \beta(\pi_k)$ or the E-recording graph of r has the edge $\beta(\pi_k) \rightarrow p$, and p and $\beta(\pi_k)$ lie in one instance in r by Lemma 83. So the participants in the roles of one base lie in one instance in r.

Second, let π_i require an atom ψ of traceable provenance an argument of which has the base π_l of a role π_j. That argument is π_l or a name term with base π_l, so $\beta(\psi)$ names $\beta(\pi_l)$, and every $p \in Q_i$ holds it, so $p = \beta(\pi_l)$ or p and $\beta(\pi_l)$ lie in one instance in r by Lemma 83. By the first step, the participants in π_i and in π_j lie in one instance in r.

Instances are the connected components of the interaction graph of r, so lying in one instance is an equivalence, and by Definition 13 the participants in the roles of one connected component of the role graph of τ lie in one instance in r. If the role graph is connected, all participants of t lie in one instance. Otherwise each connected component of it has a guarding agent role, by Definition 13, whose agent is a guard of (t, Q') and lies in the instance of the participants of that component; every instance holding a participant holds the participants of a component, and so holds a guard. □

B.5 Contracts with Wards

Lemma 30 (Reflexive predicates). *Lemmas 18 and 83 and Propositions 28 and 29 hold for a contract declaring reflexive predicates, with Definition 10 read as above.*

Proof. Each proof goes through with one case added, where an atom a role requires is, under the binding, the atom of a reflexive predicate naming the agent in that role, which $\hat{d}$ holds at every agent. Containment: an argument of an atom added, by traceable provenance an argument of an atom ψ required at a role π_j, is $\beta(\pi_j)$ itself when $\beta(\psi)$ is that atom, an agent of Q. No edge crosses instances: for an argument s, or a speech-act variable y, of the added atom that is an argument of an atom ψ required at a role j, with $p' := \beta(j)$, $\beta(\psi)$ is the atom of a reflexive predicate naming p', and then $q = p'$, or an atom of the state of p', as there. Closure: the image of the atom of traceable provenance that π_i requires naming π_j lies in the state of $p := \beta(\pi_i)$ or is the atom of a reflexive

predicate naming p, and then $q = p$ and the two lie in one instance. Openness: the extension of the run adds, at each role, every atom required there other than an atom of a reflexive predicate naming its holder, which $\hat{d}$ holds; no such atom is forbidden; and no schema adds or deletes at a role the atom of a reflexive predicate naming it, so the rest of the proof is unchanged. □

Lemma 84 (Wards and guardians). *Let C have kinds (ι, f, M) and guardianships, and let (m, v, u) be the guardianship of $m \in M$. In every configuration of a run of the protocol realising C:*

(1) an agent holds ι if and only if they hold f or a mark in M, and then they hold one of these, once;
(2) an agent holding an atom of v holds f;
(3) an agent holding an atom of u holds ι; and an agent x holds $u(b, \bar{z})$ if and only if b holds $v(x, \bar{z})$, each at most once;
(4) where every schema of C adds the atoms of u and of v at agent roles, no agent holds an atom of u or of v naming themselves.

Proof. By induction on the run. The initial state is empty. A transition is induced by a transaction determined by a schema at a binding, and changes a state only by the atoms the schema adds, deletes and clears there. No schema deletes ι or f (Definition 31, clause 1), so an agent holding either holds it from then on.

(1) A schema adds ι at a role only where it forbids it, with f or one mark in M and no other of them, and adds a mark in M only so (clause 2); so an agent gains ι once, and with it one of them, once. A schema adds f otherwise only at a role at which it deletes a mark in M, and deletes a mark in M only where it adds f (clauses 2 and 3); the agent there held that mark and no other of them, and holds f and no other of them after it.

(2) An atom of v is added at a role that requires f (Definition 32, clause 3), which is never deleted.

(3) An atom of u is added at a role that gains m, and so ι, or that requires m (clause 3), whose holder holds ι by (1). By clause 3 a transition adds $u(b, \bar{z})$ at x exactly when it adds $v(x, \bar{z})$ at b. Before it x does not hold $u(b, \bar{z})$: either the schema forbids it at x, or it adds m at x, where it adds ι and so forbids it, and an agent holding an atom of u holds ι; so by induction b does not hold $v(x, \bar{z})$, and after it each holds its atom once. By clause 4 a transition deletes $u(b, \bar{z})$ at x exactly when it deletes $v(x, \bar{z})$ at b, each held once; and it clears u at x exactly when it deletes $v(x, \bar{z})$ at every agent in the extent of the listed role, which is every b with $u(b, \bar{z})$ in the state of x, the conjuncts having no condition (Definition 6): after it x holds no atom of u and no agent holds an atom of v naming x. No schema clears an atom of v (clause 1).

(4) A transition adds $u(b, \bar{z})$ at x only where it adds $v(x, \bar{z})$ at b (Definition 32, clause 3), at two agent roles each adding an atom naming the other; Definition 5 sends two such roles to distinct agents, so x and b are distinct. □

Lemma 85 (Leaving). *Let C have kinds (ι, f, M) and guardianships. In every configuration of a run of the protocol realising C:*

(1) a ward of a passive kind has left;
(2) a ward of a kind m that is not passive, holding no atom of the u of its guardianship, has left.

Proof. Let x hold $m \in M$. By Lemma 84, x holds ι and neither f nor another mark in M. A guarding role forbids ι, requires f, requires a mark in M and an atom of the u of its guardianship, or has in the body of its schema that it can act (Definition 32); so a guarding role sent to x in a transaction that is machine-enabled requires m and an atom of u, the u of the guardianship of m, or can act by the disjunct of m, which m has only if it is not passive and which holds only if x holds an atom of u.

(1) If m is passive, no guarding role requires it, no formula that a role can act has a disjunct of it and no schema deletes it, so in every later configuration x holds m and no transaction x guards is machine-enabled.

(2) If x holds no atom of u, no transaction x guards is machine-enabled. A transition deletes m at x only by a schema requiring an atom of u there (Definition 32, clause 2), and adds an atom of u at x only by a schema that adds m there, and so forbids ι there, or that requires an atom of u there, m not being passive (clause 3); so no transition does either, and the same holds in every later configuration. □

Throughout the rest of this subsection C is a contract with an introduction with wards, traceable provenance and volition, R_C its realisation and $\mathcal{F}$ the protocol realising it.

Definition 86 (Open among the agents that have not left). A set R of volition-guarded transactions is *open among the agents that have not left* if for any two agents $p \neq q$ in any $Q \subset \Pi$, every finite safe run of $\mathcal{F}(Q)$ with no interaction between p and q, in whose last configuration neither p nor q has left, has a finite safe extension, by volition changes and transitions that are not interactions between p and q, at whose end some transaction of $R(Q)$ whose induced transitions are interactions between p and q is machine-enabled.

An open set is open among the agents that have not left.

Proposition 87 (Openness with wards). *R_C is open among the agents that have not left.*

Proof. Let $p \neq q$ in Q, let r be a finite safe run of $\mathcal{F}(Q)$ with no interaction between p and q, and let c be its last configuration, at which neither p nor q has left. Call $x \in Q$ a *ward* at c if its state holds a mark in M, and *free* otherwise; a free agent holds f and ι, or neither (Lemma 84). By Lemma 85, each of p and q that is a ward is of a kind m that is not passive and holds an atom $u(b, \bar{z})$ of the guardianship of m; we take one such b as its *guardian*, who by Lemma 84 holds $v(x, \bar{z})$, and so f, and is free. Two facts. First, the transition that added $u(b, \bar{z})$ at a ward x added $v(x, \bar{z})$ at b, a role of its schema (Definition 32, clause 3), so it changed both states (Lemma 80, neither role reading), left x recording b, and was an interaction between x and b; so the guardian of p, if p is a ward, is not q, and that of q is not p. Second, for free $x \neq y$ in Q, the introductory act τ at the introduction of x and y can be made machine-enabled by unary transitions at x and at y: where one of them holds no ι, the establishing act, which forbids only ι and requires nothing, is machine-enabled there, and leaves them holding ι and f (Definition 31, clause 4); an atom τ requires that is then absent is the one atom of a schema of arity one requiring nothing but ι and f and forbidding nothing but that atom, which is machine-enabled; and τ forbids nothing (Definition 33). Taking τ there, after volition changes of x and y, leaves x holding $e(y)$ and y holding $e(x)$, and is an interaction between x and y and no others.

If p and q are free, the second fact at $x := p$ and $y := q$ gives an extension by unary transitions after which τ at their introduction is machine-enabled, and every transaction it determines is a mutual recording of p and q, so its transitions are interactions between p and q by Lemma 81.

Otherwise let χ be the relay act for the statuses of p and q, which C has, their marks not being passive, bound with its ward roles to those of p and q that are wards, its free role to the free one, and the guardian role of each ward role, where it has one, to that ward's guardian b, an agent variable of the ward role's atom of u, where it has none, to b too, and the value variables of that atom to $\bar{z}$; and the speech-act variables of each atom of a predicate h that χ requires at a ward role, as clause 2 of Definition 34 admits, to the arguments of an atom of h held by the ward bound there, which holds one: the transition that added its mark added one, and no transition deletes one; every transition that added an atom of v naming the ward at an agent added its record there,

with the arguments of the ward's h-atom, and no transition deletes one, so the guardian holds it under the binding. The related roles are bound to distinct agents, and two guardian roles are bound to one person only where the guardians taken for p and q are one, which the schema marks as possibly one, as GnW:4 does Alice and Bob, and Definition 5 permits as the guardian roles add and delete nothing naming each other. Extend r as follows, by transitions none of which is an interaction between p and q: for every atom χ requires at a free role or a guardian role that is ι, f or the one atom of a schema of arity one and is absent at the agent bound there, who is free, the unary transitions adding it, machine-enabled as above; and for every $e(\kappa)$ required at a guardian role or free role bound to x, with κ bound to $y \neq x$, where x does not hold $e(y)$, the unary transitions and the transaction of τ at the introduction of x and y by the second fact, x and y being free, and $\{x, y\} \neq \{p, q\}$, since a guardian is bound only where its ward is one of p and q, and is not the other by the first fact; and, where χ has two ward roles, for $e(\kappa)$ required at the guardian role bound to x, with κ the other ward role, bound to the ward y, whose guardian is b, where x does not hold $e(y)$: first $e(b)$ at x, as above, and then the relay act with ward role y, free role x and guardian role b, made machine-enabled as in this paragraph, its e-atoms being between free agents, which leaves x holding $e(y)$ and the record of y, and is no interaction between p and q, x being neither; where $b = x$, x holds $e(y)$ and the record of y already, as a guardian of y. At the end χ is machine-enabled: a reflexive atom naming its holder is in $\hat{d}$; the unary atoms, ι and f are present at the free agents; a ward role bound to p has ι, m, its atom of u naming b and the atom of h its variables are bound to at p; a guardian role bound to b has its atom of v naming its ward, by Lemma 84, and the record of its ward's h-atom at b; $e(\kappa)$ at a role bound to x with κ bound to y holds by the extension where $y \neq x$ and by reflexivity where $y = x$, and the record of the other ward at its guardian role likewise; and χ forbids nothing. Every transaction χ determines there leaves p holding an atom naming q and q one naming p, changing both states, so each records the other after it, the transition restricted to p is no transition of $\mathcal{F}(\{p\})$, and it is an interaction between p and q. □

Theorem 88 (Polite among the agents that have not left). *The protocol over a contained set of volition-guarded transactions that is closed and open among the agents that have not left is volitionally grassroots.*

Proof. Let R be such a set and $\mathcal{F}$ the protocol over it. By the parts of the proof of Theorem 26 headed Oblivious and Volitionally grassroots, which use containment and closure and not openness, $\mathcal{F}$ is oblivious, and the first interaction of two disjoint sets of agents in a safe run is induced by a transaction guarded by a member of each. Let $Q \subset \Pi$, let $P, P' \subseteq Q$ be disjoint and nonempty, and let $\hat{r}$ be a prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', in whose last configuration $p \in P$ and $q \in P'$ have not left. By Definition 86 some finite safe extension of $\hat{r}$, by volition changes and transitions that are not interactions between p and q, ends where some $(t, Q') \in R(Q)$ whose induced transitions are interactions between p and q is machine-enabled. If a transition of the extension is an interaction between P and P', the extension contains one. Otherwise extend it by a volition change of each agent of Q', where needed, so that $[t]$ lies in the volitional state of each, and by the volitional machine transaction that (t, Q') then induces, an interaction between p and q and hence between P and P'. Either way the result is a finite safe run of $\mathcal{F}(Q)$ having $\hat{r}$ as a prefix and containing an interaction between P and P', and by Lemma 60 it is a prefix of a correct run of $\mathcal{F}(Q)$. So $\mathcal{F}$ is interactive among the agents that have not left (Definition 78), and volitionally grassroots (Definition 79). □

Theorem 36 (Grassroots realisation with wards). *The protocol realising a contract with an introduction with wards, traceable provenance and volition is volitionally grassroots.*

Proof. By Proposition 87 R_C is open among the agents that have not left. The proof of Proposition 29 uses traceable provenance and volition and nothing of the introductory act, so R_C is closed, and by Lemma 18, whose proof uses traceable provenance only, it is contained; both hold with reflexive predicates by Lemma 30. By Theorem 88 the protocol over R_C is volitionally grassroots. □

Proposition 37 (GnW is grassroots). *The protocol realising GnW is volitionally grassroots.*

Proof. By Section 8.1 GnW has an introduction with wards, traceable provenance and volition, and by Theorem 36 the protocol realising it is volitionally grassroots. □

B.6 Relations among Contracts

Proposition 42 (Restriction is undecidable). *Whether one syntactically grassroots contract restricts another is undecidable.*

Proof. Let M be a two-counter machine, C_M the contract of the proof of Proposition 4, and C'_M that contract without its schema $halt(Alice) \leftarrow at_{q_h}(Alice),\ \neg halt(Alice)$, which is syntactically grassroots by the argument given there. We show that C'_M restricts C_M if and only if M does not halt from empty counters, which is undecidable [59]. As in that proof, in every safe run of either protocol the states at_q and the multiplicities of ctr_1 and ctr_2 an agent holds follow the computation of M from empty counters, so an agent holds at_{q_h} only if M halts.

Suppose M does not halt. Then no transaction of *halt* is machine-enabled at a configuration of a safe run. Only *halt* adds the atom *halt*, so no other schema determines a transaction it determines, and the classes of C_M are those of C'_M and those of *halt*. So the agent configurations and the transitions of the protocol realising C'_M over P are among those of the protocol realising C_M over P, and a class of the latter enabled at every configuration of a suffix of a correct run r' of the former is a class of the former, a member of which r' takes there. So r' is a correct run of the protocol realising C_M over P, and, the predicates of C'_M being those of C_M but *halt*, C'_M restricts C_M.

Suppose M halts, and let $P = \{p\}$. Let r' be the run of the protocol realising C'_M over P in which p wills establishing an identity and takes it, wills *start* and takes it, and then takes the schemas of M, one at each step, until p holds at_{q_h}. At its last configuration p wills nothing, establishing and *start* forbid atoms p holds, no schema of M is machine-enabled, and every other schema of GKK has two agent roles and so no binding into P; so no class is enabled there, and r' is correct. A run of the protocol realising C_M over P whose machine configurations, repetitions removed, include those of r' in order and end with its last one stays at that machine configuration from some point on. There *halt*, which has no guard, is machine-enabled at the binding sending *Alice* to p, so its class is enabled at every later configuration and never taken, and the run is not live. So C'_M does not restrict C_M. □

Lemma 44 (Stronger body). *Let the body of τ' be stronger than that of τ, every role of τ' that is not one of τ an agent role, and every binding admissible for τ' admissible for τ. If τ' determines $d \to d'$ at an admissible binding β, then β is admissible for τ, τ determines at β the restriction of $d \to d'$ to the agents in the extents in d of the roles of τ, and every other agent p of d has $d'_p = d_p$.*

Proof. Write Q and Q' for the agents in the extents in d of the roles of τ and of τ'. A listed role of τ is one of τ', written $\pi[A]$ with the same anchor, so it has the same extent in d for both (Definition 6), and $Q \subseteq Q'$. Restricted to the variables of τ, β is a binding for τ, the agent roles of τ being agent roles of τ' and the variables of τ roles or variables of τ'. It is admissible for τ, by hypothesis. The name terms occurring in no atom required at a role are the same for both, so the condition of Definition 7 on fresh names is the same; and the body of τ' holds at d and β, so that of τ does (Definition 43). A role of τ' that is not one of τ holds no atom of the head, which is that of τ,

since an agent variable holding an atom of τ is a role of τ; so it adds, deletes and clears nothing. The atoms added, deleted and cleared, the conditional literals among them, are read off the common head at instances fixed by d and β, so for $p \in Q$ the joins $+_p$ and $-_p$ and the set K_p are the same for both, the condition $+_p \cap (-_p \uplus K_p) = \emptyset$ holds for τ as for τ', and d'_p is the same; and an agent p of $Q' \setminus Q$ is in the extents of such roles only, so $d'_p = d_p$. So d and d' differ on Q, and τ determines their restriction to Q at β. □

Proposition 45 (Syntactic restrictions). *Let C and C' have traceable provenance and satisfy volition, with $\Sigma' \subseteq \Sigma$. C' restricts C if it is obtained from C by one of the following, and, restriction being transitive, by a sequence of them:*

(a) replacing a schema τ with a guarding agent role by a schema τ' with its name and guards and a stronger body, every role of τ' that is not one of τ an agent role that is a reading role, or deleting such a schema, C and C' having simple unguarded classes;

(b) adding a schema with no guarding role, with the head and the roles of a schema τ of C and a body stronger than that of τ;

(c) replacing a schema τ with no guarding agent role by a schema τ' with its name, roles and guards and a stronger body, where at every configuration of every correct run of the protocol realising C' and at every admissible binding the body of τ' holds wherever that of τ does, C and C' having simple unguarded classes;

(d) replacing a schema τ with a guarding agent role by a schema τ' with its name and a stronger body, whose guarding roles are those of τ and agent roles that are no roles of τ, every other role of τ' that is not one of τ an agent role that is a reading role, and every binding admissible for τ' admissible for τ, C and C' having simple unguarded classes.

Proof. Restriction is transitive, composing the runs that two restrictions give, since including a sequence in order is transitive; so it suffices to treat one step. Write $\mathcal{F}$ and $\mathcal{F}'$ for the protocols realising C and C', let $P \subset \Pi$, and let r' be a correct run of $\mathcal{F}'(P)$.

The run. Let $(t', G') \in R_{C'}$ be determined by a schema ρ' of C' at an admissible binding β. If ρ' is a schema of C, put $\hat{t}' := t'$. Otherwise ρ' is the τ' of (a), (c) or (d) or the schema (b) adds, which meet the hypotheses of Lemma 44, a binding admissible for τ' being admissible for τ in (a), (b) and (c) as every guarding role of τ' is one of τ and the two have the same head and the same speech-act variables in the atoms they require, and $\hat{t}'$ is the restriction of t' to the agents in the extents of the roles of τ, which τ determines at β, admissible for it. Let $\hat{G}'$ be the agents in the extents of the guarding roles of the schema of C determining $\hat{t}'$, so that $(\hat{t}', \hat{G}') \in R_C$, and $\hat{G}' = G'$ but in (b) and in (d), where $\hat{G}' \subseteq G'$. The agents of t' that $\hat{t}'$ lacks are stationary in t', so the P-closure of t' is contained in that of $\hat{t}'$ (Definition 62). Let r start at the initial configuration of $\mathcal{F}(P)$, and, for each volitional machine transaction of r', with one (t', G') inducing it chosen, take a volition change of each agent of $\hat{G}'$ adding $[\hat{t}']$, its class in C, and then the volitional machine transaction of $\mathcal{F}(P)$ induced by $(\hat{t}', \hat{G}')$ between the machine configurations of that step of r', which removes $[\hat{t}']$ from every person (Definition 58); it is one, as $(\hat{t}', \hat{G}') \in R_C(P)$, R_C being contained (Lemma 18) and $S_{C'}(P) \subseteq S_C(P)$ as $\Sigma' \subseteq \Sigma$. Where r' has finitely many volitional machine transactions, r ends after the last. So r is a safe run of $\mathcal{F}(P)$ whose machine configurations, repetitions removed, are those of r', and in which a person wills a class only from the volition change before its transaction until that transaction, which removes it.

Liveness. Suppose a class X of C is enabled at every configuration of a suffix of r and no member of it occurs there. The suffix holds a configuration at which no person wills anything; from the first such, after the kth volitional machine transaction of r, each of its machine configurations is that of such a configuration, at which X is enabled only by an unguarded transaction. So at

each of these machine configurations d some $(u_d, \emptyset) \in R_C$ with $u_d \in X$ is machine-enabled, and every configuration of the suffix of r' after its kth volitional machine transaction has one of them. Below we find a class Y of C' such that (i) at each such d some $(u'_d, \emptyset) \in R_{C'}$ with $u'_d \in Y$ is machine-enabled, and (ii) $\hat{t}' \in X$ for every $(t', G') \in R_{C'}$ with $t' \in Y$. By (i) Y is enabled at every configuration of the suffix of r', and r' is live, so a transition of that suffix is induced by some $(y, G) \in R_{C'}$ with $y \in Y$. The (t', G') chosen for that transition has $t' \in Y$: the transition removes $[y]$ and $[t']$ from every person and changes no other volitional state (Definition 58), so if the two classes differ neither is held by anyone before it, G and G' are empty, and y and t' underlie unguarded transactions whose P-closures share the transition, so are of one class by Lemma 17 for C'. So the corresponding transition of r, after its kth volitional machine transaction, is induced by $(\hat{t}', \hat{G}')$ with $\hat{t}' \in X$ by (ii), a member of X, which contradicts the supposition.

It remains to find Y. In (b) take $Y := X$. The machine transactions the added schema determines at a binding are among those τ determines there (Lemma 44), and C' keeps every schema of C, so the relation generating $\sim$ has the same classes in C' as in C, X is a class of C', and each $(u_d, \emptyset)$ is in $R_{C'}$, which gives (i); and $\hat{t}' = t'$ for every t', the added schema having the roles of τ, which gives (ii).

In (a), (c) and (d), C has simple unguarded classes, so X is the set T of machine transactions that one schema σ of C with no guarding agent role determines at one admissible binding β, and no other schema or binding determines a member of it. A schema that (a) or (d) replaces or deletes has a guarding agent role, so σ is a schema of C' or the τ that (c) replaces; let σ' be σ in the first case and τ' in the second, and T' the set of machine transactions σ' determines at β, which is admissible for it, the two having the same head, guards and speech-act variables in the atoms they require. Each u_d is in T': in the first case $T' = T$; in the second τ determines u_d at β, so its body holds at d and β, d is a machine configuration of the correct run r' of $\mathcal{F}'(P)$, so the body of τ' holds there by (c), and τ', with the head and the roles of τ, determines u_d at β, as in the proof of Lemma 44. The guards of σ' at β are those of σ, so $(u_d, \emptyset) \in R_{C'}$; let Y be the class of C' holding T', which gives (i). C' has simple unguarded classes and Y holds an unguarded transaction, so $Y = T'$ and no other schema or binding of C' determines a member of it; so every $(t', G') \in R_{C'}$ with $t' \in Y$ is determined by σ' at β, $\hat{t}' = t'$, σ' having the roles of σ, and σ determines it at β (Lemma 44), so $\hat{t}' \in T = X$, which gives (ii). □

Theorem 46 (What the check preserves). *Let C be syntactically grassroots.*

(1) *If C' amends C, then C' is syntactically grassroots if and only if C' has an unobstructed introductory act, every atom an added schema adds meets Definition 11 in C' with E the predicates of Σ' other than environment predicates, every added schema meets Definition 13 in C', and every kept schema with no guarding role and a reading role has signature atoms in C'.*
(2) *If C' is obtained from C by deleting schemas and by replacing schemas τ by schemas τ' with the name and the head of τ and a stronger body, the guarding roles of τ' those of τ and agent roles that are no roles of τ, every other role of τ' that is not one of τ an agent role that is a reading role and τ then having a guarding agent role, then C' is syntactically grassroots if and only if C' has an unobstructed introductory act and every such added reading role has an edge to a guarding role from an atom that guarding role requires.*
(3) *If C' instantiates C, then C' is syntactically grassroots.*
(4) *The lift of C at any set of its predicates is syntactically grassroots; so C' extending C at it is syntactically grassroots if and only if it meets the conditions of clause 1 as an amendment of that lift deleting nothing.*
(5) *Clauses 1 to 3 hold with 'has an introduction with wards, traceable provenance and volition' (Section 8) for 'is syntactically grassroots', of C and of C', and 'an introduction with wards' for 'an unobstructed introductory act'.*

Proof. Write E and E' for the predicates of Σ and of Σ' other than environment predicates, D for the schemas an amendment deletes, A for those it adds and $K := C \setminus D$ for those it keeps. Two facts are used. First, Definition 11 for an atom φ a schema τ adds asks of each argument of φ that its base be that of a role of τ or of an argument of an atom $\psi \in =(j) \cap \Phi_E$ required at an agent role of τ, or be named by a safe formula of its body through E, and the like of its speech-act variables; it concerns τ alone and the set E, and holds of τ with a larger set or a larger body if it holds with a smaller. Second, Definition 13 for a schema τ asks of its role graph, whose edges come from atoms τ requires of traceable provenance, of its reading roles, and, where τ has no guarding role and a reading role, of signature atoms, whose last clause alone mentions the other schemas of the contract.

(1) Let $C' = K \cup A$ amend C. If C' is syntactically grassroots the four conditions hold by Definition 14, the second and third being instances of Definitions 11 and 13 for C' and the fourth part of Definition 13. Conversely, let the four hold. Every predicate of E' occurs in a schema of C'. E' has traceable provenance in C': an atom added by a schema of A meets the condition by hypothesis; one added by a schema τ of K meets it in C with E, through some ψ required at an agent role of τ, whose predicate occurs in τ and so is in E', hence it meets it in C' with E', by the first fact. Every schema of C' satisfies volition in C': one of A by hypothesis; one of K has in C' every edge it has in C, its required atoms being of traceable provenance in C' as all of E' is, so its role graph is connected or has a guarding role in each component as in C, its reading roles have their edges as in C, and where it has no guarding role and a reading role it has signature atoms in C' by hypothesis. With an unobstructed introductory act, C' is syntactically grassroots.

(2) Let C' be so obtained, with A the schemas τ' and D the schemas they replace and those deleted. If C' is syntactically grassroots it has an unobstructed introductory act, and an added reading role has the edge Definition 13 asks of a reading role of a schema with a guarding role. Conversely, let both hold; by (1) it remains to show the other three conditions. An atom τ' adds is one τ adds, and in C the condition of Definition 11 holds of it through roles of τ, which are roles of τ', and through atoms ψ required at agent roles of τ, which τ' requires there (Definition 43) and whose predicates are in E', and through safe formulas of the body of τ, which the body of τ' holds (Definition 43); a speech-act variable occurring in no atom required at a role of τ occurs in none of τ', the two having the same such variables; so the condition holds in C' with E'. The role graph of τ' has every edge of that of τ, since τ' requires every atom τ requires and all of E' has traceable provenance in C', and each added reading role has an edge to a guarding role. So every connected component of it holds a guarding agent role: one holding a role of τ holds a connected component of the role graph of τ, which has a guarding agent role where that graph is not connected and is all of it where it is, τ having a guarding agent role; and one holding added roles only holds an added guarding role, or an added reading role with its edge to a guarding role. The guarding roles of τ' are those of τ and the added ones, and its reading roles are those of τ, the heads being the same, with their edges, and the added ones. A schema of C' with no guarding role is one of C or replaces one with its head and roles, so the schemas of C' with no guarding role add, delete and clear what a subset of those of C do, and signature atoms of such a schema in C are signature atoms of it, or of its replacement, in C'.

(3) An instantiation writes, for each parameter instantiated, a predicate, a value or an utterance not occurring in C, a different one for each, so it is a bijection between the atoms of C and those of C' and between E and E' that keeps every argument and its kind, and adds to an atom of a predicate written with further value arguments a value variable for each, shared by the atoms over the same agent variables (Definition 38), a wildcard among them being a variable of its own, or universally quantified where the atom is negated (Definition 3). Definitions 9, 10, 11, 12 and 13 mention a predicate only to compare the predicates of two atoms and to form the sets Φ_E, and an argument only by its kind and its base, a value and an utterance having none, so each holds of C' exactly

when it holds of C, an added argument being read as the proof of clause 4 reads the argument a lift adds: an introduction of C' is one of C with any values for the added variables of its act; an atom it requires at a role is added by the instance of the schema of arity one that adds the atom of C, at the binding sending that schema's added variables, which no atom it requires binds, to those values; and a schema of C' adding at some binding an atom the introductory act forbids is the instance of a schema of C adding at that binding, without the values, the atom of C it forbids.

(4) Let L be the lift of C at F. An added argument is a value variable, which has no base, so Definitions 11, 12 and 13 read each schema of L as they read the schema of C it lifts, and a schema of L adds, deletes or clears an atom of a predicate exactly where its schema of C does. An introductory act of C lifts to one of L, with no constraint, its introduction choosing for the fresh variable any value (Appendix C). Clause 1 of Definition 10 holds of the lift: an atom it requires at a role, with the chosen value v, is added by the lift of the schema of arity one that adds the atom of C, at the binding sending its fresh variable to v; clause 2 holds, a schema of L adding at some binding an atom the introductory act forbids being the lift of a schema of C adding at the binding without the value the atom of C it forbids. So L is syntactically grassroots, and the rest is clause 1 with C replaced by L.

(5) Clauses 1 and 2 use of the introductory act only that C' has one, and their proofs of traceable provenance and volition stand, both being conditions on each schema given the predicates of the contract. For clause 3, Definitions 31, 32, 33, 34 and 35 mention a predicate only to compare the predicates of two atoms and to say where the schemas add, delete, clear, require and forbid its atoms, and an argument only by its kind, which the bijection keeps; an added argument is a value of a predicate that is not a mark, so Definition 31 reads the marks of C' as those of C, and the two records of one guardianship, over the same two agents, share it, as clauses 3 and 4 of Definition 32 and clause 2 of Definition 34, which admit value arguments, ask; so each holds of C' exactly when it holds of C. □

B.7 Clauses and Their Modalities

Proposition 51 (Witnessing). *A schema is witnessed against every agent role it names at another agent role.*

Proof. Let τ name π_j at π_i by an atom φ added at π_i, let β be a binding for τ, and put $a := \beta(\pi_j)$ and $b := \beta(\pi_i)$, which differ by Definition 5. Let $d \to d'$ be determined by τ at β. By Definition 7 d'_b is d_b less the atoms deleted or cleared at the roles sent to b and plus those added there, no atom added being deleted or cleared, so $d'_b \setminus d_b$ is the join of the atoms added at those roles and holds $\beta(\varphi)$, an atom naming a. □

Proposition 52 (Enforcement by guarding). *If e is guarded in C, the consent clause for e is enforced by C.*

Proof. Let r be a finite safe run of the protocol realising C and let an e-atom occur in the state of an agent at some configuration of r. The initial state is empty, so some transition of r added it, and by Definitions 16 and 7 that atom is $\beta(\varphi)$ for a binding β of a schema τ of C and some $\varphi \in +(i)$ with predicate e. By hypothesis every role φ names is a guarding role of τ, so every agent the atom names is β of a guarding role and hence in the guard of the transaction taken; by Definition 64 each held its class in its volitional state at that point, which is to say willed the act. □

B.8 Signing the Contract

Proposition 54 (Every participant has presented its undertaking). *In a presenting run, every agent of an instance of two or more agents has presented their undertaking to another agent of that instance.*

Proof. An instance is a connected component of the interaction graph of the run (Definition 69), so an agent of an instance of two or more agents has an edge to another agent q of it, put there by an interaction between the two; the run is presenting, so the agent has presented their undertaking to q. □

C VALUES IN SPEECH-ACT SCHEMAS

Speech-act schemas allow values of any type (Section 3.2). This appendix defines them and extends states and bindings to them; the compilation and the conditions then apply as stated.

Definition 89 (Schema with values). Fix a countable set V of *values*, disjoint from Π, D and $\mathcal{N}$, and a set Z of *value variables*, disjoint from R, X and Y. A *schema with values* is written

$$\tau: \quad H \leftarrow B,\ \gamma \quad \text{guarded by } G$$

as in Definition 3, except that its variables may be value variables in Z; an argument of an atom may be a value or a value variable, or the item $\mathrm{item}(y)$ of a speech-act variable y of the schema that an agent role requires, the utterance under the signatures of y, which names no agent; γ, its *constraint*, is a decidable condition on the values assigned to its value variables, written in the relations and functions the types of the values provide, as the order and the arithmetic of numbers and the distance of points of the plane, combined by the connectives, together with *signing conditions* $\mathrm{signed}(u, y)$ and $\mathrm{unsigned}(u, y)$, u an agent role or agent variable and y a speech-act variable of the schema, the *author* $\mathrm{author}(y)$, the first signer of y, in an equality with an agent role or agent variable, the *item* $\mathrm{item}(y)$, the utterance under the signatures of y, in an equality with another, and a decidable relation among utterances and values that their types provide, as that an utterance is a grassroots contract with its schemas and pragmas, or that bylaws admit an act given the votes cast, and equalities and inequalities $u = v$ and $u \neq v$ of two agent variables, two speech-act variables or two name terms, and, as a whole number, the *cardinality* $\mathrm{cardinality}(\bar{x}, A \wedge \varphi)$ of an anchored safe formula, the number of instances of its anchor A at which φ holds, each counted as many times as its multiplicity, binding $\bar{x}$ as the anchored quantifiers of Definition 2 do, as $\mathrm{cardinality}(x, \mathit{points}(\mathit{Alice}, x)) < d$, Alice pointing to fewer than d persons, which makes the body a formula of first-order logic with counting [52], and, as a value of a numeric type, the *sum* $\mathrm{sum}(\bar{x}, A \wedge \varphi, u)$ of a numeric term u over the instances of its anchor A at which φ holds, each taken as many times as its multiplicity, binding $\bar{x}$ as cardinality does; a quantified conjunct carries constraints of the same kind on its variables and those of the schema; an introductory act and a schema of arity one admitted by clause 1 of Definition 10 carry no constraint, since whether some binding satisfies a constraint is undecidable in general; and an atom of H or of B with no wildcard may carry a *multiplicity* κ, a positive whole number or a value variable, written $\kappa \cdot \varphi$, κ copies of φ, which in B are required, or forbidden when negated, the machine then holding fewer than κ, and in H added or deleted; where an atom is required with multiplicity κ and deleted with multiplicities $\kappa_1, \ldots, \kappa_n$, the schema's constraint implies $\kappa_1 + \cdots + \kappa_n \leq \kappa$, so that $-(i) \subseteq =(i)$ holds at every binding; the checker decides the implication where the constraint is linear over whole numbers, and refuses the schema otherwise, the condition being sufficient.

A speech-act variable that takes a text a person enters may carry a type too, naming the kind of the text, as an email address or a person's name, written $y{:}T$ at one of its occurrences. The type

constrains nothing: a binding, the three conditions and the compilation read $y{:}T$ as y, and the app built from the contract reads the type for the field that takes the text.

Definition 90 (Binding with values). A *binding* β for a schema with values τ is a binding for τ (Definition 5) that also maps each value variable of τ to a value, under which γ holds, signed(u, y) holding when $\beta(u)$ is the signer of $\beta(y)$ or of a speech act inside it and unsigned(u, y) otherwise, author$(y) = u$ when $\beta(u)$ is the innermost signer of $\beta(y)$, item$(y) =$ item(y') when $\beta(y)$ and $\beta(y')$ carry one utterance, and $u = v$ when $\beta(u) = \beta(v)$, and every multiplicity of τ is a positive whole number; it maps a value to itself, and $\kappa \cdot \varphi$ to the multiset holding $\beta(\varphi)$ with multiplicity $\beta(\kappa)$.

A ground atom may have values and utterances among its arguments: $S_C(P)$ is the set of finite multisets over $\Sigma[P \cup D_P \cup \mathcal{N}_P \cup V \cup \Delta]$, and a permutation of Π is the identity on values and on utterances; a binding sends item(y) to the utterance $\beta(y)$ carries, so an item holds the same program, or the same post, whoever signed it over. Which types the values have — numbers, strings, dates — is the realisation's; the contract is parametric in them, and a value variable of a schema has a type, which the schema's term form writes, so that the same clauses are realised over whole or over fractional amounts by the schemas' choice. A multiplicity counts copies of an atom, so it is a positive whole number in every contract: paying zero coins is no act. In the examples of this paper coins are whole, an amount a multiplicity: GC's payment pays k coins, $-k \cdot ¢(Bob)$ at Alice and $+k \cdot ¢(Bob)$ at Bob; the bonds of [81] number dates by days, the successor of a date being the next day. A binding chooses its values, so an introduction (Definition 9) chooses its own.

Definition 7 applies to a schema with values as written, the multisets of a role at a binding carrying each atom as many times as its multiplicity says. Definition 3 requires that some role add or delete an atom, and a multiplicity is at least one, so this holds at every binding. A value, and an utterance, is no agent role, no agent variable and no name term, so it has no base, Definitions 11 and 13 do not concern it, and no state records an agent by it (Definition 8). The conditions of Section 3.8 therefore read a schema with values as they read one without, they stay decidable, no condition reading a constraint, which the realisation evaluates when it binds the schema, and Theorem 15 and its proof stand; a signing condition and an equality name no agent a binding does not, so Lemma 18 and Proposition 20 stand, a permutation carrying signers with it and being a bijection on agents, speech acts and names.

Environment predicates. A contract may name a set of *environment predicates* in Σ, whose atoms are over V and over agents, as $nearby(Bob)$ in the state of Alice, which the platform keeps current from the persons its networking layer reports nearby: the machine holds them from startup, in place of the empty initial state, and the platform keeps them current, as it keeps $today(t)$ at the calendar's date; no schema adds or deletes one, and a schema may require or forbid one. The platform's update of an environment atom is a unary transition of the machine holding it, in no class, so it carries no liveness obligation and is no interaction; in a run over P an environment atom names only agents of P, and any update naming them may occur, so the states stay in $S_C(P)$ and Proposition 20 stands. An environment predicate has no traceable provenance (Definition 11): Provenance traces through none of its atoms, Volition takes no edge of a role graph from one, and Lemma 83 and Proposition 29 read the set E of the other predicates, so an environment atom naming an agent in another instance puts an edge in the recording graph and none in the E-recording graph, and they stand, as do Lemmas 27 and 18, the latter reading only atoms a schema adds; an unobstructed introductory act requires no environment atom, clause 1 of Definition 10 admitting only atoms a schema adds, and forbids none, and the proof of Proposition 28 stands. The bonds of [81] name *today*, which their first act requires.

GB's redemption of a matured bond, as its owner's copy has it, is the schema with values

$$\begin{aligned} \mathit{redeem_bc}: \ & \neg \mathit{bond}(\mathit{Alice}, \mathit{Bob}, d),\ \text{¢}(\mathit{Alice}, r),\ \neg\text{¢}(\mathit{Bob}, r),\ \mathit{bond}(\mathit{Bob}, \mathit{Bob}, d) \leftarrow \\ & \mathit{identity}(\mathit{Alice}),\ \mathit{knows}(\mathit{Alice}, \mathit{Bob}),\ \mathit{bond}(\mathit{Alice}, \mathit{Bob}, d), \\ & \mathit{identity}(\mathit{Bob}),\ \mathit{date}(\mathit{Bob}, t),\ \text{¢}(\mathit{Bob}, r),\ d \leq t \quad \text{guarded by } \mathit{Alice}, \end{aligned}$$

where $\mathit{bond}(\mathit{Bob}, d)$ in the state of Alice is a bond issued by Bob and maturing at d, $\mathit{date}(t)$ in the state of Bob is Bob's date, and the constraint says that the bond is mature. It compiles to

$c'_{\mathit{Alice}} := (c_{\mathit{Alice}} \setminus \{\mathit{bond}(\mathit{Bob}, d)\}) \uplus \{\text{¢}(r)\}$, $\quad c'_{\mathit{Bob}} := (c_{\mathit{Bob}} \setminus \{\text{¢}(r)\}) \uplus \{\mathit{bond}(\mathit{Bob}, d)\}$,

provided $\mathit{identity}, \mathit{knows}(\mathit{Bob}), \mathit{bond}(\mathit{Bob}, d) \in c_{\mathit{Alice}}$ and $\mathit{identity}, \mathit{date}(t), \text{¢}(r) \in c_{\mathit{Bob}}$,

provided $d \leq t$,

guarded by $\{\mathit{Alice}\}$

A binding chooses d and t, and binds the schema only if $d \leq t$; Bob's role, which is not guarding, reads Bob's date and changes state.

D RELATED WORK, EXPANDED

The paragraphs of Section 12, expanded.

Contracts as formal objects. The formal study of contracts in this field is dominated by the contract-as-automaton family. *Contract automata* [5] give the deontic modalities an operational semantics over the acts of named interactive parties; Flood and Goodenough [24] render a loan agreement as a deterministic finite automaton whose states are performing, delinquent and default and whose transitions are drawn from an alphabet of events; the L4 domain-specific language [93] has an executable semantics in which deontic clauses with deadlines become states and transitions; and timed contract automata [13] add quantitative time. Rule-based encodings represent contract clauses as defeasible deontic rules with violation and reparation chains [36], and imperative and declarative smart contracts running on a distributed ledger have been compared [35].

Three commitments are common to all of these, and this paper makes none of them. The parties are fixed when the contract is formed, so the object of study is one instance among a known set of parties, where our conditions quantify over the family of systems a contract induces on every finite set of people. There is one contract state, the automaton's state, the rule base's facts or the ledger, where the definition of *records*, and therefore of what it is for two agents to be connected, needs a local state per agent and a local-states function monotone in the set of people. And each requires something closure forbids: an alphabet of events every party is assumed to observe alike, a clock, or a ledger.

Two languages outside this venue are the nearest to ours. Symboleo [63, 85] specifies a legal contract as a set of obligations and powers over an ontology of roles, assets and events, with a semantics of logical axioms on statecharts, and properties verified in temporal logic; its purpose is monitoring, and roles are assigned to parties during each contract execution. Stipula [16] is a domain-specific language with legal constructs, agreement, permissions, obligations and violations, whose contracts are state machines over parties, fields and linear assets; its *agreement* operator is the contract's constructor, fixing the full list of parties, and thereafter each function is invoked by one named party. Catala [56] does the corresponding work for statute rather than contract, encoding the base-case and exception structure of legislation in default logic. Both contract languages make the three commitments above: a Stipula configuration carries a global clock, and Symboleo presupposes a monitor, which is a participant that is not a party.

A concrete case is befriending, which cannot be written in Stipula: it is an act that two agents must both will, available repeatedly between any two of them, and Stipula allows joint consent

only in the constructor, so each befriending would be the formation of a new contract rather than a clause of a standing one.

We claim no new language. The conditions of Section 7 are on the four things a schema carries. Stipula's function declarations name an invoking party and a precondition, and Symboleo attributes obligations to a debtor and a creditor, so the four could be extracted from either given a semantics in which each party holds its own state.

BSPL [87] is the protocol language of multiagent systems nearest to ours. A protocol is a set of message schemas between roles, each parameter adorned in or out, and a role may send a message once it knows the bindings of the message's in parameters and does not know those of its out parameters. There is no global state: the local state of a role is the history of the messages it sent and received, and the enactments of a protocol are its tuples of parameter bindings, a key keeping them unique. The meaning of a message, the commitments it creates among them, is a layer above. A speech-act schema is the counterpart of a message schema in which an act may take any number of roles, each consenting by its guard, and changes the local state of each. BSPL's well-formedness conditions make a protocol enactable. The conditions of Section 7 concern the family of systems a contract induces on every finite set of people, and the coalescence of two of them.

Dynamic and temporal logic. In first-order dynamic logic [42] a state is a valuation of the program's variables over a fixed first-order structure, among them array variables, an n-ary array f assigned by $f(t_1, \ldots, t_n) := t$; a program is built from assignments and tests by sequence, nondeterministic choice and iteration, with the wildcard assignment $x :=?$ choosing any element of the domain; a program denotes a relation on states, its input/output pairs; and a formula is a first-order formula closed under $[\alpha]\varphi$, that every halting run of α ends in a state satisfying φ. A speech-act schema $H \leftarrow B$ guarded by G is the program $x_1 :=?; \cdots; x_m :=?; B^\circ?; G^\circ?; H^\circ; W^\circ$: each predicate of arity n is an array of arity $n+1$ with values in the natural numbers, $e(p, \bar{u}) = m$ saying that the machine of p holds $e(\bar{u})$ with multiplicity m, and a person's will an array $\mathit{wills}(p, [t])$; B° tests the body, $e(\bar{u}) \geq \kappa$ for an atom and $e(\bar{u}) = 0$ for a negated one; G° tests $\mathit{wills}(a, [t]) = 1$ for each guard a; H° adds and subtracts multiplicities as the head says, a cleared atom and a quantified conjunct by a while loop with a wildcard assignment; and W° sets $\mathit{wills}(a, [t]) := 0$ at every agent. Its input/output relation is the denotation of Definition 7, and the Kripke frame of these programs is the transition system of Section 3.3. A volition change is the person's own program. What dynamic logic does not state is a property of runs: liveness, that a class enabled from some point on is eventually taken, is Manna and Pnueli's justice [55], a formula of linear temporal logic [64]; interactivity, that from every reachable state some continuation interacts, is a formula of branching time, CTL* [19], over the transition system of a set of agents; and process logic [41] interprets programs and formulas alike over traces, joining the two. Obliviousness relates the systems of two disjoint sets of agents to that of their union, which no formula over one system states, and persons with wills are in none of the three; both are [51]'s, and the paper takes its semantics from there.

Inheritance, refinement and generics. Section 9's relations each have a counterpart. Extension is subclassing, a class adding methods to its parent's, and an amendment that rewrites a clause is overriding. Behavioural subtyping [54] requires of a subtype that every property proved of the supertype's objects hold of its own, and design by contract [57] lets a redefined method weaken its precondition and strengthen its postcondition, so that a client of the parent is served by the child; the refinement calculus [6] orders programs the same way, a refinement terminating wherever its original does and choosing among fewer outcomes, and the B method [2] develops a program from a specification by such refinements. The syntactic approximations of a restriction strengthen preconditions, which all of these forbid, since a client who could act under the original may be unable to under the restriction; restriction is CSP's refinement [44], where a process refines another

when its traces are among the other's, every correct run of the restriction being, on the machines, one of the original (Definition 41). Instantiation is parametric polymorphism [58], a contract generic in a predicate as a program is in a type, checked once for all its instances; the types of Grassroots Logic Programs [84] are the realisation's. Z's schema calculus [88] builds a specification by including schemas in schemas, which is extension, and conjoining them, which is composition. In algebraic specification [31] a theory extends another by adding symbols and axioms, the extension being conservative when it proves nothing new of the old symbols, and a theory morphism carries one theory's symbols to another's with their signatures, which is instantiation; Theorem 46 shows the check of an amendment confined to what it adds, a conservativeness of the check. Contract law amends a contract by modifying its terms, supplements it by adding terms, and novates it by substituting a new contract, with a new party or between the same ones [3, §§89, 280]. None of these says what a syntactic check preserves, and Theorem 46 does.

Norms and normative systems. The formal study of what a contract obliges uses deontic and defeasible logic: contracts are represented as defeasible rules with deontic operators and compliance is checked against them [34]. Normative multi-agent systems concern how norms are operationalised, detected, and brought into conflict [9]. A volition guard [51] is a condition of enactment rather than a deontic operator: an act does not occur unless the people it obliges are willing. Where a clause of the contract describes no act the protocol carries out, Section 10 says what the realisation does with it: record its breach, or record the undertaking and the party who gave it.

Social commitments [86] give the normative concepts one form, a commitment $C(x, y, G, p)$ of a debtor x to a creditor y in a context group G to bring about p, the context group providing the norms by which disputes between the two are settled. A commitment is created, satisfied, cancelled, released, delegated and assigned, and obligations, taboos, conventions and the Hohfeldian relations are kinds of commitment. The three modalities of Section 10 classify what a realisation does with a clause, and are not kinds of obligation. An undertaken clause is a commitment in this sense of its obligor to every other participant, and another agent holds its record; a grassroots contract has no context group above the agents.

Juridical acts and declarative power. What we call an act is Sartor's *result-declaration*, one of "acts intended to produce legal determinations" [68], and the fullest model of it is Hage's [40], which analyses juridical acts as intentional changes in a world of law furnished with entities, facts and rules. A multi-party act willed by its parties is not new either. Declarative power is "the capacity of the power-holder of creating normative positions, involving other agents, simply by 'proclaiming' such positions" [29], and its authors observe that a declarative power exercised jointly by several parties, with the consent of each, is a contract [28].

We differ from that tradition on where the effect of an act comes from. There a proclamation has effect only if an institution provides for it: "when an agent j proclaims A, j brings it about that A only if the concerned institution s provides for this result" [28]; in norm-governed institutions "designated agents are *empowered* to create particular kinds of states of affairs" [45]; counts-as statements "hold only with respect to a context" [38]; and an artificial institution "presupposes an agreement on an unambiguous definition of a set of concepts" [25]. A grassroots contract has no such institution, being subject to no external authority. The effect of an act is therefore the change it makes in the agents' own states, and the condition of its effectiveness is their will rather than a conferred power. A speech-act schema is a result-declaration with the institution taken out: its precondition is on each participant's own local state rather than on an institutionally conferred power, and its guard takes the place of authorisation.

An *electronic institution* [20] is specified by a performative structure, a network of scenes connected by transitions, each scene a protocol over roles whose arcs are labelled with illocution schemes; by institutional roles, which external agents cannot play; by a role hierarchy and a separation of duties among roles; and by normative rules that infer an agent's obligations from its illocutions. Every agent adopts a role, every interaction takes place in a scene, an institutional agent at each transition brokers agents between scenes, and the institution keeps for each agent the obligations its acts incurred. A grassroots contract has no institutional role, and the records of its acts are kept in the agents' own states.

Standard-form contracting. A grassroots contract has no drafting party, so there is no party whose terms are policed and no party from whom disclosure is required; the defects the literature finds in the arrangement between a person and a proprietor are absent because the proprietor is absent.

The End-User Licence Agreement is a *contract of adhesion*. Radin [66] treats mass-market boilerplate as the deletion of rights that the legal system otherwise confers, and as a matter for the rule of law rather than for consent only. Kim [48] treats the digital instances of the form — the terms accepted by clicking, browsing, or installing — and what assent means when the act of assent is a condition of access. Ben-Shahar and Schneider [7] address the remedy most often proposed, mandated disclosure, and find that it fails: the terms are not read, and requiring more of them is not read either.

This literature takes the bilateral form as given; its questions are what may be done within the form: whether assent was real, which terms a court should refuse, what a drafter must disclose.

Terms a machine can evaluate. Surden [89] separates the terms of a contract a machine can evaluate from those it cannot; work on smart legal contracts separates the operational parts of an agreement, which code executes, from the rest, which stays in prose [14]; and in security, Schneider [69] characterises the policies an execution monitor can enforce. Each divides a contract's terms in two, by what a realisation can do with them. Section 10 divides clauses in three, distinguishing a clause no act of the contract can breach from a clause an agent may breach while the contract records the breach against them. A two-way division has no room for the second, and it is the class the law needs: a remedy requires a provable breach and an identifiable obligor.

In the monitoring literature, compliance is computed by monitors that "receive inputs from *trusted* observers" [60], and where the timestamps cannot be trusted the monitor is made robust rather than removed: compliance "is typically computed with respect to timed event traces with event timestamps assumed to be perfect", and the remedy offered is a semantics for compliance when they are not [11]. A monitor is a participant without a role in the act, and closure forbids one. Here the record is held by the parties themselves, and what makes it evidence is Definition 48: it names the party answerable and it lies in someone else's hands.

Code as a regulator. Lessig [49] argued that architecture regulates conduct as law, markets and norms do, and that in the digital realm the architecture is code. A grassroots contract is not code: the conditions of Section 7 concern who may take part in an act and whose will is required for it, rather than what the code makes an agent do, and they are checked on the text rather than on the code. The digital social contract [12] identifies the contract with code: the contract is a program, and a party can behave only according to it; the smart contract literature [17] does the same on a blockchain.

Self-governance without an external authority. Ostrom [62] documented communities that devise, adopt and enforce their own rules over shared resources, without an owner and without a state administering them, and identified the conditions under which such arrangements endure. A grassroots contract makes the same claim in the digital realm, that a community constitutes

itself, and openness is its formal counterpart: any two people can come to record each other by acts of their own, so any person may start an instance and instances coexist. Commons scholarship concerns how a resource is governed in common, for physical resources [62] and for knowledge and networked production [8, 43]; a grassroots contract governs a relationship rather than a resource, and applies where nothing is held in common.

Platform regulation binds the operator: the General Data Protection Regulation [22] imposes duties on the party that processes personal data, and so presupposes it. Where a community's platform is constituted from its members' own devices [74, 76, 77], there is no operator to bind.

Grassroots systems. The notion of a grassroots protocol is due to [74] and was recast in terms of interleaving and interaction in [51], which also introduced volition-guarded transactions and volitionally grassroots protocols. Grassroots platforms have been specified for social networking [75] and for personal cryptocurrencies [50, 73, 81]. Secure Scuttlebutt [90] is a peer-to-peer social network, started in 2014. Each person is identified by a key pair they generate themselves, so no authority introduces people to the system; each signs every message of their own append-only log; and a person's device replicates the logs of the people they follow, and of those these follow, up to a set distance, so data flows along the social graph, with no consensus and no central directory. Following is a message in the follower's log and takes no consent of the one followed, where befriending under GSG:1 takes the will of both. Its messages are signed and cannot be repudiated, as GSN's messages, signed by their senders, cannot. PGP's Web of Trust [94, 95] connects people who have never met: every person is an introducer, signing the keys of people they know to vouch that each key belongs to its owner, and each person chooses whom they trust as an introducer, with no authority above them; its 1994 guide calls this a grass-roots approach, in which allowing all users to act as trusted introducers for their friends works better than a centralized key server, and its authors call it an early peer-to-peer design and arguably one of the first social networks. Its trust is in a key, and the guide distinguishes trusting a key from trusting the key's owner. GKK:3's introduction, by which Alice makes Carol known to Bob with Bob's consent, is PGP's introducer; a friendship under GSG is trust in the owner, and a breach of an attested clause, of which the contract holds the evidence, ends it (Section 2.2). Three things are added here. The first is politeness, a condition on a protocol's transactions, which Theorem 26 proves sufficient for the protocol to be grassroots, so no proof about runs is needed. The second is the step before the specification: the legal text, and decidable conditions on it under which politeness is established from the acts rather than from the protocol. The third is what a realisation does with a clause of the contract, whether or not the contract provides an act for it, which depends on the same schemas.

Neighbouring instruments. Four instruments resemble a grassroots contract, and each differs from it in what it does rather than in how it is drafted. A *licence* states what one party may do with something another party is entitled to: the General Public License [27] presupposes the copyright it licenses, grants freedoms on conditions, and binds a taker because they would otherwise infringe. A grassroots contract licenses nothing, and its obligations are owed by each agent to every other and derive from no agent's prior entitlement, so a person who has never held any right that the others could infringe is bound as the rest are. *Bylaws* bind the members of an association symmetrically and are adopted on joining, which makes them the closest of the four, but they presuppose an association: an entity with legal personality, organs that act for it, and a membership register. A *code of conduct* states standards of behaviour, and its sanction is exclusion administered by whoever runs the forum, which is the party a grassroots contract does not have; *terms of membership* presuppose the same party, in its capacity as the one who admits.

A *Ricardian contract* [37] is a single document by which an issuer of a financial instrument offers a contract to its holders: readable by people and parsable by programs, signed by the issuer with

the keys that verify the signature included, and identified by its hash, which every transaction record of the instrument carries, so neither party can change the contract unnoticed. It binds an issuer to the holders of one instrument. Under GC (Section 2.6) every person issues their own coins, and each participant signs an undertaking to keep the contract towards all the others (Section 11).